

MODULE –1

CHAPTER 1 - INTRODUCTION

Software and Software Engineering

Software Engineering is the product of two words, software, and engineering.

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentations. Software, when made for a specific requirement is called a **software product**.

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods.

Software Engineering is an engineering branch associated with development of software product using well-defined scientific principles, methods and procedures. The outcome of software engineering is an efficient and reliable software product.

Definitions

IEEE defines software engineering as:

(1) The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.

(2) The study of approaches as in the above statement.

Fritz Bauer, a German computer scientist, defines software engineering as: Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and work efficiently on real machines.

1.1 The Nature of software

Software takes on a dual role. It is a **Product** and at the same time a **Vehicle** (Process) for delivering a product.

As a Product, it produces, manages, modifies, acquires and displays the information–It is an **information transformer** that can be a single bit or a complex multimedia presentation delivered from data acquired from dozens of independent sources.

As a Vehicle (Process), delivers the product. Software acts as the basis for:

- Control of other computer (Operating Systems)

- Communication of Information (Networks)
- Creation and control of other programs (Software Tools and Environment)

Software delivers the most important product of our time called information. It transforms personal data (Individual financial transactions), It manages business information, It provides gateway to worldwide information networks (Internet) and provides the means for acquiring information in all of its forms.

1.1.1 Software

Defining Software : Software is defined as

1. **Instructions** : Programs that when executed provide desired function, features, and performance.
2. **Data structures**: Enable the programs to adequately manipulate information.
3. **Documents**: Descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

Characteristics of software

Software has characteristics that are considerably different than those of hardware:

1) **Software is developed or engineered; it is not manufactured in the Classical Sense.**

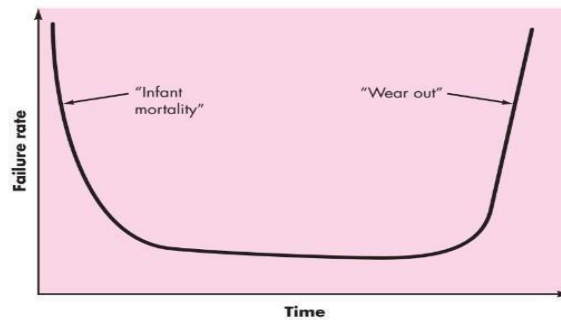
- Although some similarities exist between software development and hardware manufacturing, the two activities are fundamentally different.
- In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce **quality problems** that are nonexistent or easily corrected for software.
- Both the activities are dependent on people, but the relationship between people is totally varying. These two activities require the construction of a "product" but the approaches are different.
- Software costs are concentrated in engineering which means that software projects cannot be managed as if they were manufacturing.

2) **Software doesn't "Wear Out"**

- In early stage of hardware development process the failure rate is very high due to manufacturing defects, but after correcting defects failure rate gets reduced.
- Hardware components suffer from the growing effects of many other environmental factors. Stated simply, the hardware begins to **wear out**.
- Software is not susceptible to the environmental maladies (extreme temperature, dusts and vibrations) that cause hardware to wear out [Fig:1.1]

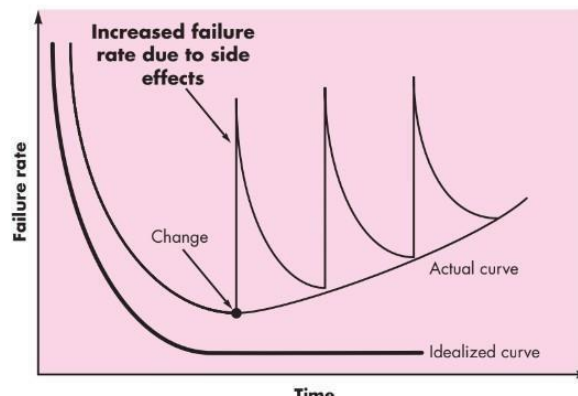
The following figure shows the relationship between failure rate and time.

FIGURE 1.1
Failure curve
for hardware



- When a hardware component wears out, it is replaced by a spare part. There are no software spare parts.
- Every software failure indicates an error in design or in the process through which the design was translated into machine-executable code. Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance. However, the implication is clear—the software doesn't wear out. But it does **deteriorate** (frequent changes in requirement) [Fig:1.2].

FIGURE 1.2
Failure curves
for software



3) Most Software is custom-built rather than being assembled from components:

- A software part should be planned and carried out with the goal that it tends to be reused in various projects (algorithms and data structures).
- Today software industry is trying to make library of reusable components E.g. Software GUI is built using the reusable components such as message windows, pull down menu and many more such components.
- In the hardware world, component reuse is a natural part of the engineering process.

1.1.2 Software Application Domains

Nowadays, seven broad categories of computer software present continuing challenges for software engineers:

1. System Software:

- A collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities). Other system applications (e.g. Operating system components, drivers, networking software,telecommunication processors) process largely indeterminate data.
- In both cases there is heavy interaction with computer hardware, heavy usage by multiple users, scheduling and resource sharing.

2. Application Software:

- Stand-alone programs that solve a specific business need. [Help users to perform specific tasks].
- Application software is used to control business functions in real time (e.g., point-of-sale transaction processing, real-time manufacturing process control).

3. Engineering/Scientific Software:

- It has been characterized by “number crunching” algorithms. (Complex numeric computations).
- Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, and from molecular biology to automated manufacturing. Computer-aided design, system simulation, and other interactive applications have begun to take a real-time and even system software characteristic.

4. Embedded Software:

- It resides within a product or system and is used to implement and control features and functions for the end user and for the system itself.
- Embedded software can perform limited and esoteric functions (e.g., keypad control for a microwave oven) or provide significant function and control capability (e.g., digital functions in an automobile such as fuel control dashboard displays, and braking systems).

5. Product-line Software:

- Designed to provide a specific capability for use by many different customers.
- Product-line software can focus on a limited and esoteric marketplace (e.g., inventory control products) or address mass consumer markets (e.g., word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, and personal and business financial applications).

6. Web Applications:

- It is a client-server computer program that the client runs on the web browser.

- These applications are called “WebApps,” this network-centric software category spans a wide array of applications. In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics.

7. Artificial Intelligence Software:

- These makes use of non-numerical algorithms to solve complex problems. Knowledge based expert systems.
- Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing.

New Software Challenges:

- **Open-world computing:** Creating software to allow machines of all sizes to communicate with each other across vast networks (Distributed computing—wireless networks).
- **Net sourcing:** Architecting simple and sophisticated applications that benefit targeted end-user markets worldwide (the Web as a computing engine).
- **Open Source:** Distributing source code for computing applications so customers can make local modifications easily and reliably (“free” source code open to the computing community).

1.1.3 Legacy Software

- Legacy software is **older programs** that are developed decades ago.
- The quality of legacy software is poor because it has inextensible design, convoluted code, poor and nonexistent documentation, test cases and results that are not achieved.

As time passes legacy systems evolve due to following reasons:

- The software must be adapted to meet the needs of new computing environment or technology.
- The software must be enhanced to implement new business requirements.
- The software must be extended to make it interoperable with more modern systems or database.
- The software must be re-architected to make it viable within a network environment.

1.2 Unique Nature of Web Apps

In the early days of the World Wide Web (1990-95), websites consisted of little more than a set of linked **hypertext files** (HTML) that presented **information using text and limited graphics**. As time passed, **the augmentation of HTML by development tools (e.g., XML, Java)** enabled Web engineers to provide **computing capability along with informational content**.

Web-based systems and applications (WebApps) were born. Today, WebApps have evolved into sophisticated computing tools that not only provide stand-alone function to the end user, but also have been integrated with corporate databases and business applications.

WebApps are one of a number of distinct software categories. Web-based systems and applications “involve a mixture between print publishing and software development, between marketing and computing, between internal communications and external relations, and between art and technology.”

The following attributes are encountered in the vast majority of WebApps.

- **Network intensiveness.** A WebApp resides on a network and must serve the needs of a diverse community of clients. The network may enable worldwide access and communication (i.e., the Internet) or more limited access and communication (e.g., a corporate Intranet).
- **Concurrency.** A large number of users may access the WebApp at one time. In many cases, the patterns of usage among end users will vary greatly.
- **Unpredictable load.** The number of users of the WebApp may vary by orders of magnitude from day to day. One hundred users may show up on Monday; 10,000 may use the system on Thursday.
- **Performance.** WebApp should work effectively in terms of processing speed. If a WebApp user must wait too long (for access, for server-side processing, for client-side formatting and display), he or she may decide to go elsewhere.
- **Availability.** Although expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a 24/7/365 basis.
- **Data driven.** The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end user. In addition, WebApps are commonly used to access information that exists on databases that are not an integral part of the Web-based environment (e.g., e-commerce or financial applications).
- **Content sensitive.** The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.

- **Continuous evolution.** Unlike conventional application software that evolves over a series of planned, chronologically spaced releases, Web applications evolve continuously.
- **Immediacy.** Although immediacy—the compelling need to get software to market quickly—is a characteristic of many application domains, WebApps often exhibit a time- to-market that can be a matter of a few days or weeks.
- **Security.** Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end users who may access the application. In order to protect sensitive content and provide secure modes
- **Aesthetics.** An undeniable part of the appeal of a WebApp is its look and feel. When an application has been designed to market or sell products or ideas, aesthetics may have as much to do with success as technical design.

1.3 Software Engineering -A Layered Technology

In order to build software that is ready to meet the challenges of the 21st century, you must recognize a few simple realities.

- **Problems should be understood before a software solution is developed.**
- **Design is a pivotal Software Engineering activity.**
- **Software should exhibit high quality.**
- **Software should be maintainable**

These simple realities lead to one conclusion. Software in all of its forms and across all of its application domains should be **engineered**.

Software Engineering:
Fritz Bauer defined as:

Software engineering is the establishment and use of sound engineering principles in order to obtain software that is reliable and works efficiently on real machines in economical manner.

IEEE has developed a more comprehensive definition as:

1) Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

2) The study approaches as in (1)

*Software Engineering is a **layered technology**. Software Engineering encompasses a **Process**, **Methods** for managing and engineering software and **tools**.*

Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer. [Fig:1.3]

FIGURE 1.3

Software
engineering
layers



Software engineering is a layered technology. Referring to above Figure, any engineering approach must rest on an organizational commitment to quality.

- The bedrock that supports software engineering is a **quality focus**.
- The foundation for software engineering is the **process** layer. The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software. Process defines a framework that must be established for effective delivery of software engineering technology.
- Software engineering **methods** provide the technical **how-to's** for building software. **Methods** encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.
- Software engineering **tools** provide **automated or semi-automated** support for the process and the methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called computer-aided software engineering is established.

1.4 THE SOFTWARE PROCESS

A **process** is a collection of **activities, actions, and tasks** that are performed when some work product is to be created.

An **activity** strives to achieve a broad objective (e.g. communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.

An **action** encompasses a set of tasks that produce a major work product (e.g., an architectural design model).

A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

A **process framework** establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity. In addition, the process framework encompasses a set of **umbrella activities** that are applicable across the entire software process.

A generic process framework for software engineering encompasses five activities:

- **Communication.** Before any technical work can commence, it is critically important to communicate and collaborate with the customer. The intent is to ‘understand stakeholders’ objectives for the project and to gather requirements that help define software features and functions.
- **Planning.** A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team as it makes the journey. The map—called a **software project plan**—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.
- **Modeling.** Creation of models to help developers and customers understand the requirements and software design.
- **Construction.** This activity combines **code generation and the testing** that is required to uncover errors in the code.
- **Deployment.** The software is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

These five generic framework activities can be used during the development of small, simple programs, the creation of large Web applications, and for the engineering of large, complex computer-based systems.

Software engineering process framework activities are complemented by a number of ***Umbrella Activities***. In general, umbrella activities are applied throughout a software project and help a software team manage and control progress, quality, change, and risk.

Typical umbrella activities include:

- **Software project tracking and control**—allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.
- **Risk management**—assesses risks that may affect the outcome of the project or the quality of the product.
- **Software quality assurance**—defines and conducts the activities required to ensure software quality.
- **Technical reviews**—assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.
- **Measurement**—defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders needs; can be used in conjunction with all other framework and umbrella activities.
- **Software configuration management**—manages the effects of change throughout the software process.
- **Reusability management**—defines criteria for work product reuse and establishes mechanisms to achieve reusable components.
- **Work product preparation and production**—encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

The Software Engineering process is not rigid---It should be agile and adaptable. Therefore, a process adopted for one project might be significantly different than a process adopted for another project.

Among the differences are:

- Overall flow and level of interdependencies among tasks
- Degree to which work tasks are defined within each framework activity
- Degree to which work products are identified and required
- Manner in which quality assurance activities are applied
- Manner in which project tracking and control activities are applied
- Overall degree of detail and rigor of process description

- Degree to which stakeholders are involved in the project
- Level of autonomy given to project team
- Degree to which team organization and roles are prescribed

1.5 THE SOFTWARE ENGINEERING PRACTICE

Generic concepts and principles that apply to framework activities:

1.5.1 The Essence of Practice

- Understand the problem (communication and analysis)
- Plan a solution (software design)
- Carry out the plan (code generation)
- Examine the result for accuracy (testing and quality assurance).

These steps lead to series of questions:

Understand the Problem

- Who are the stakeholders?
- What are the **Unknowns**? What functions and features are required to solve the problem?
- Is it possible to create smaller problems that are easier to understand?
- Can a graphic analysis model be created?

Plan the Solution

- Have you seen similar problems before? Is there existing software that implements these features.
- Has a similar problem been solved? ‘
- Can readily solvable sub problems be defined?
- Can a design model be created? –Effective implementation.

Carry Out the Plan

- Does solution conform to the plan? Is source code traceable to the design model?
- Is each solution component provably, correct? Have design and code be reviewed?

Examine the Result

- Is it possible to test each component part of the solution?
- Does the solution produce results that conform to the data, functions, and features required?

1.5.2 Software General Principles

The dictionary defines the word **principle** as “**an important underlying law or assumption** required in a system of thought.” David Hooker has Proposed seven principles that focus on software Engineering practice.

The First Principle: The Reason It All Exists

A software system exists for one reason: to provide value to its users.

The Second Principle: KISS (Keep It Simple, Stupid!)

Software design is not a haphazard process. There are many factors to consider in any design effort. All design should be as simple as possible, but no simpler.

The Third Principle: Maintain the Vision

A clear vision is essential to the success of a software project. Without one, a project almost unflinchingly ends up being “of two [or more] minds” about itself.

The Fourth Principle: What You Produce, Others Will Consume

Always specify, design, and implement knowing someone else will have to understand what you are doing.

The Fifth Principle: Be Open to the Future A system with a long lifetime has more value. Never design yourself into a corner. Before beginning a software project, be sure the software has a business purpose and that users perceive value in it.

The Sixth Principle: Plan Ahead for Reuse Reuse saves time and effort. Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.

The Seventh principle: Think! Placing clear, complete thought before action almost always produces better results. When you think about something, you are more likely to do it right.

1.6 SOFTWARE MYTHS

Software Myths- beliefs about software and the process used to build it - can be traced to the earliest days of computing. Myths have a number of attributes that have made them insidious. For instance, myths appear to be reasonable statements of fact, they have an intuitive feel, and they are often promulgated by experienced practitioners who “know the score”.

Management Myths:

Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality. Like a drowning person who grasps at a straw, a software manager often grasps at belief in a software myth.

Myth: We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?

Reality:

- The book of standards may very well exist, but is it used?
- Are software practitioners aware of its existence?
- Does it reflect modern software engineering practice?
- Is it complete?
- Is it adaptable?
- Is it streamlined to improve time to delivery while still maintaining a focus on Quality?

In many cases, the answer to this entire question is NO.

Myth: If we get behind schedule, we can add more programmers and catch up

Reality: Software development is not a mechanistic process like manufacturing. "Adding people to a late software project makes it later." At first, this statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort.

People can be added but only in a planned and well-coordinated manner.

Myth: *If we decide to outsource the software project to a third party, I can just relax and let that firm build it.*

Reality: If an organization does not understand how to manage and control software project internally, it will invariably struggle when it out sources' software project.

Customer Myths:

A customer who requests computer software may be a person at the next desk, a technical group down the hall, the marketing /sales department, or an outside company that has requested software under contract. In many cases, the customer believes myths about software because software managers and practitioners do little to correct misinformation. Myths led to false expectations and ultimately, dissatisfaction with the developers.

Myth: *A general statement of objectives is sufficient to begin writing programs - we can fill in details later.*

Reality: Although a comprehensive and stable statement of requirements is not always possible, an ambiguous statement of objectives is a recipe for disaster. Unambiguous requirements are developed only through effective and continuous communication between customer and developer.

Myth: *Project requirements continually change, but change can be easily accommodated because software is flexible.*

Reality: It's true that software requirement change, but the impact of change varies with the time at which it is introduced. When requirement changes are requested early, cost impact is relatively small. However, as time passes, cost impact grows rapidly – resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.

Practitioner's myths.

Myths that are still believed by software practitioners have been fostered by 50 years of programming culture. During the early days of software, programming was viewed as an art form. Old ways and attitudes die hard.

Myth: *Once we write the program and get it to work, our job is done.*

Reality: Someone once said that "the sooner you begin 'writing code', the longer it'll take you to get done." Industry data indicate that between 60 and 80 percent of all effort consumed on software will be consumed after it is delivered to the customer for the first time.

Myth: *Until I get the program "running" I have no way of assessing its quality.*

Reality: One of the most effective software quality assurance mechanisms can be applied from the inception of a project—the formal technical review. Software reviews are a "quality filter" that have been found to be more effective than testing for finding certain classes of software defects.

Myth: *The only deliverable work product for a successful project is the working program.*

Reality: A working program is only one part of a software configuration that includes many elements. Documentation provides a foundation for successful engineering and, more important, guidance for software support.

Myth: *Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.*

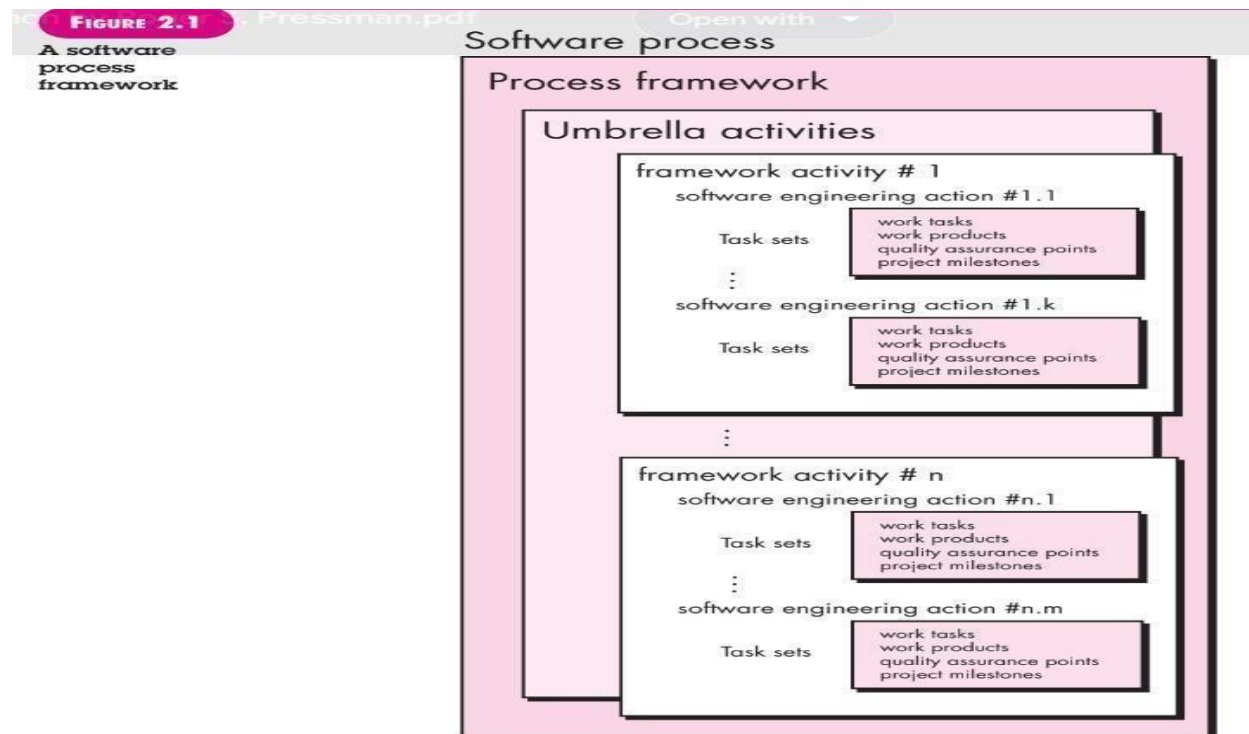
Reality: Software engineering is not about creating documents. It is about creating quality. Better quality leads to reduced rework. And reduced rework results in faster delivery times. Many software professionals recognize the fallacy of the myths just described. Regrettably, habitual attitudes and methods foster poor management and technical practices, even when reality dictates a better approach. Recognition of software realities is the first step toward formulation of practical solutions for software engineering.

CHAPTER 2 - PROCESS MODELS

2.1 A GENERIC PROCESS MODEL

A process was defined as a collection of work activities, actions, and tasks that are performed when some work product is to be created. Each of these activities, actions, and tasks reside within a framework or model that defines their relationship with the process and with one another.

The software process is represented schematically in **Figure 2.1**. Each framework activity is populated by a set of software engineering actions. Each software engineering action is defined by a task set *that identifies the work tasks that are to be completed, the work products that will be produced, the quality assurance points that will be required, and the milestones that will be used to indicate progress.*

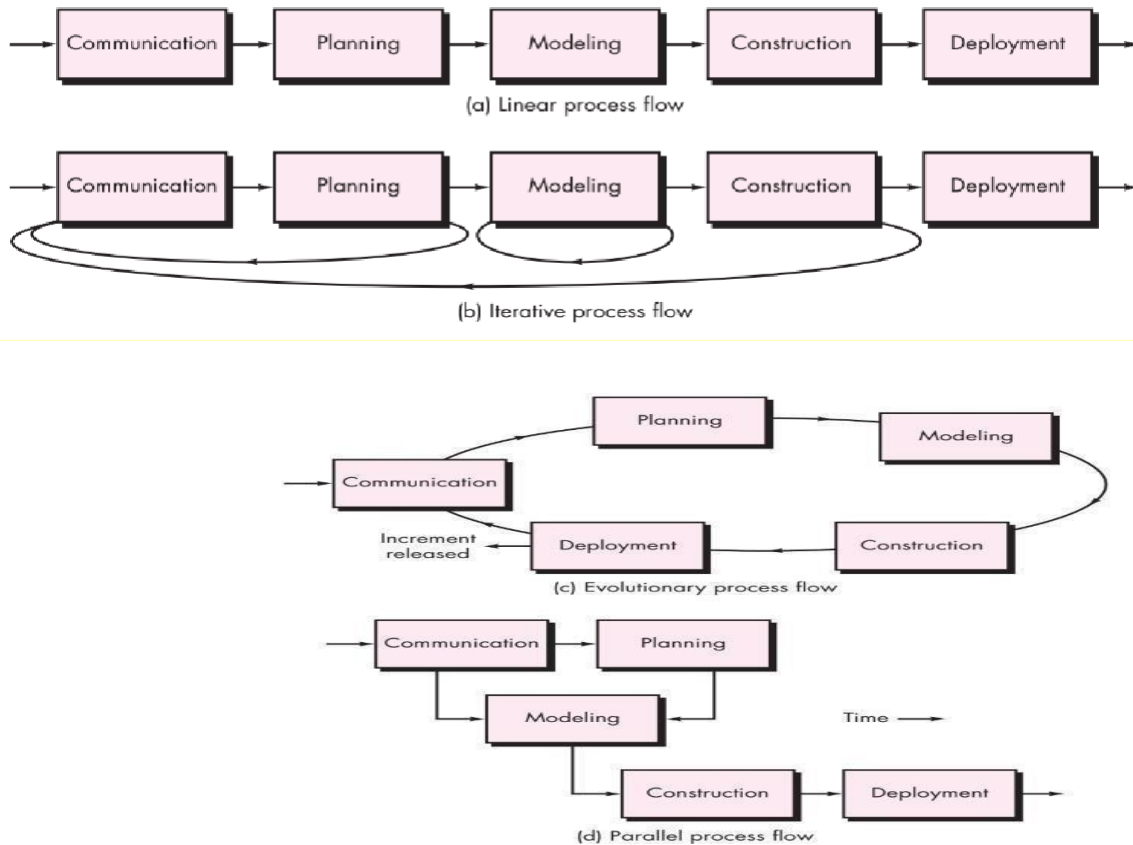


As I discussed in Chapter 1, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction, and deployment**. In addition, a set of umbrella activities—**project tracking and control, risk management, quality**

assurance, configuration management, technical reviews, and others—are applied throughout the process.

The important aspect of software process is “**Process Flow**” which describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time and is illustrated in Figure 2.2.

FIGURE 2.2 Process flow



A linear process [Fig:2.2(a)] flow executes each of the five framework activities in sequence, beginning with communication and culminating with deployment.

An iterative process flow [Fig:2.2(b)] repeats one or more of the activities before proceeding to the next.

An evolutionary process flow [Fig:2.2(c)] executes the activities in a “circular” manner. Each circuit through the five activities leads to a more complete version of the software.

A **parallel process flow [Fig:2.2(d)]** executes one or more activities in parallel with other activities (e.g. modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

2.1.1 Defining a Framework Activity

There are five framework activities, they are

- communication,
- planning,
- modeling,
- construction, and
- deployment.

These five framework activities provide a basic definition of Software Process. These Framework activities provides basic information like ***What actions are appropriate for a framework activity, given the nature of the problem to be solved, the characteristics of the people doing the work, and the stakeholders who are sponsoring the project?***

1. Make contact with stakeholder via telephone.
2. Discuss requirements and take notes.
3. Organize notes into a brief written statement of requirements.
4. E-mail to stakeholder for review and approval.

If the project was considerably more complex with many stakeholders, each with a different set of requirements, the communication activity might have six distinct actions: ***inception, elicitation, elaboration, negotiation, specification, and validation***. Each of these software engineering actions would have many work tasks and a number of distinct work products.

2.1.2 Identifying a Task Set

- Each software engineering action can be represented by a number of different task sets
- Each a collection of software engineering o work tasks,
 - related work products,
 - quality assurance points, and
 - project milestones.
- Choose a task set that best accommodates the needs of the project and the characteristics of software team.
- This implies that a software engineering action can be adapted to the specific needs of the software project and the characteristics of the project team.

2.1.3 Process Patterns

A **process pattern** describes a process-related problem that is encountered during software engineering work, identifies the environment in which the problem has been encountered, and

suggests one or more proven solutions to the problem. Stated in more general terms, a process pattern provides you with a **template** —a consistent method for describing problem solutions within the context of the software process.

Patterns can be defined at any level of abstraction. a pattern might be used to describe a problem (and solution) associated with a complete process model (e.g., prototyping). In other situations, patterns can be used to describe a problem (and solution) associated with a framework activity (e.g., planning) or an action within a framework activity (e.g., project estimating).

Ambler has proposed a template for describing a process pattern:

1. Pattern Name. The pattern is given a meaningful name describing it within the context of the software process (e.g., **Technical Reviews**).

2. Forces. The environment in which the pattern is encountered and the issues that make the problem visible and may affect its solution.

3. Type. The pattern type is specified. **Ambler** suggests three types:

I. Stage pattern—Defines a problem associated with a framework activity for the process. Since a framework activity encompasses multiple actions and work tasks, a stage pattern incorporates multiple task patterns that are relevant to the stage (framework activity).

E.g.: **Establishing Communication**.

This pattern would incorporate the task pattern **Requirements Gathering** and others.

II. Task pattern—Defines a problem associated with a software engineering action or work task and relevant to successful software engineering practice (e.g., **Requirements Gathering**).

III. Phase pattern—Define the sequence of framework activities that occurs within the process, even when the overall flow of activities is iterative in nature. eg: **Spiral Model** or **Prototyping**.

4. Initial context. Describes the conditions under which the pattern applies. Prior to the initiation of the pattern:

(1) What organizational or team-related activities have already occurred?

(2) What is the entry state for the process?

(3) What software engineering information or project information already exists?

5. Problem. The specific problem to be solved by the pattern.

6. Solution. Describes how to implement the pattern successfully. It also describes how software engineering information or project information that is available before the initiation of the pattern is transformed as a consequence of the successful execution of the pattern.

7. **Resulting Context.** Describes the conditions that will result once the pattern has been successfully implemented. Upon completion of the pattern:
 - (1) What organizational or team-related activities must have occurred?
 - (2) What is the exit state for the process?
 - (3) What software engineering information or project information has been developed?
8. **Related Patterns.** Provide a list of all process patterns that are directly related to this one. This may be represented as a hierarchy or in some other diagrammatic form.
9. **Known Uses and Examples.** Indicate the specific instances in which the pattern is applicable.

Conclusion on Process patterns

- Provide an effective mechanism for addressing problems associated with any software process.
- The patterns enable you to develop a hierarchical process description that begins at a high level of abstraction (a phase pattern).
- The description is then refined into a set of stage patterns that describe framework activities
- Once process patterns have been developed, they can be reused for the definition of process variants.

2.2 PROCESS ASSESSMENT AND IMPROVEMENT

The existence of a software process is no guarantee that software will be delivered on time, that it will meet the customer's needs.

Process patterns must be coupled with solid software engineering practice.

Assessment attempts to understand the current state of the software process with the intent on improving it.

A number of different approaches to software process assessment and improvement have been proposed over the past few decades.

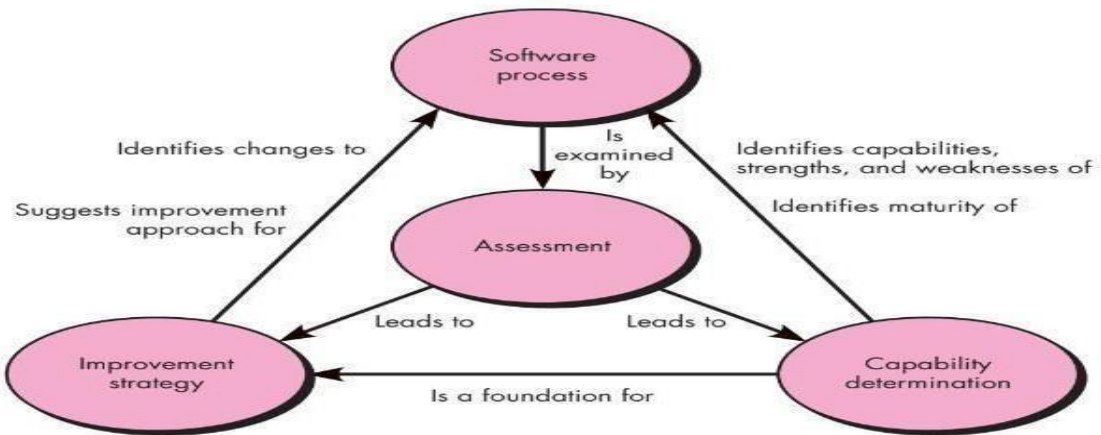
Standard CMMI Assessment Method for Process Improvement (SCAMPI)- Provides a five step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting, and learning. The SCAMPI method uses the SEI CMMI as the basis for assessment.

CMM-Based Appraisal for Internal Process Improvement (CBA IPI)-Provides a diagnostic technique for assessing the relative maturity of a software organization; uses the SEI CMM as the basis for the assessment.

SPICE (ISO/IEC15504)—a standard that defines a set of requirements for software process

assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process.

ISO 9001:2000 for Software—a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies.



Elements of SPI (Software Process Improvement) framework.

2.3 PRESCRIPTIVE PROCESS MODELS

Prescriptive process models were originally proposed to bring order to the chaos (disorder) of software development.

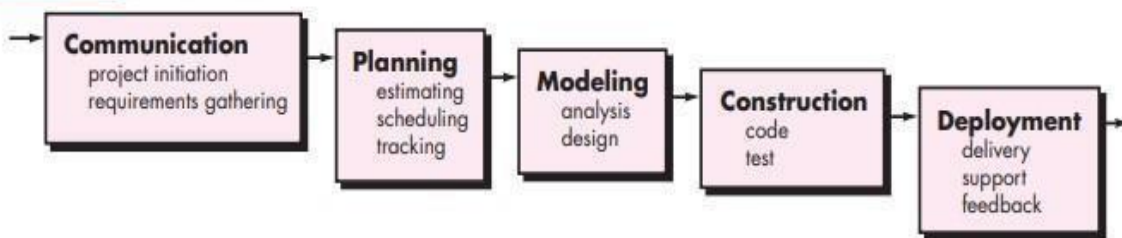
- These models have brought a certain amount of useful structure to software engineering work and have provided a reasonably effective road map for software teams.
- The edge of chaos is defined as “a natural state between order and chaos, a grand compromise between structure and surprise”.
- The edge of chaos can be visualized as an unstable, partially structured state.
- It is unstable because it is constantly attracted to chaos or to absolute order.
- The prescriptive process approach in which order and project consistency are dominant issues.
- “prescriptive” means prescribe a set of process elements
 - framework activities,
 - software engineering actions,
 - tasks,
 - work products,
 - quality assurance, and
 - change control mechanisms for each project.

- Each process model also prescribes a process flow (also called a work flow)—that is, the manner in which the process elements are interrelated to one another.
- All software process models can accommodate the generic framework activities, but each applies a different emphasis to these activities and defines a process flow that invokes each framework activity in a different manner.

The Waterfall Model

The waterfall model, **Fig [2.3]** sometimes called the *classic life cycle*, suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through **planning, modeling, construction, and deployment**.

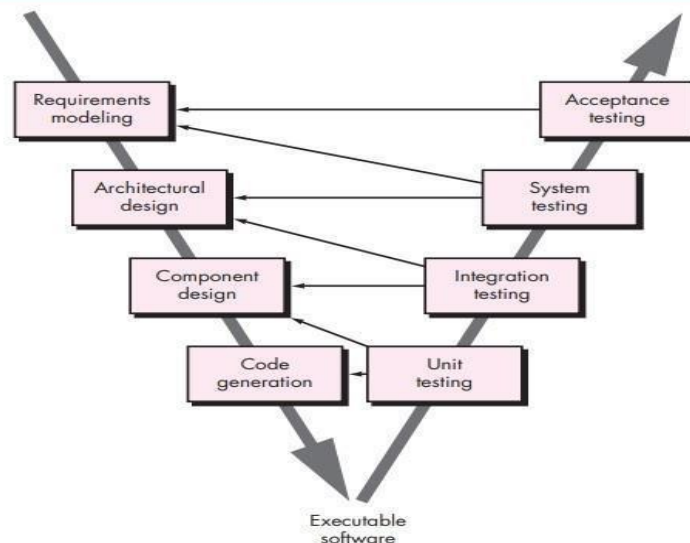
FIGURE 2.3 The waterfall model



V-model

A variation in the representation of the waterfall model is called the **V-model**, represented in the following **Fig:2.4**. The V-model depicts the relationship of quality assurance actions to the actions associated with communication, modeling, and early construction activities.

FIGURE 2.4
The V-model



As a software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution. Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests that validate each of the models created as the team moved down the left side. The V-model provides a way of visualizing how verification and validation actions are applied to earlier engineering work.

The waterfall model is the oldest paradigm for software engineering. The problems that are sometimes encountered when the waterfall model is applied are:

1. Real projects rarely follow the sequential flow that the model proposes. Although a linear model can accommodate iteration, it does so indirectly. As a result, changes can cause confusion as the project team proceeds.
2. It is often difficult for the customer to state all requirements explicitly. The waterfall model requires this and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.
3. The customer must have patience. A working version of the program(s) will not be available until late in the project time span.

This model is suitable whenever limited number of new development efforts and when requirements are well defined and reasonably stable.

2.4 INCREMENTAL PROCESS MODELS

The incremental model delivers a series of releases, called increments, that provide progressively more functionality for the customer as each increment is delivered.

The **incremental model** combines elements of linear and parallel process flows. Referring to **Fig 2.5** The *incremental* model applies linear sequences in a staggered fashion as calendar time progresses. Each linear sequence produces deliverable “increments” of the software in a manner that is similar to the increments produced by an evolutionary process flow.

For example, word-processing software developed using the incremental paradigm might deliver basic file management, editing, and document production functions in the first increment; more sophisticated editing and document production capabilities in the second increment; spelling and grammar checking in the third increment; and advanced page layout capability in the fourth increment.

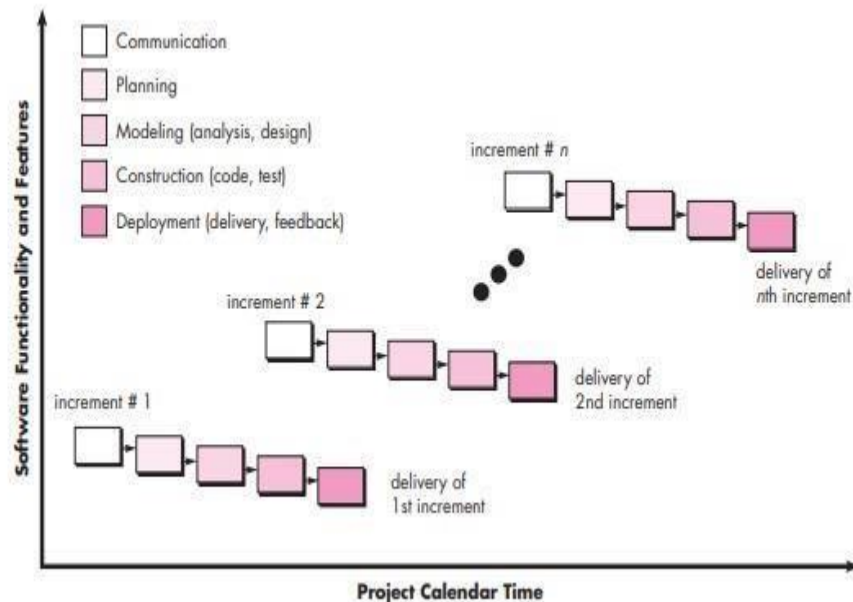
- When an incremental model is used, the first increment is often a **core product**. That is, basic requirements are addressed but many extra features remain undelivered. The core product is used by the customer. As a result of use and/or evaluation, a plan is developed

for the next increment.

- The plan addresses the modification of the core product to better meet the needs of the customer and the delivery of additional features and functionality. This process is repeated following the delivery of each increment, until the complete product is produced.
- Incremental development is particularly useful when **staffing is unavailable** for a complete implementation by the business deadline that has been established for the project. Early increments can be implemented with fewer people. If the core product is well received, then additional staff (if required) can be added to implement the next increment. In addition, increments can be planned to manage **technical risks**.

FIGURE 2.5

The incremental model



2.5 EVOLUTIONARY PROCESS MODELS

Evolutionary models are iterative. They are characterized in a manner that enables you to develop increasingly more complete versions of the software with each iteration. There are two common evolutionary process models.

2.5.1 Prototyping Model

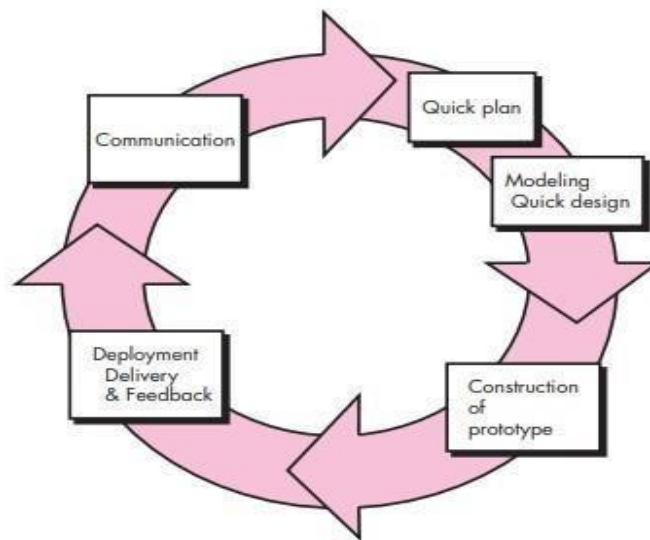
Often, a customer defines a set of general objectives for software, but does not identify detailed requirements for functions and features. In other cases, the developer may be unsure of the

efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction should take. In these, and many other situations, a prototyping paradigm may offer the best approach.

Although prototyping can be used as a stand-alone process model, it is more commonly used as a technique that can be implemented within the context of any one of the process models.

The prototyping paradigm **FIG:2.6** begins with **communication**. You meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. A prototyping iteration is planned quickly, and modeling (in the form of a “quick design”) occurs. A quick design focuses on a representation of those aspects of the software that will be visible to end users.

FIGURE 2.6
The
prototyping
paradigm



The quick design leads to the **construction of a prototype**. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements. Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done.

The prototype serves as a mechanism for identifying software requirements. If a working prototype is to be built, you can make use of existing program fragments or apply tools that enable working programs to be generated quickly. The prototype can serve as “the first system.”

Prototyping can be problematic for the following reasons:

1. Stakeholders see what appears to be a working version of the software, unaware that the prototype is held together randomly, unaware that in the rush to get it working you haven’t considered overall software quality or long-term maintainability.

2. As a software engineer, you often make implementation compromises in order to get a prototype working quickly. An inappropriate operating system or programming language may be used simply because it is available and known; an inefficient algorithm may be implemented simply to demonstrate capability.

Although problems can occur, prototyping can be an effective paradigm for software engineering.

2.5.2 The Spiral Model:

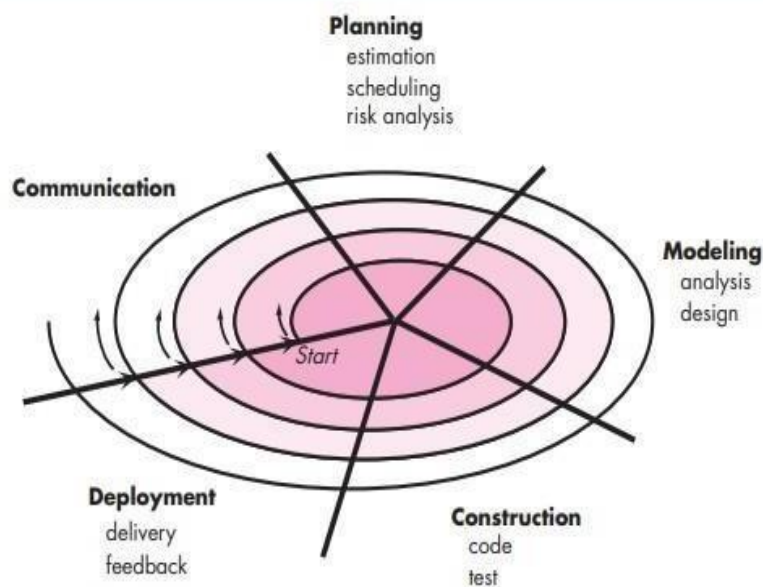
Originally proposed by Barry Boehm, the spiral model is an **evolutionary software process model** that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software. Boehm describes the model in the following manner.

The spiral development model is a **risk-driven process** model generator that is used to guide multi-stakeholder concurrent engineering of software intensive systems. It has two main distinguishing features. One is a **cyclic approach** for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk. The other is a **set of anchor point milestones** for ensuring stakeholder commitment to feasible and mutually satisfactory system solutions.

Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

FIGURE 2.7

A typical spiral model



A spiral model is divided into a set of framework activities defined by the software engineering team.[Fig:2.7] As this evolutionary process begins, the software team performs activities that are implied by a circuit around the spiral in a clockwise direction, beginning at the center.

Risk is considered as each revolution is made. Anchor point milestones are a combination of work products and conditions that are attained along the path of the spiral are noted for each evolutionary pass.

The first circuit around the spiral might result in the development of a product specification; subsequent passes around the spiral might be used to develop a prototype and then progressively more sophisticated versions of the software. Each pass through the planning region results in adjustments to the project plan.

The spiral model can be adapted to apply throughout the life of the computer software. Therefore, the first circuit around the spiral might represent a “**concept development project**” that starts at the core of the spiral and continues for multiple iterations until concept development is complete. The new product will evolve through a number of iterations around the spiral. Later, a circuit around the spiral might be used to represent a “**product enhancement project.**”

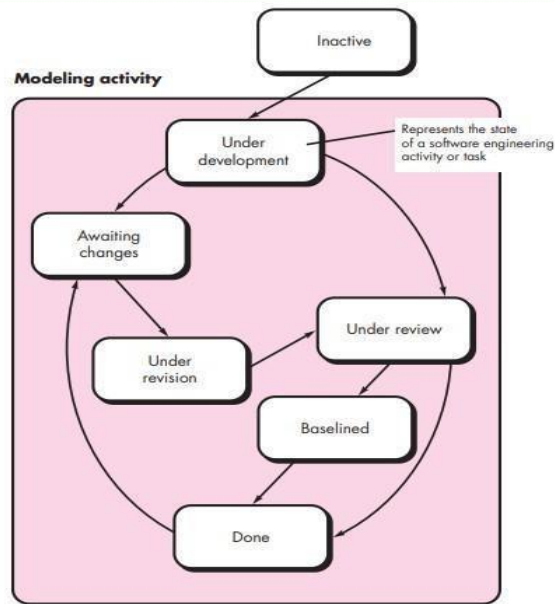
The spiral model is a realistic approach to the development of large-scale systems and software. Because software evolves as the process progresses, the developer and customer better understand and react to risks at each evolutionary level. It maintains the systematic stepwise approach suggested by the classic life cycle but incorporates it into an iterative framework that more realistically reflects the real world.

2.6 CONCURRENT MODELS

The concurrent development model sometimes called as concurrent engineering can be represented schematically as a series of framework activities, actions, tasks and their associated rules.

The Fig 2.8 represents on Software Engineering activity within the modelling activity using a concurrent model approach.

FIGURE 2.8
One element of
the concurrent
process model



The activity modelling may be in any one of the states noted at any given time, similarly other activities, actions or tasks (Communication, Construction) can be represented in analogous manner.

All Software Engineering activities exist concurrently but reside in different states. E.g. Early in a project the communication activity has completed its 1st iteration and exists in the **awaiting changes** state.

The **modelling activity** (which existed in **inactive state**) while initial communication was completed now make a transition into the **under-development state**.

If however, the customer indicates that changes in requirement must be made, the modelling activity moves from under-development state to awaiting changes state.

Concurrent modelling defines a series of events that will trigger transitions from state to state for each of the software engineering activities, actions or tasks.

Concurrent modelling is applicable for all types of software development and provides an accurate picture of the current state of the project.

2.7 SPECIALIZED PROCESS MODELS

2.7.1 Component-Based Development

- The component-based development model incorporates many of the characteristics of the spiral model. It is evolutionary in nature, demanding an iterative approach to the creation of

software. However, the component-based development model constructs applications from prepackaged software components.

- Modeling and construction activities begin with the identification of candidate components. These components can be designed as either conventional software modules or object-oriented classes or packages of classes. Regardless of the technology that is used to create the components.
- The component-based development model incorporates the following steps
 1. Available component-based products are researched and evaluated for the application domain in question.
 2. Component integration issues are considered.
 3. A software architecture is designed to accommodate the components.
 4. Components are integrated into the architecture.
 5. Comprehensive testing is conducted to ensure proper functionality.
- The component-based development model leads to software reuse, and reusability provides software engineers with a number of measurable benefits.

2.7.2 The Formal Methods Model

- The *formal methods model* encompasses a set of activities that leads to formal mathematical specification of computer software.
- Formal methods enable you to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation. A variation on this approach, called ***clean room software engineering***.
- When formal methods are used during development, they provide a mechanism for eliminating many of the problems that are difficult to overcome using other software engineering paradigms. Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily, but through the application of mathematical analysis.
- When formal methods are used during design, they serve as a basis for program verification which discover and correct errors that might otherwise go undetected. The formal methods model offers the promise of defect-free software.

Draw Backs:

- The development of formal models is currently quite time consuming and expensive.
- Because few software developers have the necessary background to apply formal methods, extensive training is required.
- It is difficult to use the models as a communication mechanism for Technically

unsophisticated customers.

2.7.3 Aspect-Oriented Software Development

- AOSD defines “aspects” that express customer concerns that cut across multiple system functions, features, and information. When concerns cut across multiple system functions, features, and information, they are often referred to as crosscutting concerns. Aspectual requirements define those crosscutting concerns that have an impact across the software architecture.
- Aspect-oriented software development (AOSD), often referred to as aspect-oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects.”

Grundy provides further discussion of aspects in the context of what he calls aspect- oriented component engineering (AOCE):

AOCE uses a concept of horizontal slices through vertically-decomposed software components, called “aspects,” to characterize cross-cutting functional and non-functional properties of components.

MODULE 2

CHAPTER 1 - UNDERSTANDING REQUIREMENTS

1.1 REQUIREMENTS ENGINEERING

- *Requirements analysis*, also called *Requirements engineering*, is the process of determining user expectations for a new or modified product.
- Requirements engineering is a major software engineering action that begins during the **communication** activity and continues into the **modeling** activity. It must be adapted to the needs of the process, the project, the product, and the people doing the work.
- Requirements engineering builds a bridge to *design and construction*.

1.1.1 Requirements Engineering Task

Requirements engineering provides the appropriate mechanism for understanding what the customer wants, analyzing need, assessing feasibility, negotiating a reasonable solution, specifying the solution unambiguously, validating the specification, and managing the requirements as they are transformed into an operational system.

It encompasses seven distinct tasks: **inception, elicitation, elaboration, negotiation, specification, validation, and management.**

1. **Inception:** It establishes a basic understanding of the problem, the people who want a solution, the nature of the solution that is desired, and the effectiveness of preliminary communication and collaboration between the other stakeholders and the software team.
2. **Elicitation:** In this stage, proper information is extracted to prepare and document the requirements. It certainly seems simple enough ask the customer, the users, and others what the objectives for the system or product are, what is to be accomplished, how the system or product fits into the needs of the business, and finally, how the system or product is to be used on a day- to-day basis.
 - **Problems of scope.** The boundary of the system is ill-defined or the customers/users specify unnecessary technical detail that may confuse, rather than clarify, overall system objectives.
 - **Problems of understanding.** The customers/users are not completely sure of what is needed, have a poor understanding of the capabilities and limitations of their computing environment, don't have a full understanding of the problem domain, have trouble communicating needs to the system engineer, omit information that is

believed to be “obvious,” specify requirements that conflict with the needs of other customers/users, or specify requirements that are ambiguous or un testable.

- **Problems of volatility.** In this problem, the requirements change from time to time and it is difficult while developing the project.
3. **Elaboration:** The information obtained from the customer during inception and elicitation is expanded and refined during elaboration. This task focuses on developing a refined requirements model that identifies various aspects of software function, behavior, and information. Elaboration is driven by the creation and refinement of user scenarios that describe how the end user (and other actors) will interact with the system.
 4. **Negotiation:** To negotiate the requirements of a system to be developed, it is necessary to identify **conflicts** and to resolve those conflicts. You have to reconcile these conflicts through a process of negotiation. Customers, users, and other stakeholders are asked to **rank requirements** and then discuss conflicts in priority. Using an iterative approach that prioritizes requirements, assesses their cost and risk, and addresses internal conflicts, requirements are eliminated, combined, and/or modified so that each party achieves some measure of **satisfaction**.
 5. **Specification:** The term specification means different things to different people. A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.
 6. **Validation:** Requirements validation **examines** the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work products conform to the standards established for the process, the project, and the product. The primary requirements validation mechanism is the **technical review**. The review team that validates requirements includes software engineers, customers, users, and other stakeholders who examine the specification looking for errors in content or interpretation, areas where clarification may be required, missing information, inconsistencies, conflicting requirements, or unrealistic requirements.
 7. **Requirements management.** Requirements for computer-based systems change, and the desire to change requirements persists throughout the life of the system. It is a set of activities that help the project team to identify, control and track the requirements and changes can be made to the requirements at any time of the ongoing project. Many of these activities are identical to the software configuration management (SCM) techniques. These tasks start with the identification and assign a **unique identifier** to each of the requirement. After finalizing the requirement **traceability table** is developed.

1.2 ESTABLISHING THE GROUNDWORK

There could be many issues to start Requirement Engineering Process:

- Customers or end users may be located in a different city/country.
- Customers do not have a clear idea of the requirements.
- Lack of technical knowledge or customers have limited time to interact with requirement engineer.

Hence by following the below steps we can start a requirement engineering process.

3.2.1 Identifying Stakeholders

- Stakeholder is the one who benefits in a direct or indirect way from the system which is being developed.
- Business operations managers, product managers, marketing people, internal and external customers, end users, consultants, product engineers, software engineers, support and maintenance engineers, and others.
- Each stakeholder has a different view of the system, achieves different benefits when the system is successfully developed, and is open to different risks if the development effort should fail.

3.2.2 Recognizing Multiple Viewpoints

- As many stakeholders exist, they all have different views regarding the system to be developed, hence it is the duty of software engineers to consider all the viewpoints of stakeholders in a way that allows decision makers to choose an internally consistent set of requirements for the system.
- For example, the marketing group is interested in functions and features that will excite the potential market, making the new system easy to sell and End users may want features that are easy to learn and use.

3.2.3 Working toward Collaboration

Collaboration does not necessarily mean that requirements are defined by committee. In many cases, stakeholders collaborate by providing their view of requirements, but a strong “project champion” (e.g., a business manager or a senior technologist) may make the final decision about which requirements make the cut.

3.2.4 Asking the First Questions

Questions asked at the inception of the project should be “**context free**”. The first set of context - free questions focuses on the customer and other stakeholders, the overall project goals and benefits. For example, you might ask:

- Who is behind the request for this work?
- Who will use the solution?
- What will be the economic benefit of a successful solution?
- Is there another source for the solution that you need?

These questions help to identify all stakeholders who will have interest in the software to be built. In addition, the questions identify the measurable benefit of a successful implementation and possible alternatives to custom software development.

The next set of questions enables you to gain a better understanding of the problem and allows the customer to voice his or her perceptions about a solution:

- How would you characterize “good” output that would be generated by a successful solution?
- What problem(s) will this solution address?
- Can you show me (or describe) the business environment in which the solution will be used?
- Will special performance issues or constraints affect the way the solution is approached?

The final set of questions focuses on the effectiveness of the communication activity itself. Gause and Weinberg call these “meta-questions” and propose the following list:

- Are you the right person to answer these questions? Are your answers “official”?
- Are my questions relevant to the problem that you have?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

These questions will help to “break the ice” and initiate the communication that is essential to successful elicitation.

1.3 ELICITING REQUIREMENTS

Requirements Elicitation (also called requirements gathering) combines elements of problem solving, elaboration, negotiation, and specification.

1) Collaborative Requirements Gathering

Many different approaches to collaborative requirements gathering have been proposed. Each makes use of a slightly different scenario, but all apply some variation on the following basic guidelines:

- Meetings are conducted and attended by both software engineers and other stakeholders.
- Rules for preparation and participation are established.
- An agenda is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
- A “facilitator” (can be a customer, a developer, or an outsider) controls the meeting.
- A “definition mechanism” (can be worksheets, flip charts, or wall stickers or an electronic bulletin board, chat room, or virtual forum) is used.

The goal is to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of solution requirements in an atmosphere that is conducive to the accomplishment of the goal.

During inception basic questions and answers establish the scope of the problem and the overall perception of a solution. Out of these initial meetings, the developer and customers write a one- or two-page “product request.”

A meeting place, time, and date are selected; a facilitator is chosen; and attendees from the software team and other stakeholder organizations are invited to participate. The product request is distributed to all attendees before the meeting date.

While reviewing the product request in the days before the meeting, each attendee is asked to make a list of objects that are part of the environment that surrounds the system, other objects that are to be produced by the system, and objects that are used by the system to perform its functions. In addition, each attendee is asked to make another list of services that manipulate or interact with the objects. Finally, lists of constraints (e.g., cost, size, business rules) and performance criteria (e.g., speed, accuracy) are also developed. The attendees are informed that the lists are not expected to be exhaustive but are expected to reflect each person’s perception of the system.

The lists of objects can be pinned to the walls of the room using large sheets of paper, stuck to the walls using adhesive-backed sheets, or written on a wall board. After individual lists are presented in one topic area, the group creates a combined list by eliminating redundant entries, adding any new ideas that come up during the discussion, but not deleting anything.

Collaborative requirements gathering --- Scenario: Campus Event Management App

Step 1: Identify Stakeholders

- **Students:** Potential users who will attend events.
- **Event Organizers:** People who will create and manage events.
- **University Administration:** They might have specific requirements or policies to comply with.
- **Developers:** The team responsible for building the app.

Step 2: Conduct Initial Meetings

- Organize a kickoff meeting with all stakeholders to introduce the project and its goals.
- Use this meeting to explain the importance of gathering accurate requirements and how it will impact the final product.

Step 3: Use Collaborative Techniques

- Brainstorming Session
- Surveys and Questionnaires
- Workshops and User Stories.

Step 4: Document and Validate Requirements

Step 5: Create a Prototype or Wireframes

Step 6: Iterate and Improve

2) Quality function deployment (QFD)

QFD is a quality management technique that translates the needs of the customer into technical requirements for software. QFD “concentrates on maximizing customer satisfaction from the software engineering process”. To accomplish this, QFD emphasizes an understanding of what is valuable to the customer and then deploys these values throughout the engineering process.

QFD identifies three types of requirements:

- **Normal requirements.** The objectives and goals that are stated for a product or system during meetings with the customer. If these requirements are present, the customer is satisfied. Examples of normal requirements might be requested types of graphical displays, specific system functions, and defined levels of performance.
- **Expected requirements.** These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them. Their absence will be a cause for significant dissatisfaction.
 - Examples of expected requirements are: ease of human/machine interaction, overall operational correctness and reliability, and ease of software installation.
- **Exciting requirements.** These features go beyond the customer’s expectations and prove to be very satisfying when present.
For example, the mobile phone with standard features, but the developer adds few additional functionalities like voice searching, multi-touch screen etc. then the customer is more excited about that feature.

Although QFD concepts can be applied across the entire software process, QFD uses customer interviews and observation, surveys, and examination of historical data as raw data for the requirements gathering activity. These data are then translated into a table of requirements—called the customer voice table—that is reviewed with the customer and other stakeholders.

3) Usage Scenarios

- As requirements are gathered, an overall vision of system functions and features begins to materialize.
- However, it is difficult to move into more technical software engineering activities until you understand how these functions and features will be used by different classes of end users.
- To accomplish this, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed. The scenarios, often called **use cases**, provide a description of how the system will be used.

4) Elicitation Work Products

The work products produced as a consequence of requirements elicitation will vary depending on the size of the system or product to be built. For most systems, the work products include:

- A statement of need and feasibility.
- A bounded statement of scope for the system or product.
- A list of customers, users, and other stakeholders who participated in requirements elicitation.
- A description of the system's technical environment.
- A list of requirements and the domain constraints that apply to each.
- A set of usage scenarios that provide insight into the use of the system or product under different operating conditions.
- Any prototypes developed to better define requirements.

Each of these work products is reviewed by all people who have participated in requirements elicitation.

1.4 DEVELOPING USE CASES

Use cases are defined from an actor's point of view. An actor is a role that people (users) or devices play as they interact with the software.

The first step in writing a use case is to define the set of "**actors**" that will be involved in the story. Actors are the different people (or devices) that use the system or product within the context of the function and behavior that is to be described.

Actors represent the roles that people (or devices) play as the system operates. Formally, an actor is anything that communicates with the system or product and that is external to the system itself.

Every actor has one or more goals when using the system. It is important to note that an actor and an end user are not necessarily the same thing. A typical user may play a number of

different roles when using a system, whereas an actor represents a class of external entities (often, but not always, people) that play just one role in the context of the use case. Different people may play the role of each actor.

Because requirements elicitation is an evolutionary activity, not all actors are identified during the first iteration. It is possible to identify primary actors during the first iteration and secondary actors as more is learned about the system.

Primary actors interact to achieve required system function and derive the intended benefit from the system. Secondary actors support the system so that primary actors can do their work. Once actors have been identified, use cases can be developed.

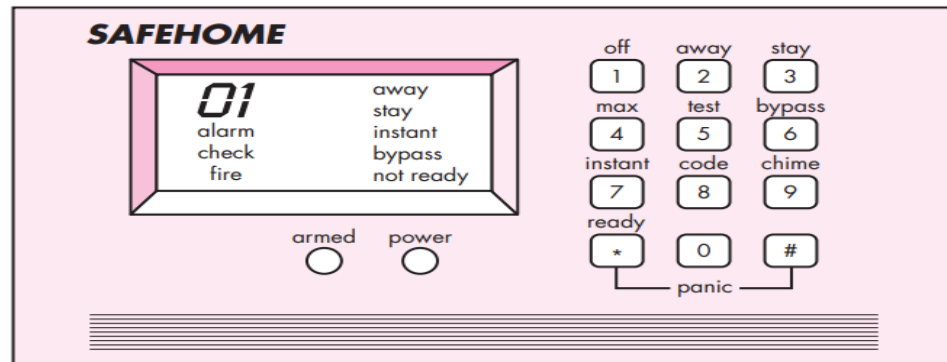
Jacobson suggests a number of questions that should be answered by a use case:

- Who is the primary actor, the secondary actor(s)?
- What are the actor's goals?
- What preconditions should exist before the story begins?
- What main tasks or functions are performed by the actor?
- What exceptions might be considered as the story is described?
- What variations in the actor's interaction are possible?
- What system information will the actor acquire, produce, or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Basic SafeHome requirements, we define four **actors**: homeowner (a user), setup manager (likely the same person as homeowner, but playing a different role), sensors (devices attached to the system), and the monitoring and response subsystem (the central station that monitors the SafeHome home security function).

For the purposes of this example, we consider only the homeowner actor. The homeowner actor interacts with the home security function in a number of different ways using either the alarm control panel or a PC:

Considering the situation in which the **homeowner uses the control panel**, the **basic use case for system activation follows**:

FIGURE 5.1**SafeHome
control panel**

1. The homeowner observes the SafeHome control panel as shown in Fig 5.1 to determine if the system is ready for input. If the system is not ready, a not ready message is displayed on the LCD display.
2. The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect and reset itself for additional input. If the password is correct, the control panel awaits further action.
3. The homeowner select the keys in stay or away to activate the system.
 - ✓ Stay activates only perimeter sensors (inside motion detecting sensors are deactivated).
 - ✓ Away activates all sensors.
4. When activation occurs, a red alarm light can be observed by the homeowner. The basic use case presents a high-level story that describes the interaction between the actor and the system.

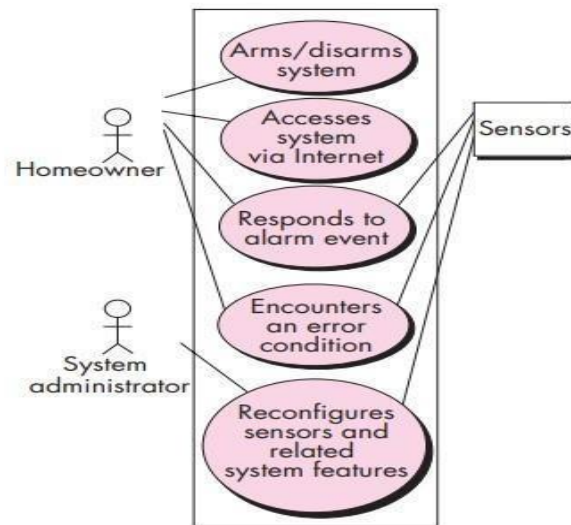
Cockburn provides Detailed Description of Use Case:

USE CASE	Initiate Monitoring
Primary actor	Home Owner
Goal in context	To set the system to monitor sensors when the homeowner leaves the house or remains inside
Preconditions:	System has been programmed for a password and to recognize various sensors.
Trigger:	The homeowner decides to “set” the system, i.e., to turn on the alarm functions

Scenario:	1. Homeowner: observes control panel 2. Homeowner: enters password 3. Homeowner: selects “stay” or “away” 4. Homeowner: observes read alarm light to indicate that SafeHome has been armed
Exceptions:	1. Control panel is not ready: homeowner checks all sensors to determine which are open; closes them. 2. Password is incorrect (control panel beeps once): homeowner reenters correct password. 3. Password not recognized: monitoring and response subsystem must be contacted to reprogram password. 4. Stay is selected: control panel beeps twice and a stay light is lit; perimeter sensors are activated. 5. Away is selected: control panel beeps three times and an away light is lit; all sensors are activated.
When available:	First increment
Priority:	Essential, must be implemented
Frequency of use:	Many times per day
Channel to actor	Via control panel interface
Secondary actors:	Support technician, sensors
Channels to secondary actors:	1. Support technician: phone line 2. Sensors: hardwired and radio frequency interfaces
Open issues:	1. Should there be a way to activate the system without the use of a password or with an abbreviated password? 2. Should the control panel display additional text messages? 3. How much time does the homeowner have to enter the password from the time the first key is pressed? 4. Is there a way to deactivate the system before it actually activates?

FIGURE 5.2

UML use case diagram for *SafeHome* home security function



Note:

The basic use case presents a high-level story that describes the interaction between the actor and the system.

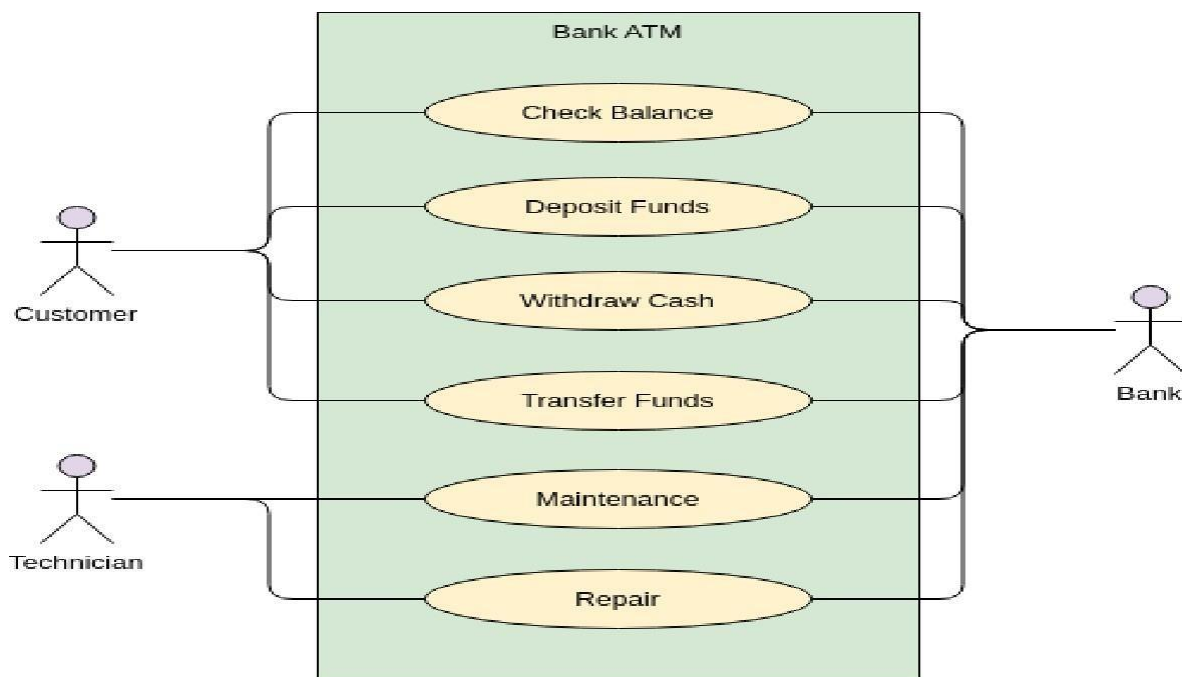
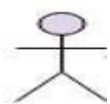
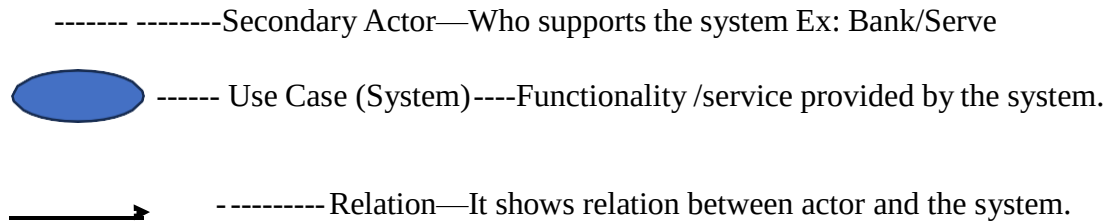


Fig 3.1-Use Case Diagram



-----Primary Actors-Who initiate/interact directly Ex: Customer and Technician



1.5 BUILDING THE REQUIREMENT MODEL

The intent of the analysis model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system. The analysis model is a snapshot of requirements at any given time.

3.5.1 Elements of the Requirements Model

The specific elements of the requirements model are dictated by the analysis modeling method that is to be used. However, a set of generic elements is common to most requirements models.

Scenario Based Elements: The system is described from the user's point of view using a scenario-based approach **Ex: Use Case diagrams and Activity diagrams.**

Class-based Elements: Each usage scenario implies a set of **objects** that are manipulated as an actor interacts with the system. These objects are categorized into **classes**—a collection of things that have similar attributes and common behaviors(operations). **Ex: Class diagram, Collaboration diagram.**

Behavioral Elements: In Software Engineering, the Behavioral Elements Model is a concept used to describe the dynamic behavior of a software system. It focuses on how the system's components interact with each other and with external entities to achieve specific tasks or behaviors. The state diagram is one method for representing the behavior of a system by depicting its states and the events that cause the system to change state. This model indicates how the software will respond on occurrence of external event. **Ex: State diagram and Sequential Diagram.**

Flow-Oriented elements: Information is transformed as it flows through a computer- based system. The system accepts input in a variety of forms, applies functions to transform it, and produces output in a variety of forms. **Ex: Data-Flow diagram, Control Flow diagram.**

1.5.1 Analysis Patterns

Analysis patterns suggest solutions (e.g., a class, a function, a behavior) within the application domain that can be reused when modeling many applications.

Geyer-Schulz and Hahsler suggest two benefits that can be associated with the use of analysis patterns:

First, analysis patterns speed up the development of abstract analysis models that capture the main requirements of the concrete problem by providing reusable analysis models with examples as well as a description of advantages and limitations.

Second, analysis patterns facilitate the transformation of the analysis model into a design model by suggesting design patterns and reliable solutions for common problems. Analysis patterns are integrated into the analysis model by reference to the pattern name. They are also stored in a repository so that requirements engineers can use

1.6 NEGOTIATING REQUIREMENTS

The intent of negotiation is to develop a **project plan** that meets stakeholder needs while at the same time reflecting the real-world constraints (e.g., time, people, budget) that have been placed on the software team. The best negotiations strive for a “**win-win**” result. That is, stakeholders win by getting the system or product that satisfies the majority of their needs and you win by working to realistic and achievable budgets and deadlines.

Boehm defines a set of negotiation activities at the beginning of each software process iteration. Rather than a single customer communication activity, the following activities are defined:

1. Identification of the system or subsystem’s key stakeholders.
2. Determination of the stakeholders’ “win conditions.”
3. Negotiation of the stakeholders’ win conditions to reconcile them into a set of win-win conditions for all concerned.

Successful completion of these initial steps achieves a win-win result, which becomes the key criterion for proceeding to subsequent software engineering activities.

1.7 VALIDATING REQUIREMENTS

As each element of the requirements model is created, it is examined for inconsistency, omissions, and ambiguity. The requirements represented by the model are prioritized by the

stakeholders and grouped within requirements packages that will be implemented as software increments.

A review of the requirements model addresses the following questions:

- Is each requirement consistent with the overall objectives for the system/product?
- Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?
- Is each requirement achievable in the technical environment that will house the system or product?
- Is each requirement testable, once implemented?
- Does the requirements model properly reflect the information, function, and behavior of the system to be built?
- Has the requirements model been “partitioned” in a way that exposes progressively more detailed information about the system?
- Have requirements patterns been used to simplify the requirements model?
- Have all patterns been properly validated? Are all patterns consistent with customer requirements?

These and other questions should be asked and answered to ensure that the requirements model is an accurate reflection of stakeholder needs and that it provides a solid foundation for design

CHAPTER 2 - REQUIREMENT MODELLING SCENARIOS, INFORMATION AND ANALYSIS CLASSES

2.1 REQUIREMENT ANALYSIS

- Requirements analysis results in the specification of software's operational characteristics, indicates software's interface with other system elements, and establishes constraints that software must meet.
- Requirements analysis allows you to elaborate on basic requirements established during the inception, elicitation, and negotiation tasks that are part of requirements engineering.
- The requirements modeling action results in one or more of the following types of models:
 - ***Scenario-based models*** of requirements from the point of view of various system “actors”
 - ***Data models*** that depict the information domain for the problem.
 - ***Class-oriented models*** that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements.

- **Flow-oriented models** that represent the functional elements of the system and how they transform data as it moves through the system.
- **Behavioral models** that depict how the software behaves as a consequence of external “events.”
- The intent of the analysis model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system. **The analysis model is a snapshot of requirements at any given time.**
- These models provide a software designer with information that can be translated to architectural, interface, and component-level designs. Finally, the requirements model provides the developer and the customer with the means to assess quality once software is built.
- Throughout requirements modeling, primary focus is on **what, not how**. What user interaction occurs in a particular circumstance, what objects does the system manipulate, what functions must the system perform, what behaviors does the system exhibit, what interfaces are defined, and what constraints apply?

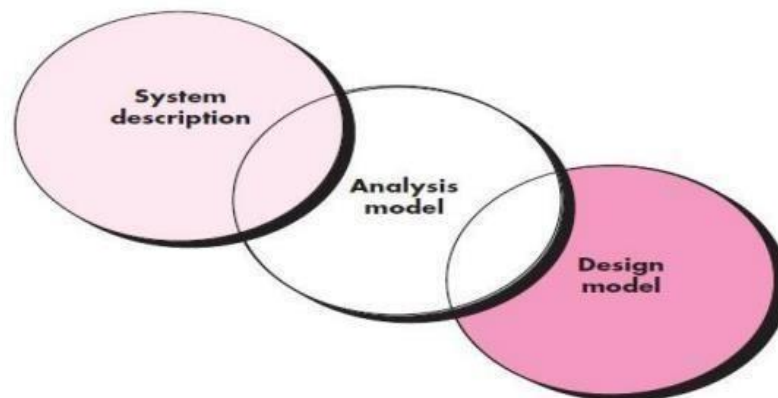


Fig : The requirements model as a bridge between the system description and the design model

The requirements model must achieve three primary objectives:

- (1) To describe what the customer requires,
 - (2) to establish a basis for the creation of a software design, and
 - (3) to define a set of requirements that can be validated once the software is built.
- The analysis model bridges the gap between a system-level description that describes overall system or business functionality as it is achieved by applying software, hardware, data, human, and other system elements and a software design that describes the software’s application architecture, user interface, and component-level structure.

2.1.1 Analysis Rules of Thumb

Arlow and Neustadt suggest a number of worthwhile rules of thumb that should be followed when creating the analysis model:

- ***The model should focus on requirements that are visible within the problem or business domain.*** The level of abstraction should be relatively high.
- ***Each element of the requirements model should add to an overall understanding of software requirements and provide insight into the information domain, function, and behavior of the system.***
- ***Delay consideration of infrastructure and other nonfunctional models until design.*** That is, a database may be required, but the classes necessary to implement it, the functions required to access it, and the behavior that will be exhibited as it is used should be considered only after problem domain analysis has been completed.
- ***Minimize coupling throughout the system.*** It is important to represent relationships between classes and functions. However, if the level of “interconnectedness” is extremely high, effort should be made to reduce it.
- ***Be certain that the requirements model provides value to all stakeholders.*** Each constituency has its own use for the model
- ***Keep the model as simple as it can be.*** Don’t create additional diagrams when they add no new information. Don’t use complex notational forms, when a simple list will do.

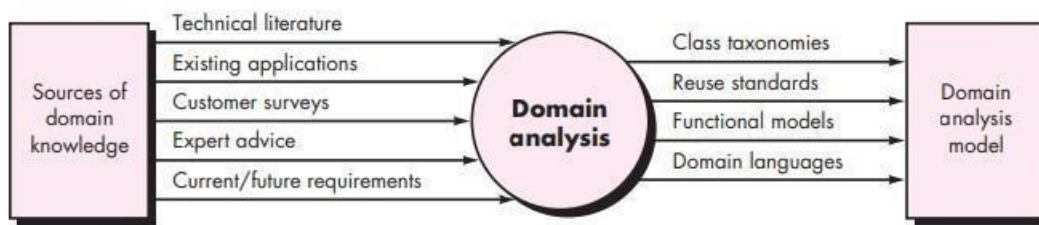
2.1.2 Domain Analysis

Domain analysis doesn’t look at a specific application, but rather at the domain in which the application resides.

The “specific application domain” can range from avionics to banking, from multimedia video games to software embedded within medical devices.

The goal of domain analysis is straightforward: to identify common problem-solving elements that are applicable to all applications within the domain, to find or create those analysis classes and/or analysis patterns that are broadly applicable so that they may be reused.

FIGURE 6.2 Input and output for domain analysis



2.1.3 Requirements Modeling Approaches

- One view of requirements modeling, called **structured analysis**, considers data and the processes that transform the data as separate entities. Data objects are modeled in a way that defines their *attributes and relationships*.
- A second approach to analysis modeling, called **object-oriented analysis**, focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements. UML and the Unified Process are predominantly object oriented.

Each element of the requirements model is represented in following figure presents the problem from a different point of view.

- **Scenario-based elements** depict how the user interacts with the system and the specific sequence of activities that occur as the software is used.
- **Class-based elements model** the objects that the system will manipulate, the operations that will be applied to the objects to effect the manipulation, relationships between the objects, and the collaborations that occur between the classes that are defined.
- **Behavioral elements** depict how external events change the state of the system or the classes that reside within it.
- **Flow-oriented elements** represent the system as an information transform, depicting how data objects are transformed as they flow through various system functions.

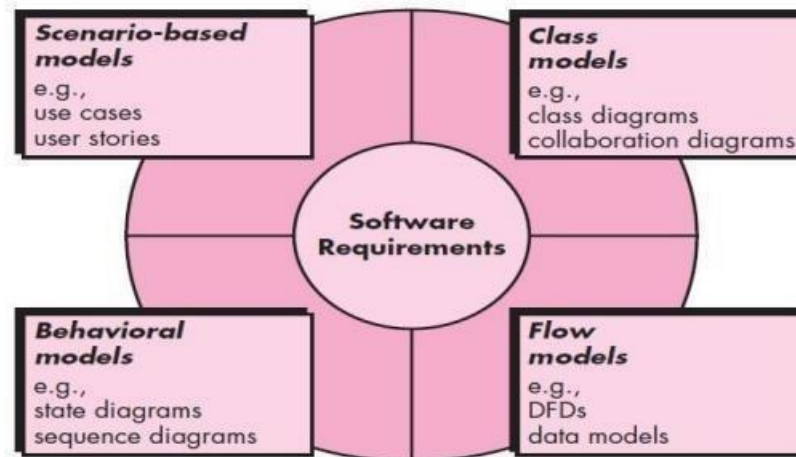


Fig : Elements of the analysis model

2.2 SCENARIO-BASED MODELLING

Scenario-based elements depict how the user interacts with the system and the specific sequence of activities that occur as the software is used.

2.2.1 Creating a Preliminary Use Case

Alistair Cockburn characterizes a use case as a “contract for behavior”, the “contract” defines the way in which an actor uses a computer-based system to accomplish some goal. In essence, a use case captures the interactions that occur between producers and consumers of information and the system itself.

A use case describes a specific usage scenario in straightforward language from the point of view of a defined actor.

These are the questions that must be answered if use cases are to provide value as a requirement modeling tool.

- what to write about,
- how much to write about it,
- how detailed to make your description, and
- how to organize the description? To begin developing a set of use cases, list the functions or activities performed by a specific actor.

1) Creating a Preliminary Use Case:

Home Surveillance Functions:

Actor: Home Owner

- Select Camera View
- Request Thumbnails from all cameras
- Display camera view in PC window
- Selectively record camera output
- Replay Camera output
- Access camera surveillance via Internet

Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)

Actor: homeowner

- The homeowner logs onto the **SafeHome** Products website.
- The homeowner enters his or her user ID.
- The homeowner enters two passwords (each at least eight characters in length).
- The system displays all major function buttons.
- The homeowner selects the “surveillance” from the major function buttons.
- The homeowner selects “pick a camera.”
- The system displays the floor plan of the house.
- The homeowner selects a camera icon from the floor plan.
- The homeowner selects the “view” button.
- The system displays a viewing window that is identified by the camera ID.
The system displays video output within the viewing window at one frame per second.

*****Use cases of this type are sometimes referred to as primary scenarios***.**

2.2.2 Refining a Preliminary Use Case

Each step in the primary scenario is evaluated by asking the following questions:

- Can the actor take some other action at this point?
- Is it possible that the actor will encounter some error condition at this point? If so, what might it be?
- Is it possible that the actor will encounter some other behavior at this point (e.g., behavior that is invoked by some event outside the actor’s control)? If so, what might it be?

Answers to these questions result in the creation of a set of **secondary scenarios**.

1. View thumbnail snapshots for all cameras.
2. No floor plan configured for this house.
3. Alarm condition encountered.

Cockburn recommends using a “**brainstorming**” session to derive a reasonably complete set of exceptions for each use case. In addition to the three generic questions suggested earlier in this section, the following issues should also be explored:

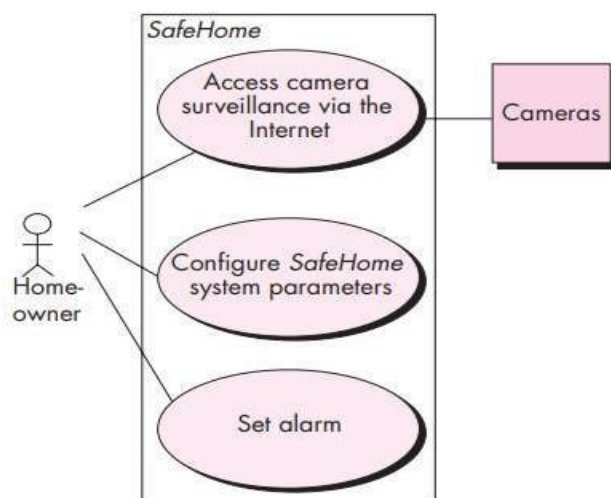
- Are there cases in which some “validation function” occurs during this use case? This implies that validation function is invoked and a potential error condition might occur.
- Are there cases in which a supporting function (or actor) will fail to respond appropriately? For example, a user action awaits a response but the function that is to respond times out.
- Can poor system performance result in unexpected or improper user actions?

2.2.3 Writing a Formal Use Case

When a use case involves a critical activity or describes a complex set of steps with a significant number of exceptions, a more formal approach may be desirable.

The typical outline for formal use cases can be in following manner:

- The **goal in context** identifies the overall scope of the use case.
- The **precondition** describes what is known to be true before the use case is initiated.
- The **trigger** identifies the event or condition that “gets the use case started”
- The **scenario** lists the specific actions that are required by the actor and the appropriate system responses.
- **Exceptions** identify the situations uncovered as the preliminary use case is refined



Preliminary Use-Case diagram for the Safe-Home Systems

However, **scenario-based modeling** is appropriate for a significant majority of all situations that you will encounter as a software engineer.

2.3 UML MODELS THAT SUPPLEMENT THE USE CASE

2.3.1 Developing an Activity Diagram

The UML activity diagram supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario. Similar to the flowchart,

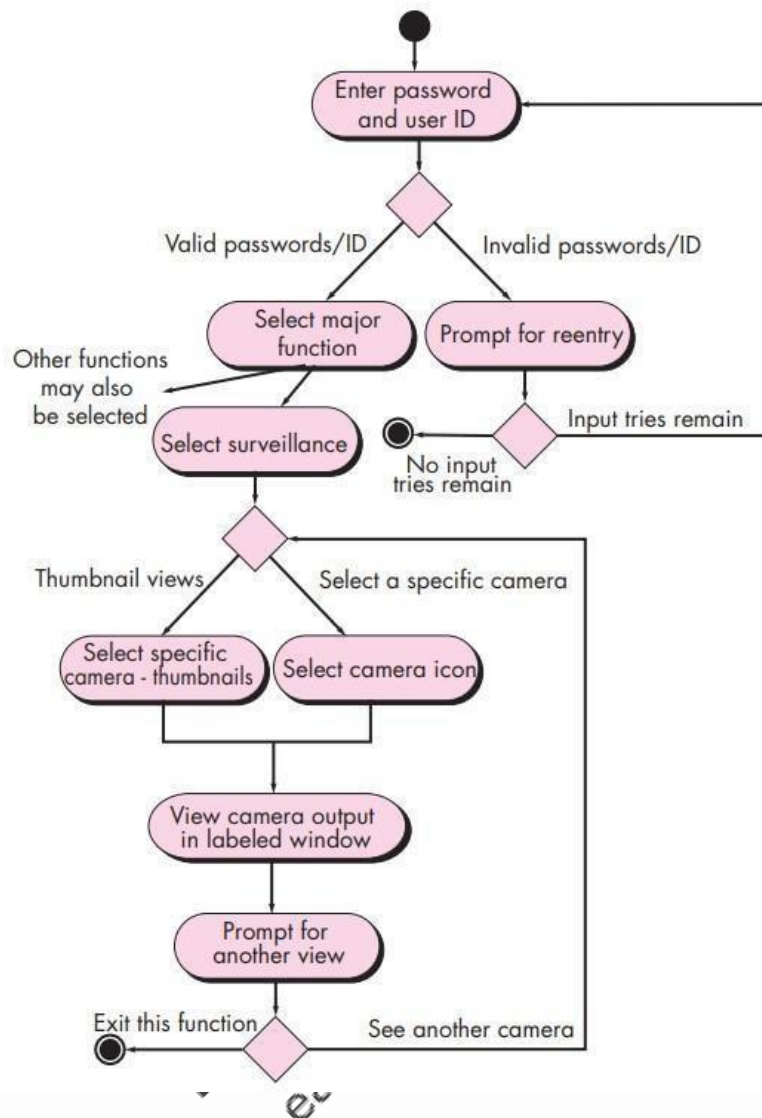
An activity diagram uses:

- **Rounded rectangles** to imply a specific system function
- **Arrows** to represent flow through the system
- **Decision diamonds** to depict a branching decision.
- **Solid horizontal lines** to indicate that parallel activities are occurring.

A UML activity diagram represents the actions and decisions that occur as some function is performed.

FIGURE 6.5

Activity diagram for Access camera surveillance via the Internet—display camera views function.



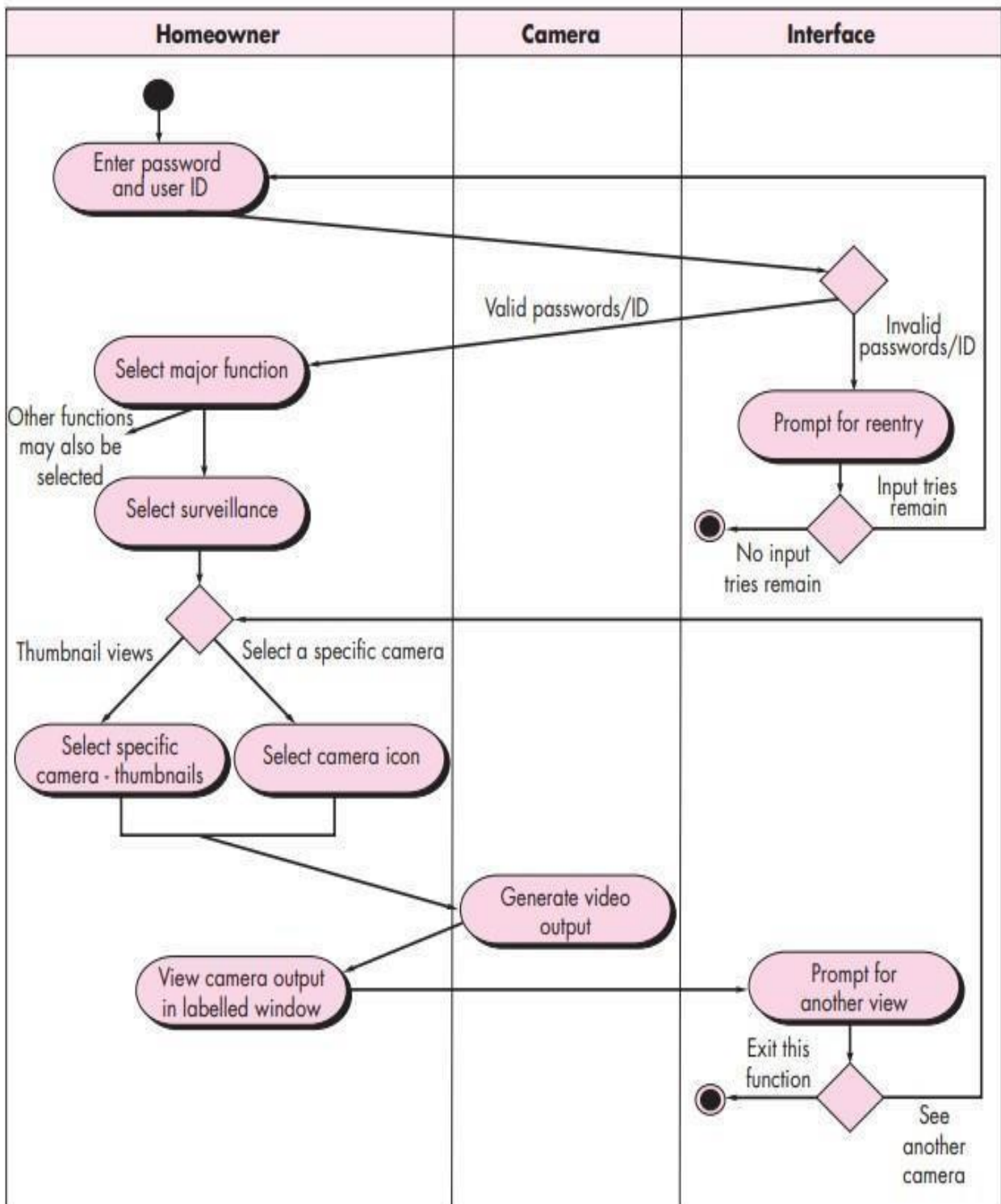
2.3.2 Swimlane Diagrams

The UML swimlane diagram is a useful variation of the activity diagram and allows you to represent the flow of activities described by the use case and at the same time indicate which actor or analysis class has responsibility for the action described by an activity rectangle.

Responsibilities are represented as parallel segments that divide the diagram vertically, like the lanes in a swimming pool.

Three analysis classes—Homeowner, Camera, and Interface—have direct or indirect responsibilities in the context of the activity diagram represented in Figure 6.5.

FIGURE 6.6 Swimlane diagram for Access camera surveillance via the Internet—display camera views function



2.4 DATA MODELING CONCEPTS

Data modeling is the process of documenting a complex software system design as an easily understood diagram, using text and symbols to represent the way data needs to flow.

The diagram can be used as a blueprint for the construction of new software or for re-engineering a legacy application.

The most widely used data Model by the Software engineers is **Entity-Relationship Diagram (ERD)**, it addresses the issues and represents all data objects that are entered, stored, transformed, and produced within an application.

2.4.1 Data Objects

A data object is a representation of composite information that must be understood by software. Therefore width (a single value) would not be a valid data object, but dimensions (incorporating height, width, and depth) could be defined as an object.

A data object can be an **external entity** (e.g., anything that produces or consumes information), a **thing** (e.g., a report or a display), **an occurrence** (e.g., a telephone call) or event (e.g., an alarm), **a role** (e.g., salesperson), **an organizational unit** (e.g., accounting department), **a place** (e.g., a warehouse), or a **structure** (e.g., a file).

For example, **a person or a car** can be viewed as a data object in the sense that either can be defined in terms of a set of attributes. The description of the data object incorporates the data object and all of its attributes.

A data object encapsulates data only—there is no reference within a data object to operations that act on the data. Therefore, the data object can be represented as a table as shown in following table. The headings in the table reflect attributes of the object.

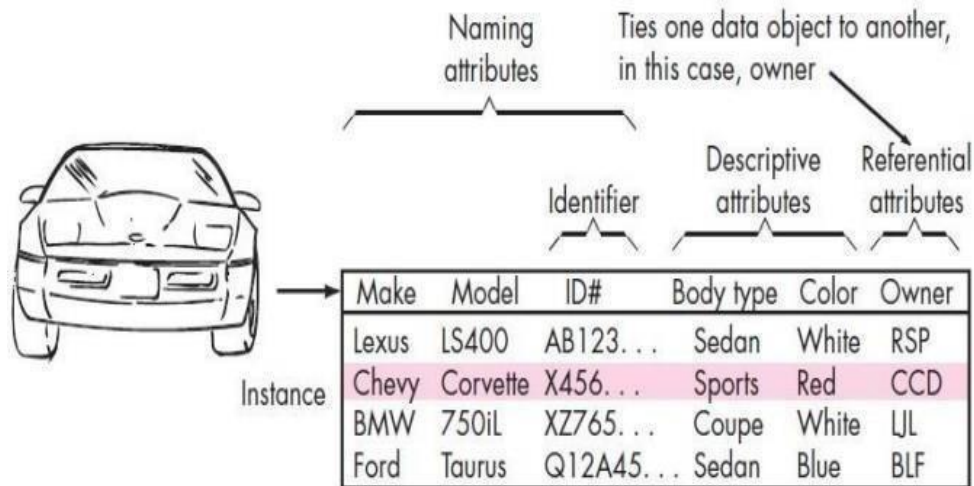


Fig : Tabular representation of data objects

2.4.2 Data Attributes

Data attributes define the properties of a data object and take on one of three different characteristics. They can be used to (1) name an instance of the data object, (2) describe the instance, or (3) make reference to another instance in another table.

2.4.3 Relationships

Data objects are connected to one another in different ways. Consider the two data objects, **person** and **car**. These objects can be represented using the following simple notation and relationships are 1) A person owns a car, 2) A person is insured to drive a car.

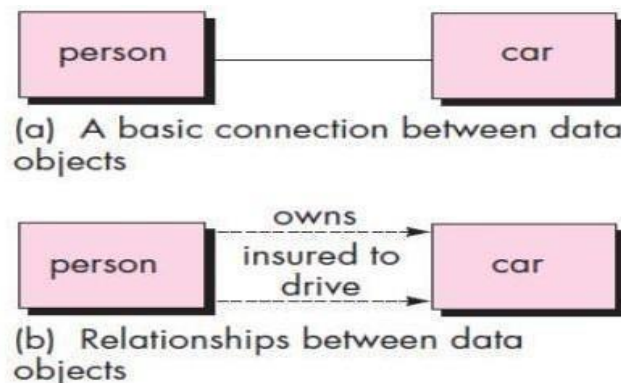


Fig : Relationships between data objects

2.5 CLASS-BASED MODELING

Class-based modeling represents the objects that the system will manipulate, the operations that will be applied to the objects to effect the manipulation, relationships between the objects, and the collaborations that occur between the classes that are defined.

The elements of a class-based model include classes and objects, attributes, operations, class responsibility collaborator (CRC) models, collaboration diagrams, and packages.

2.5.1 Identifying Analysis Classes

We can begin to identify classes by examining the usage scenarios developed as part of the requirements model and performing a “**grammatical parse**” on the use cases developed for the system to be built.

Analysis classes manifest themselves in one of the following ways:

- **External entities** (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- **Things** (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
- **Occurrences or events** (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- **Roles** (e.g., manager, engineer, salesperson) played by people who interact with the system.
- **Organizational units** (e.g., division, group, team) that are relevant to an application.
- **Places** (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- **Structures** (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

Coad and Yourdon suggest six selection characteristics that should be used as you consider each potential class for inclusion in the **analysis model**:

1. Retained information. The potential class will be useful during analysis only if information about it must be remembered so that the system can function.

2. Needed services. The potential class must have a set of identifiable operations that can change the value of its attributes in some way.

3. Multiple attributes. During requirement analysis, the focus should be on “major” information; a class with a single attribute may, in fact, be useful during design, but is probably better represented as an attribute of another class during the analysis activity.

4. Common attributes. A set of attributes can be defined for the potential class and these attributes apply to all instances of the class.

5. Common operations. A set of operations can be defined for the potential class and these operations apply to all instances of the class.

6. Essential requirements. External entities that appear in the problem space and produce or consume information essential to the operation of any solution for the system will almost always be defined as classes in the requirements model.

2.5.2 Specifying Attributes

Attributes describe a class that has been selected for inclusion in the requirements model. In essence, it is the attributes that define the class—that clarify what is meant by the class in the context of the problem space.

To develop a meaningful set of attributes for an analysis class, you should study each use case and select those “things” that reasonably “belong” to the class.

2.5.3 Defining Operations

Operations define the behavior of an object. Although many different types of operations exist, they can generally be divided into four broad categories: (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting), (2) operations that perform a computation, (3) operations that inquire about the state of an object, and (4) operations that monitor an object for the occurrence of a controlling event.

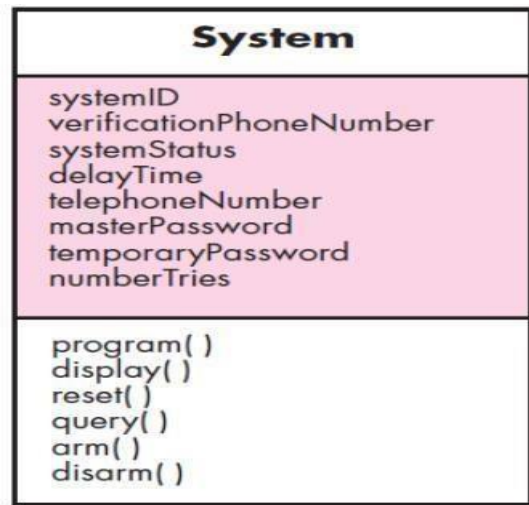
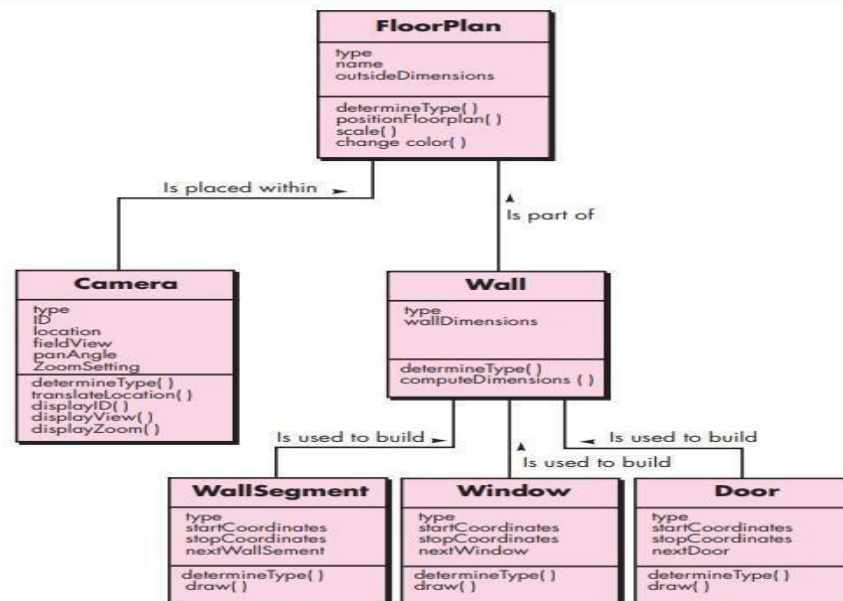


Fig : Class diagram for the system class

FIGURE 6.10

Class diagram for FloorPlan (see sidebar discussion)



2.5.4 Class-Responsibility-Collaborator (CRC) Modeling

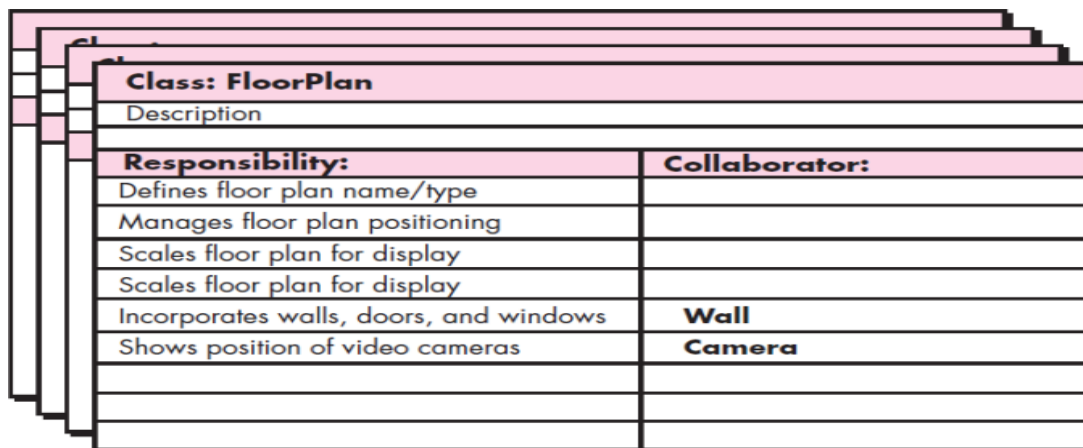
Class-responsibility-collaborator (CRC) modeling provides a simple means for identifying and organizing the classes that are relevant to system or product requirements.

Ambler describes CRC modeling in the following way:

A CRC model is really a collection of standard **index cards** that represent classes.

The cards are divided into **three sections**. Along the top of the card, you write the name of the **class**. In the body of the card, you list the class responsibilities on the left and the collaborators on the right. The CRC model may make use of actual or virtual index cards. The intent is to develop an organized representation of classes. **Responsibilities** are the attributes and operations that are relevant for the class. i.e., a responsibility is “anything the class knows or does” **Collaborators** are those classes that are required to provide a class with the information needed to complete a responsibility. In general, a collaboration implies either a request for information or a request for some action.

A simple CRC index card is illustrated in following figure.



Class: FloorPlan	
Description	
Responsibility:	Collaborator:
Defines floor plan name/type	
Manages floor plan positioning	
Scales floor plan for display	
Scales floor plan for display	
Incorporates walls, doors, and windows	Wall
Shows position of video cameras	Camera

Fig : A CRC model index card

Classes: The taxonomy of class types can be extended by considering the following categories:

- **Entity classes**, also called model or business classes, are extracted directly from the statement of the problem. These classes typically represent things that are to be stored in a database and persist throughout the duration of the application.
- **Boundary classes** are used to create the interface that the user sees and interacts with as the software is used. Boundary classes are designed with the responsibility of managing the way entity objects are represented to users.

- **Controller** classes manage a “unit of work” from start to finish. That is, controller classes can be designed to manage (1) the creation or update of entity objects, (2) the instantiation of boundary objects as they obtain information from entity objects, (3) complex communication between sets of objects, (4) validation of data communicated between objects or between the user and the application. In general, controller classes are not considered until the design activity has begun.

Responsibilities: Wirfs-Brock and her colleagues suggest five guidelines for allocating responsibilities to classes:

1. **System intelligence should be distributed across classes to best address the needs of the problem.** Every application encompasses a certain degree of intelligence; that is, what the system knows and what it can do.
2. **Each responsibility should be stated as generally as possible.** This guideline implies that general responsibilities should reside high in the class hierarchy.
3. **Information and the behavior related to it should reside within the same class.** This achieves the object-oriented principle called encapsulation. Data and the processes that manipulate the data should be packaged as a cohesive unit.
4. **Information about one thing should be localized with a single class, not distributed across multiple classes.** A single class should take on the responsibility for storing and manipulating a specific type of information. This responsibility should not, in general, be shared across a number of classes. If information is distributed, software becomes more difficult to maintain and more challenging to test.
5. **Responsibilities should be shared among related classes, when appropriate.** There are many cases in which a variety of related objects must all exhibit the same behavior at the same time.

Collaborations: Classes fulfill their responsibilities in one of two ways:

1. A class can use its own operations to manipulate its own attributes, thereby fulfilling a particular responsibility, or
2. A class can collaborate with other classes.

When a complete CRC model has been developed, stakeholders can review the model using the following approach:

1. All participants in the review (of the CRC model) are given a subset of the CRC model index cards. Cards that collaborate should be separated (i.e., no reviewer should have two cards that collaborate).
2. All use-case scenarios (and corresponding use-case diagrams) should be organized into categories.
3. The review leader reads the use case deliberately. As the review leader comes to a named object, she passes a token to the person holding the corresponding class index card.
4. When the token is passed, the holder of the card is asked to describe the responsibilities noted on the card. The group determines whether one (or more) of the responsibilities satisfies the use-case requirement.
5. If the responsibilities and collaborations noted on the index cards cannot accommodate the use case, modifications are made to the cards. This may include the definition of new classes (and corresponding CRC index cards) or the specification of new or revised responsibilities or collaborations on existing cards.

2.5.5 Associations and Dependencies:

An association defines a relationship between classes. An association may be further defined by indicating ***multiplicity***.

Multiplicity defines how many of one class are related to how many of another class. A client-server relationship exists between two analysis classes. In such cases, a client class depends on the server class in some way and a ***dependency relationship*** is established. Dependencies are defined by a ***stereotype***. A ***stereotype*** is an “***extensibility mechanism***” within UML that allows you to define a special modeling element whose semantics are custom defined. In UML. Stereotypes are represented in double angle brackets (e.g., <<***stereotype***>>).

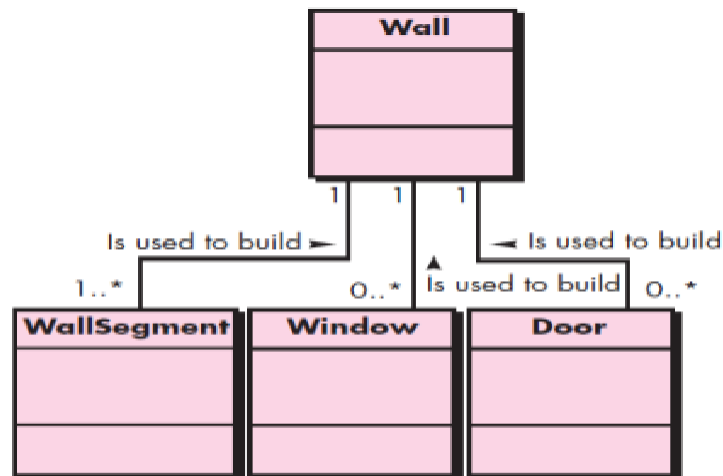


Fig : Multiplicity

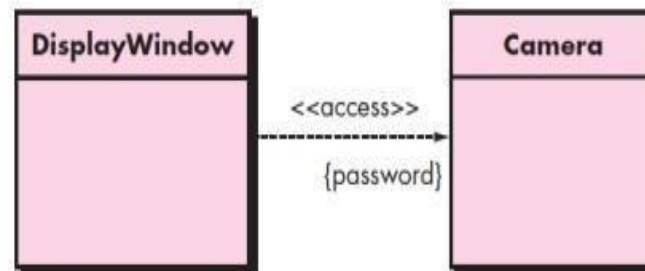


Fig : Dependencies

2.5.6 Analysis Packages: An important part of analysis modeling is categorization. That is, various elements of the analysis model (e.g., use cases, analysis classes) are categorized in a manner that packages them as a grouping—called an analysis package—that is given a representative name.

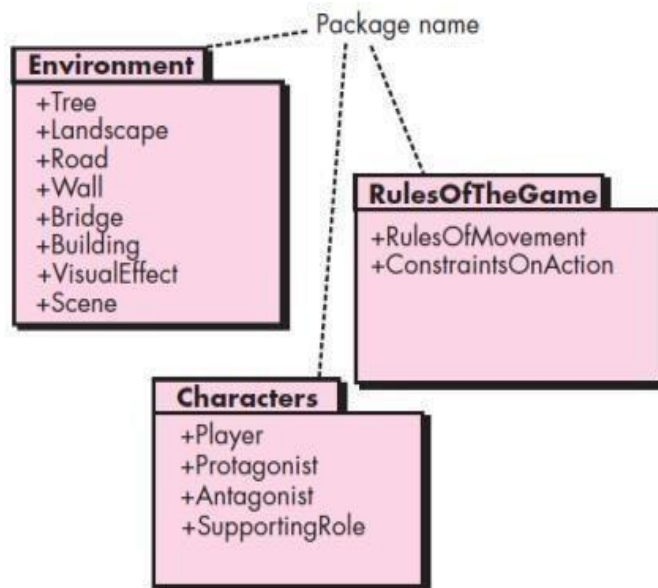


Fig : Packages

MODULE 3

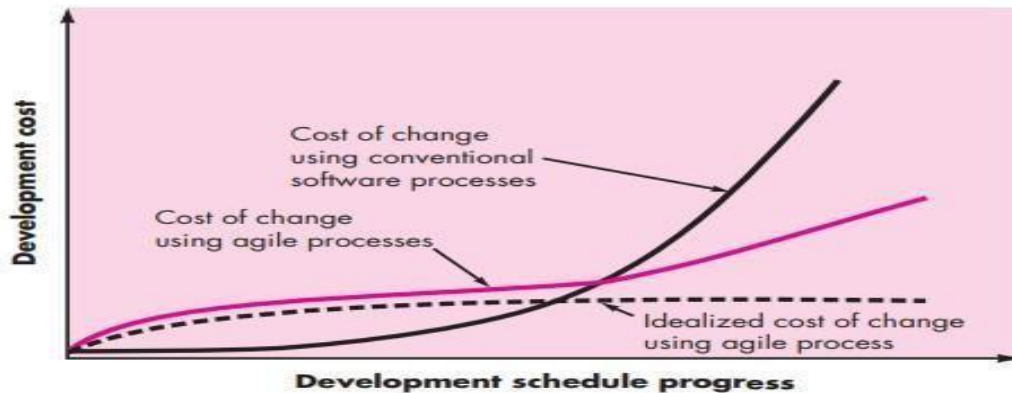
CHAPTER 1 - AGILE DEVELOPMENT

1.1 WHAT IS AGILITY?

- Agile is a software development methodology to build software incrementally using short iterations of 1 to 4 weeks so that the development process is aligned with the changing business needs.
- An agile team is a nimble team able to appropriately respond to changes. Change is what software development is very much about. Changes in the software being built, changes to the team members, changes because of new technology, changes of all kinds that may have an impact on the product they build or the project that creates the product. Support for changes should be built- in everything we do in software, something we embrace because it is the heart and soul of software.
- An agile team recognizes that software is developed by individuals working in teams and that the skills of these people, their ability to collaborate is at the core for the success of the project.
- The aim of agile process is to deliver the working model of software quickly to the customer For example: Extreme programming is the best known of agile process.

1.2 AGILITY AND THE COST OF CHANGE

- In conventional software development the cost of change increases non linearly as a project progresses (**Fig Solid Black curve**).
- **An agile process reduces the cost of change because software is released in increments** and change can be better controlled within an increment.
- Agility argue that a well-designed agile process “flattens” the cost of change curve shown in following figure (**shaded, solid curve**), allowing a software team to accommodate changes late in a software project without dramatic cost and time impact.
- When incremental delivery is coupled with other agile practices such as continuous unit testing and pair programming, the cost of making a change is attenuated(reduced). Although debate about the degree to which the cost curve flattens is ongoing, there is evidence to suggest that a significant reduction in the cost of change can be achieved. application, design, architecture etc. The verification process involves activities like reviews, walk-throughs and inspection.



1.3 WHAT IS AN AGILE PROCESS?

Any agile software process is characterized in a manner that addresses a number of key assumptions about the majority of software projects:

1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
3. Analysis, design, construction, and testing are not as predictable.

Given these three assumptions, an important question arises:

- 1) How do we create a process that can manage unpredictability?

It lies in process **adaptability**. **An agile process, therefore, must be adaptable.** But continual adaptation without forward progress accomplishes little. Therefore, an agile software process must adapt incrementally.

To accomplish incremental adaptation, an agile team requires customer feedback. An effective catalyst for customer feedback is an operational prototype or a portion of an operational system. Hence, an incremental development strategy should be instituted. Software increments must be delivered in short time periods so that adaptation keeps pace with change.

This iterative approach enables the customer to evaluate the software increment regularly, provide necessary feedback to the software team, and influence the process adaptations that are made to accommodate the feedback.

1.3.1 Agility Principles

Agility principles for those who want to achieve agility:

1. Our highest priority is to **satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers **must work together** daily throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team **is face-to-face conversation**.
7. **Working software** is the primary measure of progress.
8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to **technical excellence and good design** enhances agility.
10. **Simplicity**—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from **self-organizing teams**
12. At regular intervals, the team reflects on how to become more **effective**, then tunes and adjusts its behavior accordingly.

Not every agile process model applies these 12 principles with equal weight, and some models choose to ignore (or at least downplay) the importance of one or more of the principles.

1.3.2 The Politics of Agile Development

- There is debate about the benefits and applicability of agile software development as opposed to more conventional software engineering processes (produces documents rather than working product).
- Even within the agile, there are many proposed process models each with a different approach to the agility.

1.3.3 Human Factors

Agile development focuses on the talents and skills of individuals, molding the process to specific people and teams.” The key point in this statement is that *the process molds to the needs of the people and team*.

1. **Competence.** In an agile development context, “competence” encompasses innate talent, specific software-related skills, and overall knowledge of the process that the team has chosen to apply. Skill and knowledge of process can and should be taught to all people who serve as agile team members.
2. **Common focus.** Although members of the agile team may perform different tasks and bring different skills to the project, all should be focused on one goal—to deliver a working software increment to the customer within the time promised. To achieve this goal, the team will also focus on continual adaptations (small and large) that will make the process fit the needs of the team.
3. **Collaboration.** Software engineering (regardless of process) is about assessing, analyzing, and using information that is communicated to the software team; creating information that will help all stakeholders understand the work of the team; and building information (computer software and relevant databases) that provides business value for the customer. To accomplish these tasks, team members must collaborate—with one another and all other stakeholders.
4. **Decision-making ability.** Any good software team (including agile teams) must be allowed the freedom to control its own destiny. This implies that the team is given autonomy— decision-making authority for both technical and project issues.
5. **Fuzzy problem-solving ability.** Software managers must recognize that the agile team will continually have to deal with ambiguity and will continually be buffeted by change.
6. **Mutual trust and respect.** The agile team must become what DeMarco and Lister call a “jelled” team. A jelled team exhibits the trust and respect that are necessary to make them “so strongly knit that the whole is greater than the sum of the parts.”
7. **Self-organization.** In the context of agile development, self-organization implies three things: (1) the agile team organizes itself for the work to be done, (2) the team organizes the process to best accommodate its local environment, (3) the team organizes the work schedule to best achieve delivery of the software increment. Self-organization has a number of technical benefits, but more importantly, it serves to improve collaboration and boost team morale.

1.4 EXTREME PROGRAMMING (XP)

Extreme Programming (XP), the most widely used approach to agile software development, emphasizes business results first and takes an incremental, get-something-started approach to building the product, using continual testing and revision. XP proposed by **Kent beck** during the late 1980's.

1.4.1 XP Values

Beck defines a set of **five** values that establish a foundation for all work performed as part of XP—**communication, simplicity, feedback, courage, and respect**. Each of these values is used as a driver for specific XP **activities, actions, and tasks**.

In order to achieve effective **communication** between software engineers and other stakeholders, XP emphasizes close, yet informal collaboration between customers and developers, the establishment of effective metaphors for communicating important concepts, continuous feedback, and the avoidance of voluminous documentation as a communication medium.

To achieve **simplicity**, **XP** restricts developers to design only for immediate needs, rather than consider future needs. The intent is to create a simple design that can be easily implemented in code. If the design must be improved, it can be refactored at a later time.

Feedback is derived from three sources: the implemented software itself, the customer, and other software team members. By designing and implementing an effective testing strategy the software provides the agile team with feedback. XP makes use of the **unit test** as its primary testing tactic. As each class is developed, the team develops a unit test to exercise each operation according to its specified functionality.

Beck argues that strict adherence to certain XP practices demands **courage**. A better word might be **discipline**. An agile XP team must have the discipline (courage) to design for today, recognizing that future requirements may change dramatically, thereby demanding substantial rework of the design and implemented code.

By following each of these values, the agile team inculcates **respect** among its members, between other stakeholders and team members, and indirectly, for the software itself. As they achieve successful delivery of software increments, the team develops growing respect for the XP process.

1.4.2 The XP Process

Extreme Programming uses an object-oriented approach as its preferred development paradigm and encompasses a set of rules and practices that occur within the context of **four framework activities: planning, design, coding, and testing**. Following figure illustrates the XP process and notes some of the key ideas and tasks that are associated with each framework activity.

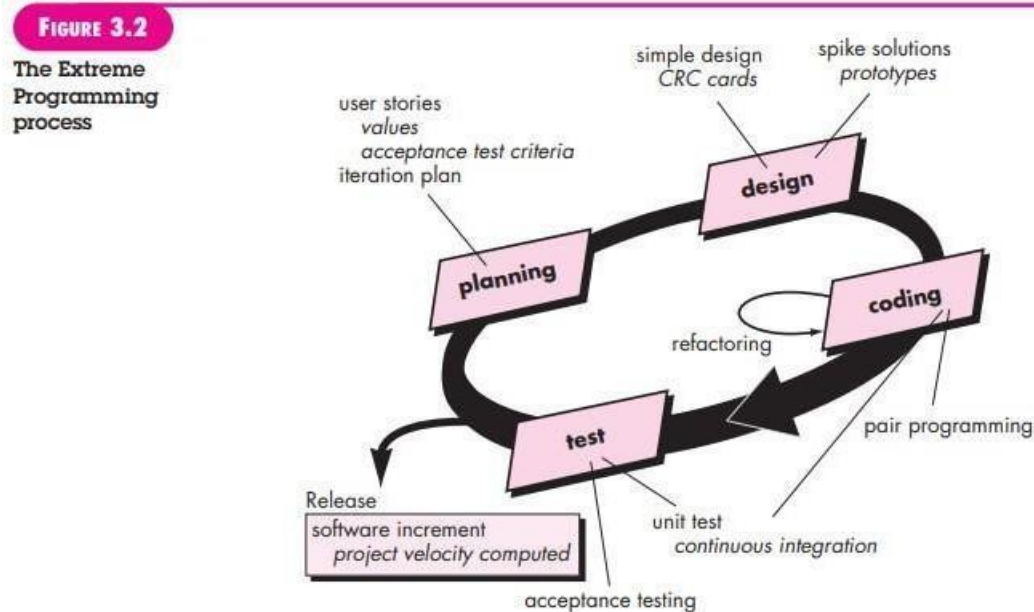


Fig : The Extreme Programming process

Key XP activities are

1) Planning. The planning activity begins with listening—a requirements gathering activity.

- **Listening** leads to the creation of a set of “stories” (also called user stories) that describe required output, features, and functionality for software to be built.
- Each story is written by the customer and is placed on an index card. The customer assigns a value (i.e., a priority) to the story based on the overall business value of the feature or function.
- Members of the XP team then assess each story and assign a cost— measured in development weeks—to it.
- If the story is estimated to require more than three development weeks, the story into smaller stories and the assignment of value and cost occurs again. It is important to note that new stories can be written at any time.

- The stories with highest value will be moved up in the schedule and implemented first

2) Design. XP design rigorously follows the KIS (keep it simple) principle.

- If a difficult design problem is encountered as part of the design of a story, XP recommends the immediate creation of an operational prototype of that portion of the design. **Called a spike solution.**
- XP encourages **refactoring**—a construction technique that is also a method for design optimization.
- Refactoring is the process of changing a software system in a way that it does not change the external behavior of the code and improves the internal structure.

3) Coding. After stories are developed and preliminary design work is done, the team does not move to code, develops a series of unit tests for each of the stories that is to be included in the current release (software increment).

- Once the unit test has been created, the developer is better able to focus on what must be implemented to pass the test.
- Once the code is complete, it can be unit-tested immediately, and providing feedback to the developers.
- A key concept during the coding activity is pair programming. i.e., two people work together at one computer workstation to create code for a story.
- As pair programmers complete their work, the code they develop is integrated with the work of others.

4) Testing. The creation of unit tests before coding commences is a key element of the XP approach. The unit tests that are created should be implemented using a framework that enables them to be automated. This encourages a regression testing strategy whenever code is modified.

- As the individual unit tests are organized into a “universal testing suite” integration and validation testing of the system can occur on a daily basis. This provides the XP team with a continual indication of progress and also can raise warning flags early if things go awry. Wells states: “Fixing small problems every few hours takes less time than fixing huge problems just before the deadline.”

- **XP acceptance tests**, also called **customer tests**, are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer. Acceptance tests are derived from user stories that have been implemented as part of a software release.

1.4.3 Industrial XP

Joshua Kerievsky describes Industrial Extreme Programming (IXP) in the following manner: “IXP is an organic evolution of XP. It is imbued with XP’s minimalist, customer-centric, test-driven spirit. IXP differs most from the original XP in its greater inclusion of management, its expanded role for customers, and its upgraded technical practices.” IXP incorporates six new practices that are designed to help ensure that an XP project works successfully for significant projects within a large organization.

Readiness assessment. Prior to the initiation of an IXP project, the organization should conduct a readiness assessment. The assessment ascertains whether (1) an appropriate development environment exists to support IXP, (2) the team will be populated by the proper set of stakeholders, (3) the organization has a distinct quality program and supports continuous improvement, (4) the organizational culture will support the new values of an agile team, and (5) the broader project community will be populated appropriately.

Project community. Classic XP suggests that the right people be used to populate the agile team to ensure success. The implication is that people on the team must be well-trained, adaptable and skilled, and have the proper temperament to contribute to a self-organizing team. When XP is to be applied for a significant project in a large organization, the concept of the “team” should morph into that of a community. A community may have a technologist and customers who are central to the success of a project as well as many other stakeholders (e.g., legal staff, quality auditors, manufacturing or sales types) who “are often at the periphery of an IXP project yet they may play important roles on the project”. In IXP, the community members and their roles should be explicitly defined and mechanisms for communication and coordination between community members should be established.

Project chartering. The IXP team assesses the project itself to determine whether an appropriate business justification for the project exists and whether the project will further the overall goals and objectives of the organization. Chartering also examines the context of the project to determine how it complements, extends, or replaces existing systems or processes.

Test-driven management. An IXP project requires measurable criteria for assessing the state of the project and the progress that has been made to date. Test-driven management establishes a series of measurable “destinations” and then defines mechanisms for determining whether or not these destinations have been reached.

Retrospectives. An IXP team conducts a specialized technical review after a software increment is delivered. Called a retrospective, the review examines “issues, events, and lessons-learned” across a software increment and/or the entire software release. The intent is to improve the IXP process.

Continuous learning. Because learning is a vital part of continuous process improvement, members of the XP team are encouraged (and possibly, incited) to learn new methods and techniques that can lead to a higher quality product.

1.4.4 The XP Debate

- **Requirements volatility.** The customer is an active member of the XP team, changes to requirements are requested informally. As a consequence, the scope of the project can change and earlier work may have to be modified to accommodate current needs.
- **Conflicting customer needs.** Many projects have multiple customers, each with his own set of needs.
- **Requirements are expressed informally.** User stories and acceptance tests are the only explicit manifestation of requirements in XP. specification is often needed to remove inconsistencies, and errors before the system is built.
- **Lack of formal design:** when complex systems are built, design must have the overall structure of the software then it will exhibit quality.

1.5 OTHER AGILE PROCESS MODELS

Other agile process models have been proposed and are in use across the industry. Among the most common are:

- Adaptive Software Development (ASD)
- Scrum
- Dynamic Systems Development Method (DSDM)
- Crystal
- Feature Drive Development (FDD)
- Lean Software Development (LSD)
- Agile Modeling (AM)
- Agile Unified Process (AUP)

Adaptive Software Development (ASD)

Adaptive Software Development (ASD) has been proposed by Jim Highsmith as a technique for building complex software and systems. The philosophical underpinnings of ASD focus on human collaboration and team self-organization.

Highsmith argues that an agile, adaptive development approach based on collaboration is “as much a source of order in our complex interactions as discipline and engineering.” He defines an ASD “life cycle” that incorporates three phases, speculation, collaboration, and learning.

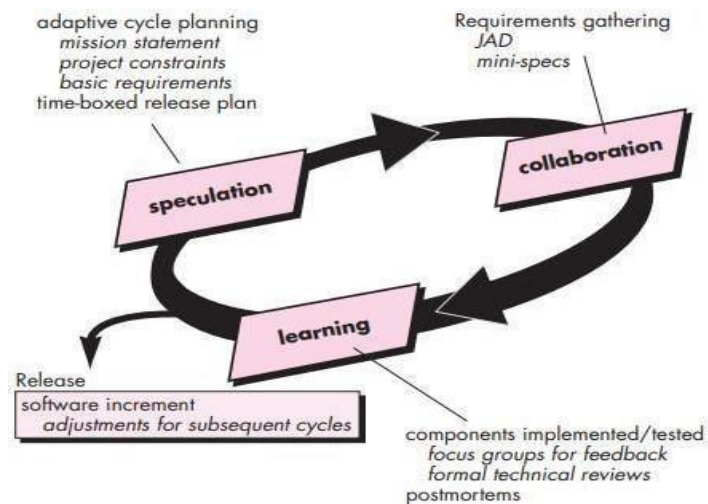


Fig: Adaptive software development

During **speculation**, the project is initiated and **adaptive cycle planning** is conducted. Adaptive cycle planning uses project initiation information—the customer’s mission statement, project constraints (e.g., delivery dates or user descriptions), and basic requirements—to define the set of release cycles (software increments) that will be required for the project.

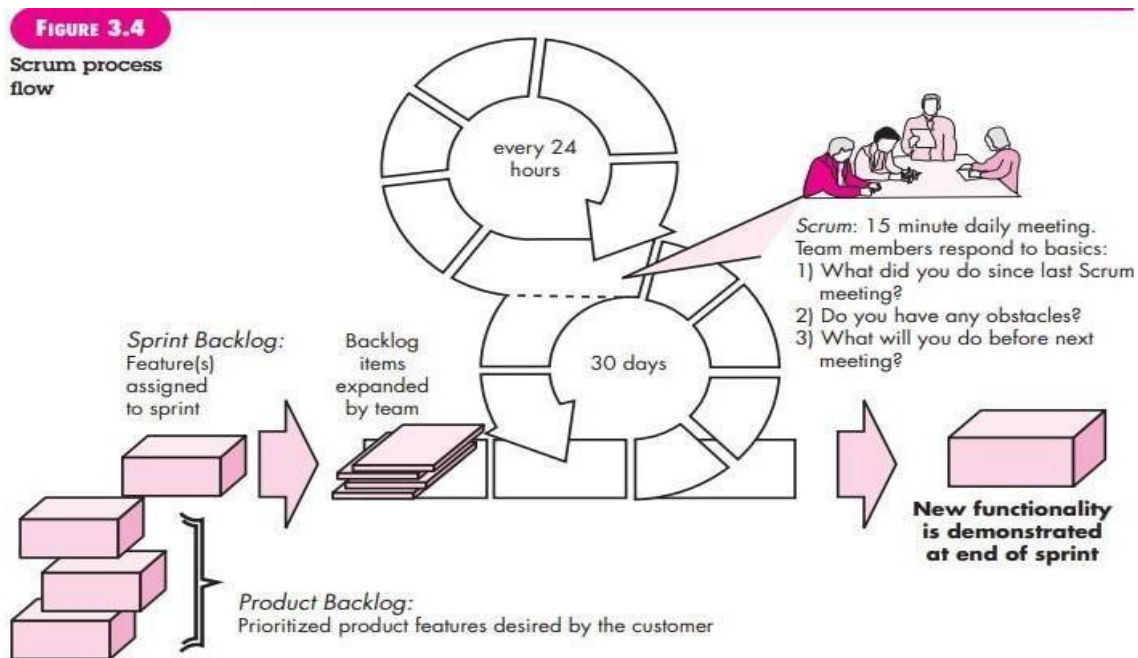
Motivated people use **collaboration** in a way that multiplies their talent and creative output beyond their absolute numbers. This approach is a recurring theme in all agile methods. But collaboration is not easy. It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking. It is, above all, a matter of trust. People working together must trust one another to (1) criticize without animosity, (2) assist without resentment, (3) work as hard as or harder than they do, (4) have the skill set to contribute to the work at hand, and (5) communicate problems or concerns in a way that leads to effective action.

As members of an ASD team begin to develop the components that are part of an adaptive cycle, the emphasis is on “**learning**” as much as it is on progress toward a completed cycle.

ASD teams learn in three ways: **focus groups, technical reviews, and project postmortems**. ASD's overall emphasis on the dynamics of self-organizing teams, interpersonal collaboration, and individual and team learning yield software project teams that have a much higher likelihood of success.

Scrum

Scrum is an agile software development method that was conceived by Jeff Sutherland and his development team in the early 1990s. Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery. Within each framework activity, work tasks occur within a process pattern called a sprint. The work conducted within a sprint is adapted to the problem at hand and is defined and often modified in real time by the Scrum team. The overall flow of the Scrum process is illustrated in following figure.



Scrum emphasizes the use of a set of software process patterns that have proven effective for projects with tight timelines, changing requirements, and business criticality. Each of these process patterns defines a set of development actions:

Backlog—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time. The product manager assesses the backlog and updates priorities as required.

Sprints—consist of work units that are required to achieve a requirement defined in the backlog that must be fit into a predefined time-box (typically 30 days). Changes (e.g., backlog work items) are not introduced during the sprint. Hence, the sprint allows team members to work in a short-term, but stable environment.

Scrum meetings—are short (typically 15 minutes) meetings held daily by the Scrum team. Three key questions are asked and answered by all team members

- What did you do since the last team meeting?
- What obstacles are you encountering?
- What do you plan to accomplish by the next team meeting?

A team leader, called a **Scrum master**, leads the meeting and assesses the responses from each person. The Scrum meeting helps the team to uncover potential problems as early as possible. Also, these daily meetings lead to “knowledge socialization”

Demos—deliver the software increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer. It is important to note that the demo may not contain all planned functionality, but rather those functions that can be delivered within the time-box that was established.

Dynamic Systems Development Method (DSDM)

The Dynamic Systems Development Method (DSDM) is an agile software development approach that “provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment” The DSDM philosophy is borrowed from a modified version of the **Pareto principle—80 percent of an application can be delivered in 20 percent of the time. It would take to deliver the complete (100percent) application.** DSDM is an iterative software process in which each iteration follows the 80 percent rule. That is, only enough work is required for each increment to facilitate movement to the next increment. The remaining detail can be completed later when more business requirements are known or changes have been requested and accommodated.

The *DSDM life cycle* that defines three different iterative cycles, preceded by two additional life cycle activities:

- **Feasibility study**—establishes the basic business requirements and constraints associated with the application to be built and then assesses whether the application is a viable candidate for the DSDM process

- **Business study**—establishes the functional and information requirements that will allow the application to provide business value; also, defines the basic application architecture and identifies the maintainability requirements for the application.
- **Functional model iteration**—produces a set of incremental prototypes that demonstrate functionality for the customer. The intent during this iterative cycle is to gather additional requirements by eliciting feedback from users as they exercise the prototype.
- **Design and build iteration**—revisits prototypes built during functional model iteration to ensure that each has been engineered in a manner that will enable it to provide operational business value for end users. In some cases, functional model iteration and design and build iteration occur concurrently.
- **Implementation**—places the latest software increment into the operational environment. It should be noted that (1) the increment may not be 100 percent complete or (2) changes may be requested as the increment is put into place. In either case, DSDM development work continues by returning to the functional model iteration activity.

Crystal

Alistair Cockburn and Jim Highsmith created *the Crystal family of agile methods* in order to achieve a software development approach that puts a premium on “maneuverability” during what Cockburn characterizes as “a resource limited, cooperative game of invention and communication, with a primary goal of delivering useful, working software and a secondary goal of setting up for the next game”

The Crystal family is actually a set of example agile processes that have been proven effective for different types of projects. The intent is to allow agile teams to select the member of the crystal family that is most appropriate for their project and environment.

Feature Driven Development (FDD)

Feature Driven Development (FDD) was originally conceived by Peter Coad and his colleagues as a practical process model for object-oriented software engineering. Stephen Palmer and John Felsing have extended and improved Coad’s work, describing an adaptive, agile process that can be applied to moderately sized and larger software projects.

Like other agile approaches, FDD adopts a philosophy that (1) emphasizes collaboration among people on an FDD team; (2) manages problem and project complexity using feature-based decomposition followed by the integration of software increments, and (3) communication of technical detail using verbal, graphical, and text-based means.

FDD emphasizes software quality assurance activities by encouraging an incremental development strategy, the use of design and code inspections, the application of software quality assurance audits, the collection of metrics, and the use of patterns (for analysis, design, and construction).

In the context of FDD, a *feature* “is a client-valued function that can be implemented in two weeks or less” The emphasis on the definition of features provides the following benefits:

- Because features are small blocks of deliverable functionality, users can describe them more easily; understand how they relate to one another more readily; and better review them for ambiguity, error, or omissions.
- Features can be organized into a hierarchical business-related grouping.
- Since a feature is the FDD deliverable software increment, the team develops operational features every two weeks.
- Because features are small, their design and code representations are easier to inspect effectively.
- Project planning, scheduling, and tracking are driven by the feature hierarchy, rather than an arbitrarily adopted software engineering task set.

Coad and his colleagues suggest the following template for defining a feature:

<action> the <result> <by|for| of |to> a(n) <object>
Where an **<object>** is “a person, place or thing.

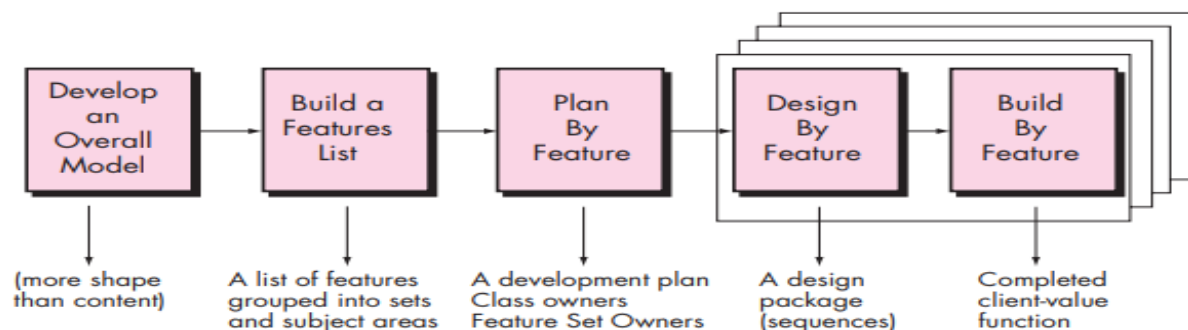


Fig: Feature Driven Development (FDD)

FDD provides greater emphasis on project management guidelines and techniques than many other agile methods. FDD defines six milestones during the design and implementation of a feature: “design walkthrough, design, design inspection, code, code inspection, promote to build”

Lean Software Development (LSD)

Lean Software Development (LSD) has adapted the principles of lean manufacturing to the world of software engineering. The lean principles that inspire the LSD process can be summarized as eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, and optimize the whole. Each of these principles can be adapted to the software process.

For example, eliminate waste within the context of an agile software project as

- 1) adding no extraneous features or functions
- (2) assessing the cost and schedule impact of any newly requested requirement,
- (3) removing any superfluous process steps,
- (4) establishing mechanisms to improve the way team members find information,
- (5) ensuring the testing finds as many errors as possible,

Agile Modeling (AM)

Agile Modeling (AM) is a practice-based methodology for effective modeling and documentation of software-based systems. Simply put, Agile Modeling (AM) is a collection of values, principles, and practices for modeling software that can be applied on a software development project in an effective and light-weight manner. Agile models are more effective than traditional models because they are just barely good, they don't have to be perfect.

Agile modeling adopts all of the values that are consistent with the agile manifesto. The agile modeling philosophy recognizes that an agile team must have the courage to make decisions that may cause it to reject a design and refactor. The team must also have the humility to recognize that technologists do not have all the answers and that business experts and other stakeholders should be respected and embraced.

Agile Modeling suggests a wide array of “core” and “supplementary” modeling principles, those that make AM unique are:

- **Model with a purpose.** A developer who uses AM should have a specific goal in mind before creating the model. Once the goal for the model is identified, the type of notation to be used and level of detail required will be more obvious.

- **Use multiple models.** There are many different models and notations that can be used to describe software. Only a small subset is essential for most projects. AM suggests that to provide needed insight, each model should present a different aspect of the system and only those models that provide value to their intended audience should be used.
- **Travel light.** As software engineering work proceeds, keep only those models that will provide long-term value and jettison the rest. Every work product that is kept must be maintained as changes occur. This represents work that slows the team down. Ambler notes that “Every time you decide to keep a model you trade-off agility for the convenience of having that information available to your team in an abstract manner
- **Content is more important than representation.** Modeling should impart information to its intended audience. A syntactically perfect model that imparts little useful content is not as valuable as a model with flawed notation that nevertheless provides valuable content for its audience. • Know the models and the tools you use to create them. Understand the strengths and weaknesses of each model and the tools that are used to create it.
- **Adapt locally.** The modeling approach should be adapted to the needs of the agile team.

Agile Unified Process (AUP)

The *Agile Unified Process* (AUP) adopts a “serial in the large” and “iterative in the small” philosophy for building computer-based systems. By adopting the classic UP phased activities—*inception*, *elaboration*, *construction*, and *transition*—AUP provides a serial overlay that enables a team to visualize the overall process flow for a software project. However, within each of the activities, the team iterates to achieve agility and to deliver meaningful software increments to end users as rapidly as possible. Each AUP iteration addresses the following activities. •

- **Modeling.** UML representations of the business and problem domains are created.
- **Implementation.** Models are translated into source code.
- **Testing.** Like XP, the team designs and executes a series of tests to uncover errors and ensure that the source code meets its requirements.
- **Deployment.** Like the generic process activity deployment in this context focuses on the delivery of a software increment and the acquisition of feedback from end users.
- **Configuration and project management.** In the context of AUP, configuration management addresses change management, risk management, and the control of any persistent work products

that are produced by the team. Project management tracks and controls the progress of the team and coordinates team activities.

- **Environment management.** Environment management coordinates a process infrastructure that includes standards, tools, and other support technology available to the team.

1.6 A TOOL SET FOR AGILE PROCESS

Some proponents of the agile philosophy argue that automated software tools (e.g., design tools) should be viewed as a minor supplement to the team's activities, and not at all pivotal to the success of the team.

However, Alistair Cockburn [Coc04] suggests that tools can have a benefit and that “agile teams stress using tools that permit the rapid flow of understanding. Some of those tools are social, starting even at the hiring stage. Some tools are technological, helping distributed teams simulate being physically present.

Many tools are physical, allowing people to manipulate them in workshops.” Because acquiring the right people (hiring), team collaboration, stakeholder communication, and indirect management are key elements in virtually all agile process models, Cockburn argues that “tools” that address these issues are critical success factors for agility.

For example, a hiring “tool” might be the requirement to have a prospective team member spend a few hours pair programming with an existing member of the team. The “fit” can be assessed immediately.

Collaborative and communication “tools” are generally low tech and incorporate any mechanism (“physical proximity, whiteboards, poster sheets, index cards, and sticky notes” [Coc04]) that provides information and coordination among agile developers

Active communication is achieved via the team dynamics (e.g., pair programming), while passive communication is achieved by “information radiators” (e.g., a flat panel display that presents the overall status of different components of an increment).

Project management tools deemphasize the Gantt chart and replace it with earned value charts or “graphs of tests created versus passed . . . other agile tools are used to optimize the environment in which the agile team works (e.g., more efficient meeting areas), improve the team culture by nurturing social interactions (e.g., collocated teams), physical devices (e.g., electronic whiteboards), and process enhancement (e.g., pair programming or time-boxing)” [Coc04].

CHAPTER 2

PRINCIPLES THAT GUIDE THE PRACTICE

Practice is a collection of concepts, principles, methods, and tools that a software engineer calls upon on a daily basis.

Practice allows managers to manage software projects and software engineers to build computer programs.

Practice populates a software process model with the necessary technical and management how-to's to get the job done.

1.1 SOFTWARE ENGINEERING KNOWLEDGE

In an editorial published in IEEE Software a decade ago, Steve McConnell [McC99] made the following comment:

Many software practitioners think of software engineering knowledge almost exclusively as knowledge of specific technologies: Java, Perl, html, C++ Linux, Windows NT, and so on.

Knowledge of specific technology details is necessary to perform computer programming. If someone assigns you to write a program in C++, you have to know something about C++ to get your program to work.

You often hear people say that software development knowledge has a 3-year half-life: half of what you need to know today will be obsolete within 3 years. In the domain of technology-related knowledge, that's probably about right. But there is another kind of software development knowledge—a kind that I think of as “software engineering principles”—that does not have a three- year half-life. These software engineering principles are likely to serve a professional programmer throughout his or her career.

McConnell goes on to argue that the body of software engineering knowledge (circa the year 2000) had evolved to a “stable core” that he estimated represented about “75 percent of the knowledge needed to develop a complex system.” But what resides within this stable core? As McConnell indicates, core principles—the elemental ideas that guide software engineers in the work that they do—now provide a foundation from which software engineering models, methods, and tools can be applied and evaluated.

1.8 CORE PRINCIPLES

Software engineering is guided by a collection of core principles that help in the application of a meaningful software process and the execution of effective software engineering methods. At the process level, core principles establish a philosophical foundation that guides a software team as it performs framework and umbrella activities, navigates the process flow, and produces a set of software engineering work products.

At the level of practice, core principles establish a collection of values and rules that serve as a guide as you analyze a problem, design a solution, implement and test the solution, and ultimately deploy the software in the user community identified a set of general principles that span software engineering process and practice:

- (1) **provide value to end users,**
- (2) **keep it simple,**
- (3) **maintain the vision (of the product and the project),**
- (4) **recognize that others consume (and must understand) what you produce,**
- (5) **be open to the future,**
- (6) **plan ahead for reuse, and**
- (7) **think!**

Although these general principles are important, they are characterized at such a high level of abstraction that they are sometimes difficult to translate into day-to-day software engineering practice.

1.8.1 Principles that Guide Process

The following set of core principles can be applied to the framework, and by extension, to every software process.

Principle 1: Be agile. Whether the process model you choose is prescriptive or agile.

1. keep your technical approach as simple as possible
2. keep the work products you produce as concise(short) as possible
3. Make decisions locally whenever possible.

Principle 2. Focus on quality at every step. For every process activity, action, and task should focus on the quality of the work product that has been produced.

Principle 3. Be ready to adapt. adapt your approach to conditions imposed by the problem, the people, and the project itself.

Principle 4. Build an effective team. Build a self-organizing team that has mutual trust and respect.

Principle 5. Establish mechanisms for communication and coordination. Projects fail because stakeholders fail to coordinate their efforts to create a successful end product.

Principle 6. Manage change. The methods must be established to manage the way changes are requested, approved, and implemented.

Principle 7. Assess risk. Lots of things can go wrong as software is being developed.

Principle 8. Create work products that provide value for others. Create only those work products that provide value for other process activities, actions and tasks.

1.8.2 Principles That Guide Practice

- Software engineering practice has a single goal i.e., to deliver on-time, high quality, operational software that contains functions and features that meet the needs of all stakeholders.
- To achieve this goal, should adopt a set of core principles that guide the technical work.
- The following **set of core principles are fundamental to the practice of software engineering**:

Principle 1. Divide and conquer. A large problem is easier to solve if it is subdivided into a collection of elements (or modules or components). Ideally, each element delivers distinct functionality that can be developed.

Principle 2. Understand the use of abstraction. At its core, an abstraction(overview) is a simplification of some complex element of a system used to communicate meaning in a single phrase.

Principle 3. Strive for consistency. Whether it's creating a requirements model, developing a software design, generating source code, or creating test cases. All these are consistent so that the software is easier to develop.

Principle 4. Focus on the transfer of information. Software is about information transfer—from a database to an end user, from an operating system to an application,

Principle 5. Build software that exhibits effective modularity. Modularity provides a mechanism for any complex system can be divided into modules (components).

Principle 6. Look for patterns. Brad Appleton suggests that: The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development process.

Principle 7. When possible, represent the problem and its solution from a number of different perspectives. When a problem and its solution are examined from a number of different perspectives(ways).

Principle 8. Remember that someone will maintain the software. software will be corrected as defects are remove, adapted as its environment changes, and enhanced as stakeholders request more capabilities.

1.9 PRINCIPLES THAT GUIDE EACH FRAMEWORK ACTIVITY

1.9.1 Communication Principles

Customer requirements must be gathered through the **communication activity**. Communication has begun.

Principle 1. Listen.

Try to focus on the speaker's words, rather than formulating your response to those words. Ask for clarification if something is unclear, but avoid constant interruptions. Never become contentious in your words or actions (e.g., rolling your eyes or shaking your head) as a person is talking.

Principle 2. Prepare before you communicate. Spend the time to understand the problem before you meet with others. If necessary, do some research to understand business domain jargon. If you have responsibility for conducting a meeting, prepare an agenda in advance of the meeting.

Principle 3. Someone should facilitate the activity. Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction, (2) to mediate any conflict that does occur, and (3) to ensure than other principles are followed.

Principle 4. Face-to-face communication is best. But it usually works better when some other representation of the relevant information is present. For example, a participant may create a drawing or a "strawman" document that serves as a focus for discussion.

Principle 5. Take notes and document decisions. Things have a way of falling into the cracks. Someone participating in the communication should serve as a "recorder" and write down all important points and decisions.

Principle 6. Strive for collaboration. Collaboration and consensus occur when the collective knowledge of members of the team is used to describe product or system functions or features. Each small collaboration serves to build trust among team members and creates a common goal for the team.

Principle 7. Stay focused; modularize your discussion. The more people involved in any communication, the more likely that discussion will bounce from one topic to the next. The facilitator should keep the conversation modular, leaving one topic only after it has been resolved

Principle 8. If something is unclear, draw a picture. Verbal communication goes only so far. A sketch or drawing can often provide clarity when words fail to do the job.

Principle 9. (a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified at the moment, move on. Communication, like any software engineering activity, takes time. Rather than iterating endlessly, the people who participate should recognize that many topics require discussion (see Principle 2) and that "moving on" is sometimes the best way to achieve communication agility.

Principle 10. Negotiation is not a contest or a game. It works best when both parties win. There are many instances in which you and other stakeholders must negotiate functions and features, priorities, and delivery dates. If the team has collaborated well, all parties have a common goal. Still, negotiation will demand compromise from all parties.

1.9.2 Planning Principles

- The communication activity helps you to define your overall goals and objectives.
- The planning activity encompasses a set of management and technical practices that enable the software team to define a road map as it travels toward the objectives.

The following principles always apply:

Principle 1. Understand the scope of the project. It's impossible to use a road map if you don't know where you're going. Scope provides the software team with a destination.

Principle 2. Involve stakeholders in the planning activity. Stakeholders define priorities and establish project constraints. To accommodate these realities, software engineers must often negotiate order of delivery, time lines, and other project-related issues.

Principle 3. Recognize that planning is iterative. A project plan is never engraved in stone. As work begins, it is very likely that things will change. As a consequence, the plan must be adjusted to accommodate these changes. In addition, iterative, incremental process models dictate replanning after the delivery of each software increment based on feedback received from users.

Principle 4. Estimate based on what you know. The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done. If information is vague or unreliable, estimates will be equally unreliable.

Principle 5. Consider risk as you define the plan. If you have identified risks that have high impact and high probability, contingency planning is necessary. In addition, the project plan (including the schedule) should be adjusted to accommodate the likelihood that one or more of these risks will occur.

Principle 6. Be realistic. People don't work 100 percent of every day. Noise always enters into any human communication. Omissions and ambiguity are facts of life. Change will occur. Even the best software engineers make mistakes. These and other realities should be considered as a project plan is established.

Principle 7. Adjust granularity as you define the plan. Granularity refers to the level of detail that is introduced as a project plan is developed. A "high-granularity" plan provides significant work task detail that is planned over relatively short time increments (so that tracking and control occur frequently). A "low-granularity" plan provides broader work tasks that are planned over longer time periods. In general, granularity moves from high to low as the project time line moves away from the current date. Over the next few weeks or months, the project can be planned in significant detail. Activities that won't occur for many months do not require high granularity (too much can change).

Principle 8. Define how you intend to ensure quality. The plan should identify how the software team intends to ensure quality. If technical reviews³ are to be conducted, they should be scheduled. If pair programming (Chapter 3) is to be used during construction, it should be explicitly defined within the plan.

Principle 9. Describe how you intend to accommodate change. Even the best planning can be obviated by uncontrolled change. You should identify how changes are to be accommodated as software engineering work proceeds. For example, can the customer request a change at any time? If a change is requested, is the team obliged to implement it immediately? How is the impact and cost of the change assessed?

Principle 10. Track the plan frequently and make adjustments as required. Software projects fall behind schedule one day at a time. Therefore, it makes sense to track progress on a daily basis, looking for problem areas and situations in which scheduled work does not conform to actual work conducted. When slippage is encountered, the plan is adjusted accordingly.

1.9.3 Modeling Principles

Create models to gain a better understanding of the actual entity to be built. The modeling principles are:

Principle 1. The primary goal of the software team is to build software, not create models. Agility means getting software to the customer in the fastest possible time. Models that make this

happen are worth creating, but models that slow the process down or provide little new insight should be avoided.

Principle 2. Travel light—don't create more models than you need. Every model that is created must be kept up-to-date as changes occur. More importantly, every new model takes time that might otherwise be spent on construction (coding and testing). Therefore, create only those models that make it easier and faster to construct the software.

Principle 3. Strive to produce the simplest model that will describe the problem or the software. Don't overbuild the software [Amb02b]. By keeping models simple, the resultant software will also be simple. The result is software that is easier to integrate, easier to test, and easier to maintain (to change). In addition, simple models are easier for members of the software team to understand and critique, resulting in an ongoing form of feedback that optimizes the end result.

Principle 4. Build models in a way that makes them amenable to change. Assume that your models will change, but in making this assumption don't get sloppy. For example, since requirements will change, there is a tendency to give requirements models short shrift. Why? Because you know that they'll change anyway. The problem with this attitude is that without a reasonably complete requirements model, you'll create a design (design model) that will invariably miss important functions and features.

Principle 5. Be able to state an explicit purpose for each model that is created. Every time you create a model, ask yourself why you're doing so. If you can't provide solid justification for the existence of the model, don't spend time on it.

Principle 6. Adapt the models you develop to the system at hand. It may be necessary to adapt model notation or rules to the application; for example, a video game application might require a different modeling technique than real-time, embedded software that controls an automobile engine.

Principle 7. Try to build useful models, but forget about building perfect models. When building requirements and design models, a software engineer reaches a point of diminishing returns. That is, the effort required to make the model absolutely complete and internally consistent is not worth the benefits of these properties. Am I suggesting that modeling should be sloppy or low quality? The answer is "no." But modeling should be conducted with an eye to the next software engineering steps. Iterating endlessly to make a model "perfect" does not serve the need for agility.

Principle 8. Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary. Although everyone on a software team should try to use consistent notation during modeling, the most important characteristic of the model is to communicate information that enables the next software engineering task. If a model does this successfully, incorrect syntax can be forgiven.

Principle 9. If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned. If you are an experienced software engineer, trust your instincts. Software work teaches many lessons—some of them on a subconscious level. If something tells you that a design model is doomed to fail (even though you can't prove it explicitly), you have reason to spend additional time examining the model or developing a different one.

Principle 10. Get feedback as soon as you can. Every model should be reviewed by members of the software team. The intent of these reviews is to provide feedback that can be used to correct modeling mistakes, change misinterpretations, and add features or functions that were inadvertently omitted.

Requirements modeling principles.

Principle 1. The information domain of a problem must be represented and understood. The information domain encompasses the data that flow into the system (from end users, other systems, or external devices), the data that flow out of the system (via the user interface, network interfaces, reports, graphics, and other means), and the data stores that collect and organize persistent data objects (i.e., data that are maintained permanently).

Principle 2. The functions that the software performs must be defined. Software functions provide direct benefit to end users and also provide internal support for those features that are user visible. Some functions transform data that flow into the system. In other cases, functions effect some level of control over internal software processing or external system elements. Functions can be described at many different levels of abstraction, ranging from a general statement of purpose to a detailed description of the processing elements that must be invoked.

Principle 3. The behavior of the software (as a consequence of external events) must be represented. The behavior of computer software is driven by its interaction with the external environment. Input provided by end users, control data provided by an external system, or monitoring data collected over a network all cause the software to behave in a specific way.

Principle 4. The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion. Requirements modeling is the first step in software engineering problem solving. It allows you to better understand the problem and establishes a basis for the solution (design). Complex problems are difficult to solve in their entirety. For this reason, you should use a divide-and-conquer strategy. A large, complex problem is divided into subproblems until each subproblem is relatively easy to understand. This concept is called partitioning or separation of concerns, and it is a key strategy in requirements modeling.

Principle 5. The analysis task should move from essential information toward implementation detail. Requirements modeling begins by describing the problem from the end-user's perspective. The "essence" of the problem is described without any consideration of how a solution will be

implemented. For example, a video game requires that the player “instruct” its protagonist on what direction to proceed as she moves into a dangerous maze. That is the essence of the problem. Implementation detail (normally described as part of the design model) indicates how the essence will be implemented. For the video game, voice input might be used. Alternatively,

Design Modeling Principles

Principle 1. Design should be traceable to the requirements model. The requirements model describes the information domain of the problem, user-visible functions, system behavior, and a set of requirements classes that package business objects with the methods that service them. The design model translates this information into an architecture, a set of subsystems that implement major functions, and a set of components that are the realization of requirements classes. The elements of the design model should be traceable to the requirements model.

Principle 2. Always consider the architecture of the system to be built. Software architecture is the skeleton of the system to be built. It affects interfaces, data structures, program control flow and behavior, the manner in which testing can be conducted, the maintainability of the resultant system, and much more. For all of these reasons, design should start with architectural considerations. Only after the architecture has been established should component-level issues be considered.

Principle 3. Design of data is as important as design of processing functions. Data design is an essential element of architectural design. The manner in which data objects are realized within the design cannot be left to chance. A well-structured data design helps to simplify program flow, makes the design and implementation of software components easier, and makes overall processing more efficient.

Principle 4. Interfaces (both internal and external) must be designed with care. The manner in which data flows between the components of a system has much to do with processing efficiency, error propagation, and design simplicity. A well-designed interface makes integration easier and assists the tester in validating component functions.

Principle 5. User interface design should be tuned to the needs of the end user. However, in every case, it should stress ease of use. The user interface is the visible manifestation of the software. No matter how sophisticated its internal functions, no matter how comprehensive its data structures, no matter how well designed its architecture, a poor interface design often leads to the perception that the software is “bad.”

Principle 6. Component-level design should be functionally independent. Functional independence is a measure of the “single-mindedness” of a software component. The functionality that is delivered by a component should be cohesive—that is, it should focus on one and only one function or subfunction.

Principle 7. Components should be loosely coupled to one another and to the external environment. Coupling is achieved in many ways— via a component interface, by messaging, through global data. As the level of coupling increases, the likelihood of error propagation also increases and the overall maintainability of the software decreases. Therefore, component coupling should be kept as low as is reasonable.

Principle 8. Design representations (models) should be easily understandable. The purpose of design is to communicate information to practitioners who will generate code, to those who will test the software, and to others who may maintain the software in the future. If the design is difficult to understand, it will not serve as an effective communication medium.

Principle 9. The design should be developed iteratively. With each iteration, the designer should strive for greater simplicity. Like almost all creative activities, design occurs iteratively. The first iterations work to refine the design and correct errors,

1.9.4 Construction Principles

The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user. coding may be

- (1) the direct creation of programming language source code (e.g., Java),
- (2) the automatic generation of source code using an Intermediate design like representation of the component to be built.

The initial focus of testing is at the component level, often called **unit testing**.

Other levels of testing include

- i) **integration testing** (conducted as the system is constructed).
- ii) **validation testing** that assesses whether requirements have been met for the complete system (or software increment)
- iii) **acceptance testing** that is conducted by the customer in an effort to exercise all required features and functions.

The following set of fundamental principles and concepts are applicable to coding and testing:

Coding Principles. The principles that guide the coding task are closely aligned with programming style, programming languages, and programming methods. However, there are a number of fundamental principles that can be stated:

Preparation principles: Before you write one line of code, be sure you

- Understand of the problem you're trying to solve.
- Understand basic design principles and concepts.
- Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
- Select a programming environment that provides tools that will make your work easier.
- Create a set of unit tests that will be applied once the component you code is completed.

Programming principles: As you begin writing code, be sure you

- Constrain your algorithms by following structured programming practice.
- Consider the use of pair programming.
- Select data structures that will meet the needs of the design.
- Understand the software architecture and create interfaces that are consistent with it.
- Keep conditional logic as simple as possible.
- Create nested loops in a way that makes them easily testable.
- Select meaningful variable names and follow other local coding standards.
- Write code that is self-documenting.
- Create a visual layout (e.g., indentation and blank lines) that aids understanding.

Validation Principles: After you've completed your first coding pass, be sure you

- Conduct a code walkthrough when appropriate.
- Perform unit tests and correct errors you've uncovered.
- Refactor the code.

Testing Principles: Glen Myers states a number of rules that can serve well as testing objectives:

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has a high probability of finding an as-yet undiscovered error.
- A successful test is one that uncovers an as-yet-undiscovered error.

Davis suggests a set of testing principles that have been adapted for use.

Principle 1. All tests should be traceable to customer requirements. The objective of software testing is to uncover errors. It follows that the most severe defects (from the customer's point of view) are those that cause the program to fail to meet its requirements.

Principle 2. Tests should be planned long before testing begins. Test planning can begin as soon as the requirements model is complete. Detailed definition of test cases can begin as soon as the design model has been solidified. Therefore, all tests can be planned and designed before any code has been generated.

Principle 3. The Pareto principle applies to software testing. In this context the Pareto principle implies that 80 percent of all errors uncovered during testing will likely be traceable to 20 percent of all program components. The problem, of course, is to isolate these suspect components and to thoroughly test them.

Principle 4. Testing should begin “in the small” and progress toward testing “in the large.” The first tests planned and executed generally focus on individual components. As testing progresses, focus shifts in an attempt to find errors in integrated clusters of components and ultimately in the entire system.

Principle 5. Exhaustive testing is not possible. The number of path permutations for even a moderately sized program is exceptionally large. For this reason, it is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component-level design have been exercised.

Deployment Principles

Principle 1. Customer expectations for the software must be managed. Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately. This results in feedback that is not productive and ruins team morale. In her book on managing expectations, Naomi Karten states: “The starting point for managing expectations is to become more conscientious about what you communicate and how.” She suggests that a software engineer must be careful about sending the customer conflicting messages (e.g., promising more than you can reasonably deliver in the time frame provided or delivering more than you promise for one software increment and then less than promised for the next).

Principle 2. A complete delivery package should be assembled and tested. A CD-ROM or other media (including Web-based downloads) containing all executable software, support data files, support documents, and other relevant information should be assembled and thoroughly beta-tested with actual users. All installation scripts and other operational features should be thoroughly exercised in as many different computing configurations (i.e., hardware, operating systems, peripheral devices, networking arrangements) as possible.

Principle 3. A support regime must be established before the software is delivered. An end user expects responsiveness and accurate information when a question or problem arises. If support is ad hoc, or worse, nonexistent, the customer will become dissatisfied immediately. Support should be planned, support materials should be prepared, and appropriate recordkeeping mechanisms should be established so that the software team can conduct a categorical assessment of the kinds of support requested.

Principle 4. Appropriate instructional materials must be provided to end users. The software team delivers more than the software itself. Appropriate training aids (if required) should be developed; troubleshooting guidelines should be provided, and when necessary, a “what’s different about this software increment” description should be published.

Principle 5. Buggy software should be fixed first, delivered later. Under time pressure, some software organizations deliver low-quality increments with a warning to the customer those bugs “will be fixed in the next release.” This is a mistake. There’s a saying in the software business: “Customers will forget you delivered a high-quality product a few days late, but they will never forget the problems that a low-quality product caused them. The software reminds them every day.”

MODULE-4

1.1 INTRODUCTION TO SOFTWARE PROJECT MANAGEMENT

1. Software Project Management is an art & Science of planning & leading software Projects from **ideas to reality**.
2. A Software Project is the complete procedure of software development from requirement gathering to testing and maintenance, carried out according to the execution methodologies, in a specified period of time to achieve intended software product
3. Project management is the discipline of defining and achieving targets while optimizing the new resources (time, money, people, materials, energy, space , etc.) over the course of a project (a set of activities of finite duration).
4. Project management involves the planning, monitoring, and control of people, process, and events that occur during software development.

Everyone manages, but the scope of each person's management activities varies according his or her role in the project.

Software needs to be managed because it is a complex undertaking with a long duration time.

Managers must focus on the fours P's to be successful (people, product, process, and project).

A project plan is a document that defines the four P's in such a way as to ensure a cost effective, high quality software product.

The only way to be sure that a project plan worked correctly is by observing that a high-quality product was delivered on time and under budget.

1.2 WHY IS SOFTWARE PROJECT MANAGEMENT IMPORTANT?

- Large amounts of money are spent on ICT (information and communication technology) e.g. UK government in 2003-04 spent € 2.3 billion on contracts for ICT and only € 1.4 billion on road building. (1 billion =100 crore).
- Project often fail – Standish Group claim only a third of ICT projects are successful. 82 % were late and 43 % exceeded their budget. Poor project management a major factor in these failures.
- The methodology used by the Standish Group to arrive at their findings has been criticized, but the general perception of the prevalence of ICT project failure is still clear.

Software Development Life Cycle:

The Software Development life cycle is a methodology that also forms the framework for planning and controlling the creation, testing, and delivery of an information system.

The software development life cycle concept acts as the foundation for multiple different development and delivery methodologies, such as the Hardware development life -cycle and software development life -cycle . While Hardware development life -cycle deal specially with hardware and Software development life -cycle deal with software, a systems development life -cycle differs from each in that it can deal with any combination of hardware and software , as a system can be composed of hardware only , software only, or a combination of both.

Four Project Dimensions:

- People
- Process
- Product
- Technology

The 5 Variables of Project Control

1. Time: amount of time required to complete the project.
2. Cost: calculated from the time variable
3. Quality: The amount of time put into individual tasks determines the overall quality of the project.
4. Scope: Requirements specified for the end result.
5. Risk: Potential points of failure.

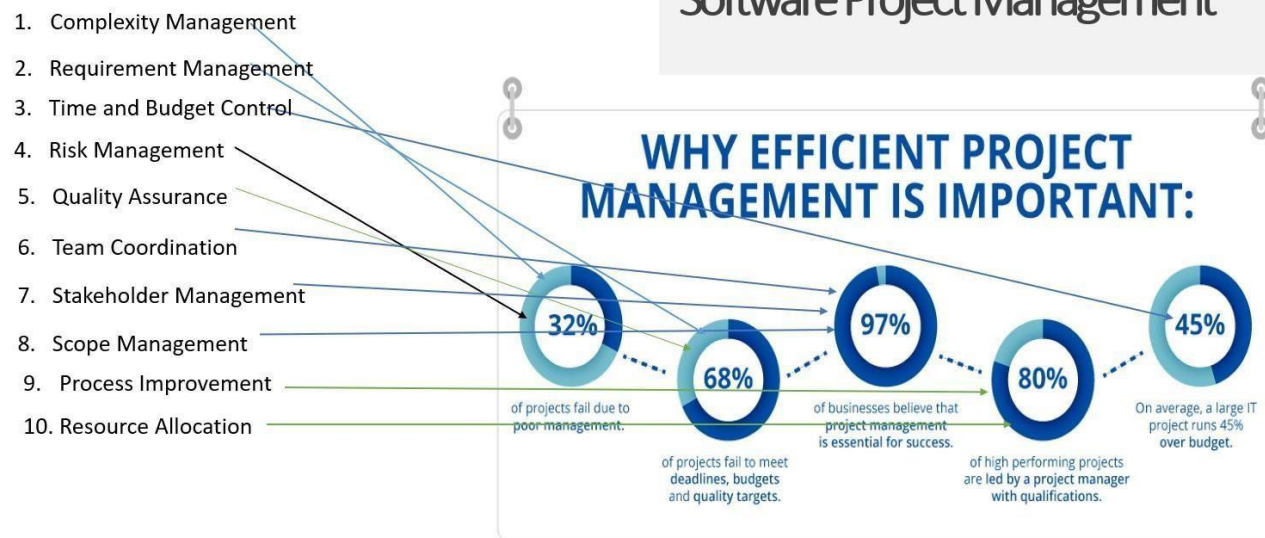
Trade -off Triangle



The triangle illustrates the relationship between three primart forces in a project. Time is the available time to deliver the project. Cost represents the amount of money or resources available and quality represents the fit-to-purpose that the project must achieve to be a scuccess.

The normal situation is that one of thse factors is fixed and the other two will vary in inverse proportion to each other. For example , time is often fixed and the quality of the end product will depend on the cost and resources available. Similarly if you are working to a fixed level of quality then the cost of the project will largely be dependable upon the time available(if you have longer you can do it with fewer people).

Software project management is particularly important due to the unique challenges and complexities associated with software development.



1. Complexity Management

- Software projects often involve intricate systems and interdependencies. Effective management of this complexity ensures that the project remains coherent and manageable.

2. Requirement Management

- Clear and precise requirement management is essential to ensure that the final product meets user needs and expectations. Mismanagement here can lead to scope creep and project failure.

3. Time and Budget Control

- Monitoring and controlling the project timeline and budget is vital. This includes planning, estimating, and adhering to schedules and financial constraints to prevent overruns.

4. Risk Management

- Identifying, assessing, and mitigating risks can prevent unforeseen issues from derailing a project. This proactive approach helps in managing uncertainties effectively.
- 5. Quality Assurance**
 - Ensuring that the project meets quality standards is crucial for user satisfaction and reducing post-release defects. Continuous testing and validation are key practices.
 -
- 6. Team Coordination**
 - Effective communication and coordination among team members are essential for collaboration and timely problem-solving, ensuring that everyone is aligned with project goals.
 -
- 7. Stakeholder Management**
 - Engaging and managing stakeholders helps in gaining their support and addressing their concerns, which is critical for project acceptance and success.
- 8. Scope Management**
 - Defining and controlling what is included in the project prevents scope creep, ensures that all necessary features are delivered, and avoids unnecessary work.
- 9. Process Improvement**
 - Continuously improving processes ensures that the project is using the most efficient methods and practices, leading to better performance and outcomes.
- 10. Resource Allocation**
 - Efficient allocation and management of resources (human, financial, and material) ensure that the project has what it needs to succeed without wastage.

Statistics Highlighting the Importance of Efficient Project Management

- 1. 32% of Projects Fail Due to Poor Management**
 - This statistic underscores the critical impact of project management on the overall success of software projects. Poor management can lead to project failures, highlighting the need for skilled project managers.
- 2. 68% of Projects Fail to Meet Deadlines, Budgets, and Quality Targets**
 - This indicates that a significant majority of projects struggle with time, budget, and quality control. Effective project management practices in these areas can significantly improve success rates.
- 3. 97% of Businesses Believe Project Management is Essential for Success**
 - This near-unanimous belief among businesses highlights the recognized value of project management. It underscores that investing in good project management practices is seen as crucial for achieving business objectives.
- 4. 80% of High-Performing Projects are Led by a Project Manager with Qualifications**
 - This shows a clear correlation between the qualifications of the project manager and the performance of the project. Qualified project managers bring skills and knowledge that drive project success.

5. On Average, a Large IT Project Runs 45% Over Budget

- This statistic points to common budget overruns in large IT projects, emphasizing the need for rigorous budget control and efficient resource management to prevent financial overshooting.

Conclusion

Effective software project management is essential due to the inherent complexities and challenges of software development. The key areas outlined require diligent attention and management to ensure project success. The statistics provided illustrate the high stakes involved and the substantial impact that good project management can have on the success rates of software projects. By focusing on these areas, businesses can significantly improve their chances of delivering successful projects that meet deadlines, stay within budget, and satisfy quality standards.

1.3 WHAT IS A PROJECT:

The definition of a project as being planned assume that to a large extent we can determine how we are going to carry out a task before we start. There may be some projects of an exploratory nature where this might be quite hard. Planning is in essence thinking carefully about something before you do it and even in the case of uncertain projects this is worth doing as long as it is accepted that the resulting plans will have provisional and speculative elements. Other activities, concerning, for example, to routine maintenance, might have been performed so many times that everyone involved knows exactly what needs to be done. In these cases, planning hardly seems necessary, although procedures might need to be documented to ensure consistency and to help newcomers to the job.

Dictionary definitions of ‘project’ include:

- A specific plan or design
- A planned undertaking
- A large undertaking e.g. a public works scheme”

Key points above are planning and size of task

Here are some definitions of ‘project’. No doubt there are other ones: for example,

‘Unique process, consisting of a set of coordinated and controlled activities with start and finish dates, undertaken to achieve an objective conforming to specific requirements, including constraints of time, cost and resources.

There is a hazy boundary between the non-routine project and the routine job. The first time you do a routine task, it will be like a project. On the other hand, a project to develop a system similar to previous ones you have developed will have a large element of the routine.

Jobs versus projects

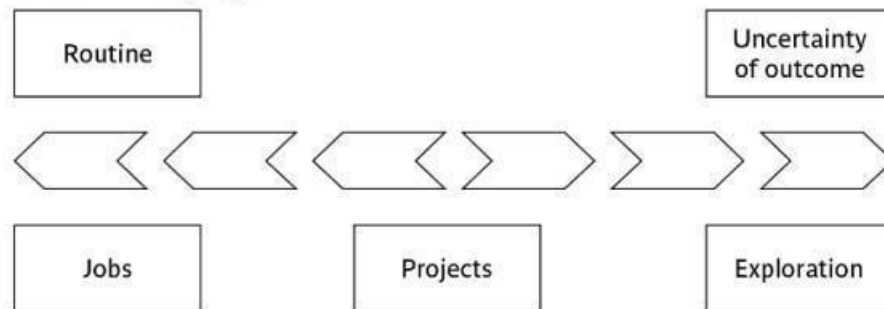


Fig:1.1 Activities most likely to benefit from project management.

1.3.1 Characteristics of Project:

- Non-routine tasks are involved
- Planning is required
- Specific objectives are to be met or a specified product is to be created
- The project has a pre-determined time span
- Work is carried out for someone other than yourself
- Work involves several specialism's
- Work is carried out in several phases
- The resources that are available for use on the project are constrained
- The project is large or complex.

The project that employs 20 developers is likely to be disproportionately more difficult than one with only 20 staff because of the need for additional coordination.

1.3.2 Software Projects versus Other Types of Project:

Many of the techniques of general project management are applicable to software project management. One way of perceiving software project management is as the process of making visible that which is invisible.

Invisibility: When a physical artifact such as a bridge or road is being constructed the progress being made can actually be seen. With software, progress is not immediately visible.

Complexity: Software products contain more complexity than other engineered artifacts.

Conformity: The 'traditional' engineer is usually working with physical systems and physical materials like cement and steel. These physical systems can have some complexity, but are governed by physical laws that are consistent. Software developers have to conform to the requirements of human clients. It is not just that individuals can be inconsistent.

Flexibility: The ease with which software can be changed is usually seen as one of its strengths. However, this means that where the software system interfaces with a physical or organizational system, it is expected that, where necessary, the software will change to accommodate the other components rather than vice versa. This means the software systems are likely to be subject to a high degree of change.

An example for infrastructure project is construction of a flyover. An example for a software project is development of a payroll management system for an organization using Oracle 10g and Oracle Forms 10G.

1.4 CONTRACT MANAGEMENT

- **In-house projects** are where the users and the developers of new software work for the same organization.
 - However, increasingly organizations **contract** out ICT development to **outside developers**. Here, the client organization will often appoint a 'project manager' to supervise the contract who will delegate many technically oriented decisions to the contractors.
 - Thus, the project manager will not worry about estimating the effort needed to write individual software components as long as the overall project is within budget and on time. On the supplier side, there will need to be project managers who deal with the more technical issues.
-
- **Contract management** is the process of managing the creation, execution, and analysis of contracts to maximize operational and financial performance and minimize risk.
 - It involves various activities from the initial request for a contract, through negotiation, execution, compliance, and renewal. Effective contract management ensures that all parties to a contract fulfill their obligations as efficiently as possible.



Various Stages of Contract Management

1. Request and Creation:

Request: Identifying the need for a contract and gathering the necessary information to draft it.

Creation: Drafting the contract terms and conditions that align with the requirements and objectives of all parties involved.

Example: A software company needs to hire a third-party developer to work on a new project.

The project manager identifies the need for a contract and gathers details about the scope of work, timelines, payment terms, and other specifics.

2. Negotiation:

Parties involved discuss and negotiate the terms of the contract to reach a mutual agreement. This stage often involves revisions and adjustments.

Example: The software company and the third-party developer negotiate the terms. The developer might request more time or a higher payment, while the company might request milestones for progress checks.

3. Approval and Execution:

Approval: Obtaining necessary approvals from stakeholders and legal departments.

Execution: Signing the contract, making it a legally binding document.

Example: Once the terms are finalized, the contract is reviewed by both parties' legal teams. After approval, both the software company and the developer sign the contract.

4. Obligations and Performance:

Ensuring that all parties adhere to the terms and conditions agreed upon in the contract. Monitoring performance and compliance.

Example: The developer starts working on the project, adhering to the deadlines and deliverables specified in the contract. The software company provides the necessary resources and makes payments as per the contract.

5. Modification and Renewal:

Making necessary amendments if any changes occur during the contract period. Reviewing and renewing contracts as needed.

Example: Midway through the project, the software company requests additional features not covered in the original contract. An amendment is made to include these new features and adjust the payment terms accordingly. As the project nears completion, the company and developer may negotiate a renewal for ongoing maintenance.

6. Closure:

Completing all contractual obligations, ensuring all parties have met their requirements, and formally closing the contract.

Example: The developer finishes the project, and the software company conducts a final review to ensure all deliverables meet the agreed-upon standards. Once confirmed, the contract is closed, and a final payment is made.

Benefits of Effective Contract Management

Risk Mitigation: Identifies and manages potential risks early in the contract lifecycle.

Improved Compliance: Ensures that all parties comply with legal and regulatory requirements.

Cost Savings: Avoids unnecessary costs and penalties by managing contracts efficiently.

Performance Tracking: Monitors performance against contract terms to ensure objectives are met.

Relationship Management: Maintains positive relationships between contracting parties through clear and consistent communication.

Speed to Market: Accelerates project timelines by leveraging the vendor's expertise and resources.

COMPANY: XYZ Tech

1. **Identifying Needs:** XYZ Tech identifies a need for a mobile app to complement its existing software suite.
2. **Selecting a Vendor:** XYZ Tech shortlists several development firms based in India, known for their expertise in mobile app development.
3. **Defining Requirements:** XYZ Tech provides detailed specifications, including desired features, user interface design, and performance metrics.
4. **Contract Negotiation:** XYZ Tech negotiates terms, focusing on deliverables, timelines, and confidentiality clauses.
5. **Project Management:** XYZ Tech assigns a project manager to liaise with the vendor, ensuring regular updates and adherence to milestones.

6. **Delivery and Integration:** The vendor delivers the app, which is integrated with XYZ Tech's software suite after thorough testing.
7. **Post-Delivery Support:** The vendor provides ongoing maintenance and support, addressing any post-launch issues. By following these steps and learning from real-world examples, software companies can effectively outsource projects to thirdparty vendors, ensuring successful project completion and maximizing business value.

1.5 ACTIVITIES COVERED BY SOFTWARE PROJECT MANAGEMENT(hemanth gowda m)

The activities covered by Software Project management are diagrammatically illustrated as follows:

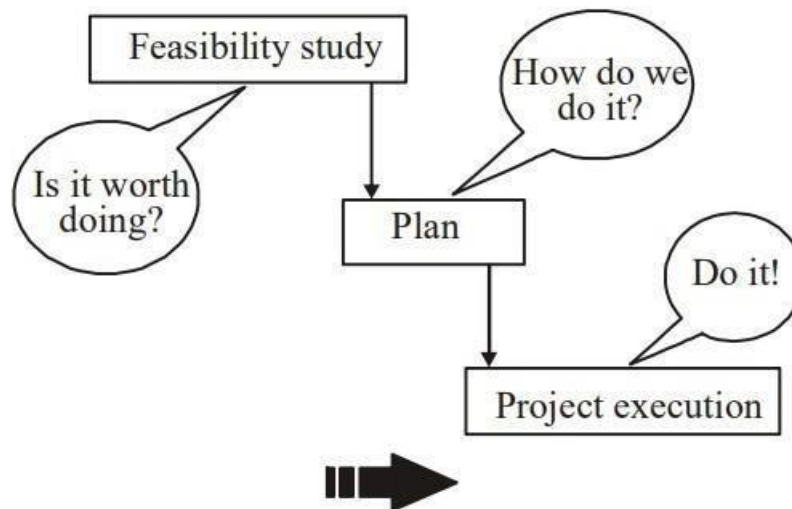


Figure 1.1: The Feasibility Study / Plan / Execution Cycle

1.5.1 The Feasibility Study:

This is an investigation into whether a prospective project is worth starting that it has a valid *business case*. Information is gathered about the requirements of the proposed application. The probable developmental and operational costs, along with the value of the benefits of the new system, are estimated. The study could be part of a strategic planning exercise examining and prioritizing a range of potential software developments.

1.5.2 Planning:

If the feasibility study produces results which indicate that the prospective project appears viable, planning of the project can take place. However, for a large project, we would not do all our detailed planning right at the beginning. We would formulate an outline plan for the whole project and a detailed one for the first stage. More detailed planning of the later stages would be done as they approached. This is because we would have more detailed and accurate information upon which to base our plans nearer to the start of the later stages.

1.5.3 Project Execution:

The project can now be executed. The execution of a project often contains *design* and *implementation* subphases. The same is illustrated in **Figure 1.2** which shows the typical sequence of software development activities recommended in the international standard ISO 12207.

1.5.3.1 Requirements Analysis:

This starts with requirement elicitation or requirement gathering which establishes what the users require of the system that the project is to implement. Some work along these lines will almost certainly have been carried out when the project was evaluated, but now the original information obtained needs to be updated and supplemented.

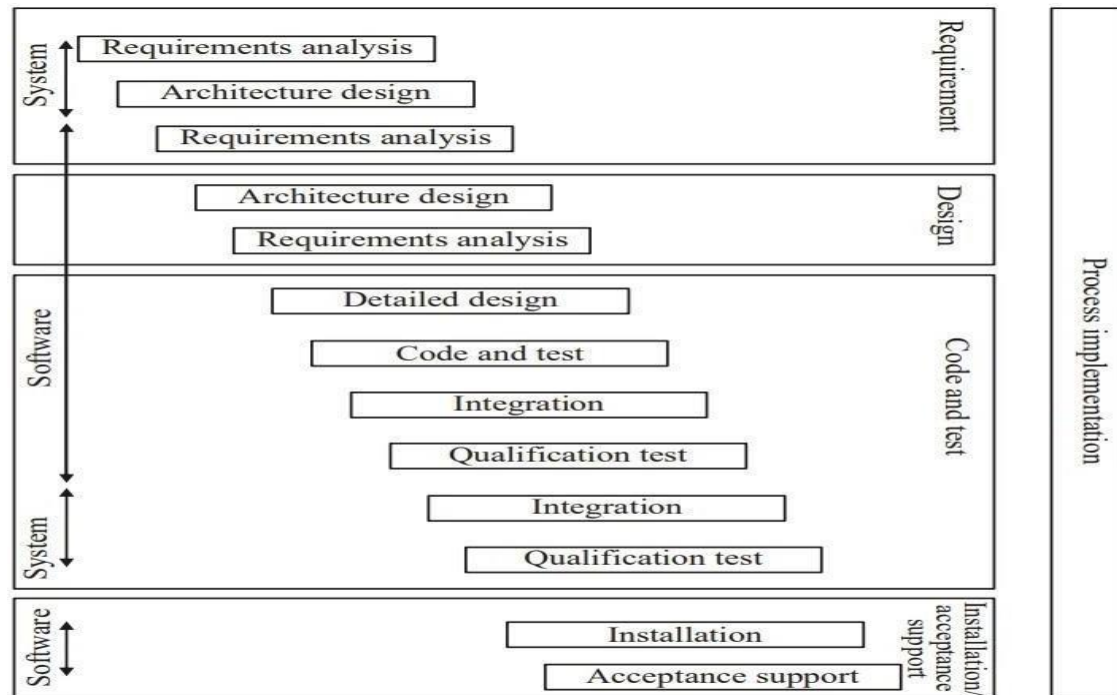


Figure 1.2: Sub Phases in Project Execution

1.5.3.2 Specification:

Detailed documentation of what the proposed system is to do.

1.5.3.3 Design:

A design has to be drawn up which meets the specification. This design will be in two stages. One will be the external or user design concerned with the external appearance of the application. The other produces the physical design which tackles the way that the data and software procedures are to be structured internally.

- **Architecture Design:** This maps the requirements to the components of the system that is to be built. At the system level, decisions will need to be made about which processes in the new system will be carried out by the user and which can be computerized. This design of the system architecture thus forms an input to the development of the software requirements. A second architecture design process then takes place which maps the software requirements to software components.
- **Detailed Design:** Each software component is made up of a number of software units that can be separately coded and tested. The detailed design of these units is carried out separately.

1.5.3.4 Coding:

This may refer to writing code in a procedural language or an object-oriented language or could refer to the use of an application-builder. Even where software is not being built from scratch, some modification to the base package could be required to meet the needs of the new application.

1.5.3.5 Testing (Verification and Validation):

Whether software is developed specially for the current application or not, careful testing will be needed to check that the proposed system meets its requirements.

Integration: The individual components are collected together and tested to see if they meet the overall requirements. Integration could be at the level of software where different software components are combined, or at the level of the system as a whole where the software and other components of the system such as the hardware platforms and networks and the user procedures are brought together.

Qualification Testing: The system, including the software components, has to be tested carefully to ensure that all the requirements have been fulfilled.

1.5.3.6 Implementation/ Installation:

Some system development practitioners refer to the whole of the project after design as 'implementation' (that is, the implementation of the design) while others insist that the term refers to the installation of the system after the software has been developed.

1.5.3.7 Acceptance Support:

Once the system has been implemented there is a continuing need for the correction of any errors that may have crept into the system and for extensions and improvements to the system. Maintenance and support activities may be seen as a series of minor software projects.

1.6 PLANS, METHODS AND METHODOLOGIES(gokul)

A plan for an activity must be based on some idea of a *method of work*. To take a simple example, if you were asked to test some software, even though you do not know anything about the software to be tested, you could assume that you would need to:

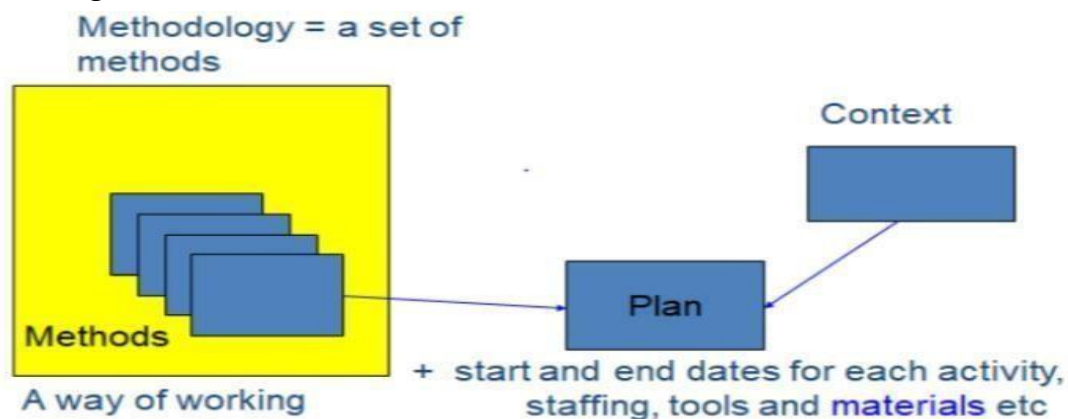
- Analyze the requirements for the software
- Devise and write test cases that will check that each requirement has been satisfied

- Create test scripts and expected results for each test case
- Compare the actual results and the expected results and identify discrepancies

While a method relates to a type of activity in general, a plan takes that method (and perhaps others) and converts it to real activities, identifying for each activity:

- Its start and end dates
- Who will carry it out?
- What tools and materials will be used?

‘Materials’ in this context could include information, for example a requirements document. With complex procedures, several methods may be deployed, in sequence or in parallel. The output from one method might be the input to another. Groups of methods or techniques are often referred to as methodologies.



1.7 SOME WAYS OF CATEGORIZING SOFTWARE PROJECTS

Distinguishing different types of projects is important as different types of tasks need different project approaches e.g.

- Changes to the characteristics of software projects
- Voluntary systems (such as computer games) versus compulsory systems e.g. the order processing system in an organization
- Information systems versus embedded systems.
- Software Products verses services
- Product-development versus outsourced.
- Object-driven development.

1.7.1 Changes to the characteristics of software projects

- Over the last few decades, the characteristics of software projects have undergone drastic changes. In earlier days of software development every software was being written from scratch and there was no code reusability. In contrast, at present almost every programming language supports ways of reusing existing code, by customizing and extending existing code, efficiently and dynamically linking library routines and support for frameworks.
- Project durations have now shrunk to only a few months compared to multi-year projects.
- In past, customer participation in software projects was largely restricted to only initial interactions, gathering and specification and taking delivery of the developed software, but at present, customer participation in almost every aspect of a project.

1.7.2 Compulsory versus voluntary users

In workplaces there are systems that staff have to use if they want to do something, such as recording a sale. However, use of a system is increasingly voluntary, as in the case of computer games. Here it is difficult to elicit precise requirements from potential users as we could with a business system. What the game will do will thus depend much on the informed ingenuity of the developers, along with techniques such as market surveys, focus groups and prototype evaluation.

1.7.3 Information Systems versus Embedded systems

A traditional distinction has been between information systems which enable staff to carry out office processes and *embedded systems* which control machines. A stock control system would be an information system. An embedded, or process control, system might control the air conditioning equipment in a building. Some systems may have elements of both where, for example, the stock control system also controls an automated warehouse.

1.7.4 Software Products verses services

All types of software projects can broadly be classified into software product development projects and software services projects. It can be further classified as shown in below Fig.1.7 A software product development concerns developing the software by keeping the requirements to the general customers in mind and developed software is usually sold-off-the

shelf to a large number of customers. Examples of generic software development are Microsoft's Windows operating system and Oracle Corporation's Oracle 8i database management software. Domain-specific software targets specific segments of customers (verticals) Example BANCS from TCS. FINACLE from Infosys.

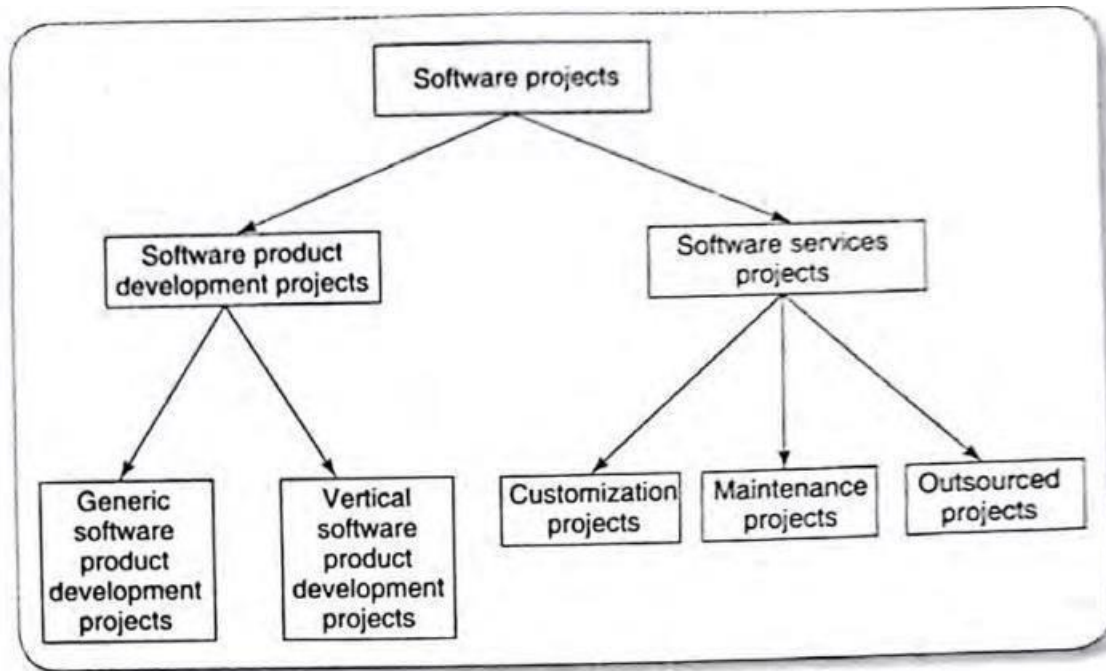


Fig: 1.7 A Classification of software projects

Software services cover a large gamut of software projects such as customization, outsourcing, maintenance, testing and consultancy.

Projects may be distinguished by whether their aim is to produce a product or to meet certain objectives.

1.7.5 Outsourced Projects

While developing a large project, it makes good commercial sense for a company to outsource some parts of its work to other companies.

For example, A company may consider outsourcing as a good option, if it feels that it does not have sufficient expertise to develop some specific parts of the product or if it determines that some parts can be developed cost-effectively another company.

1.7.6 Object-driven development

Projects may be distinguished by whether their aim is to produce or to meet certain objective.

Many software projects have two stages, First is an object-driven project resulting in recommendations which identify the need for a new software system and next stage is a project actually to create the software product.

1.8 STAKEHOLDERS(Bharath)

These are people who have a stake or interest in the project. It is important that they be identified as early as possible, because you need to set up adequate communication channels with them right from the start. The project leader also has to be aware that not everybody who is involved with a project has the same motivation and objectives. The end-users might, for instance, be concerned about the ease of use of the system while their managers might be interested in the staff savings the new system will allow.

Boehm and Ross proposed a '*Theory W*' of software project management where the manager concentrates on creating the role and format situations where all parties benefit from a project and therefore have an of communication interest in its success. (The 'W' stands for 'win-win'.)

Stakeholders might be internal to the project team, external to the project team but in the same organization, or totally external to the organization.

- **Internal to the project team:** This means that they will be under the direct managerial control of the project leader.
- **External to the project team but within the same organization:** For example, the project leader might need the assistance of the information management group in order to add some additional data types to a database or the assistance of the users to carry out systems testing. Here the commitment of the people involved has to be negotiated.
- **External to both the project team and the organization:** External stakeholders may be customers (or users) who will benefit from the system that the project implements or contractors who will carry out work for the project. One feature of the relationship with these people is that it is likely to be based on a legally binding contract.

Different types of Stakeholders may have different objectives and one of the jobs of the successful project leader is to recognize these different interests and to be able to reconcile them. It should therefore come as no surprise that the project leader needs to be a good communicator and negotiator.

1.9 SETTING OBJECTIVES

- The objectives should define what the project team must achieve for project success.
- Objectives focus on the desired outcomes of the project rather than the tasks within it-they are the ‘post-conditions’ of the project.
- Objectives could be set of statements following the opening words ‘*the project will be a success if*’.
- To have a successful software project, the manager and the project team members must know what will constitute success. This will make them concentrate on what is essential to project success.
- There may be several sets of users of a system and there may be several different groups of specialists involved its development. There is a need for well-defined objectives that are accepted by all these people. Where there is more than one user group, a project authority needs to be identified which has overall authority over what the project is to achieve.
- This authority is often held by a *project steering committee* (or project board or project management board) which has overall responsibility for setting, monitoring and modifying objectives. The project manager still has responsibility for running the project on a day-to-day basis, but has to report to the steering committee at regular intervals. Only the steering committee can authorize changes to the project objectives and resources.

1.9.1 Sub-objectives and Goals:

Setting objectives can guide and motivate individuals and groups of staff. An effective objective for an individual must be something that is within the control of that individual. An objective might be that the software application to be produced must pay for itself by reducing staff costs over two years. As an overall business objective this might be reasonable. For software developers it would be unreasonable as, though they can control development costs, any reduction in operational staff costs depends not just on them but on the operational management after the application has ‘gone live’. What would be appropriate would be to set a goal or sub-objective for the software developers to keep development costs within a certain budget.

Thus, objectives will need be broken down into goals or sub-objectives. Here we say that in order to achieve the objective we must achieve certain goals first. These goals are steps on the way to achieving an objective, just as goals scored in a football match are steps towards the objective of winning the match.

The mnemonic SMART is sometimes used to describe well defined objectives:

- **Specific:** Effective objectives are *concrete and well defined*. Vague aspirations such as 'to improve customer relations' are unsatisfactory. Objectives should be defined in such a way that it is obvious to all whether the project has been successful or not.
- **Measurable:** Ideally there should be measures of effectiveness which tell us how successful the project has been. For example, 'to reduce customer complaints' would be more satisfactory as an objective than 'to improve customer relations'. The measure can, in some cases, be an answer to simple yes/no questions, e.g. 'Can we install the new software by 1 November 2011?'
- **Achievable:** It must be within the power of the individual or group to achieve the objective.
- **Relevant:** The objective must be relevant to the true purpose of the project.
- **Time constrained:** There should be a defined point in time by which the objective should have been achieved.

1.9.2 Measures of effectiveness

Measures of effectiveness provide practical methods of ascertaining whether an objective has been met. 'Mean time between failures' (mtbf) is used to measure reliability. A measure of effectiveness will usually be related to the installed operational system.

1.10 BUSINESS CASE

- Most projects need to have a justification or business case: the effort and expense of pushing the project through must be seen to be worthwhile in terms of the benefits that will eventually be felt.
- The quantification of benefits will often require the formulation of a business model which explains how the new application can generate the claimed benefits.

Any project plan must ensure that the business case is kept intact. For example:

- The development costs are not allowed to rise to a level which threatens to exceed the value of benefits.
- The features of the system are not reduced to a level where the expected benefits cannot be realized.

- The delivery date is not delayed so that there is an unacceptable loss benefit.

1.11 PROJECT SUCCESS AND FAILURE(atul kumar)

- The project plan should be designed to ensure project success preserving the business case for the project.
- Different stakeholders have different interests, some stakeholders in a project might see it as a success while others do not.
- The project objectives are the targets that the project team is expected to achieve—They are summarized as delivering:
 - The agreed functionality
 - To the required level of quality
 - In time
 - Within budget
- A project could meet these targets but the application, once delivered could fail to meet the business case. A computer game could be delivered on time and within budget, but might then not sell.
- In business terms, the project is a success if the value of benefits exceeds the costs.
- A project can be a success on delivery but then be a business failure, On the other hand, a project could be late and over budget, but its deliverables could still, over time, generate benefits that outweigh the initial expenditure.
- The possible gap between project and business concerns can be reduced by having a broader view of projects that includes business issues.
- **Technical learning** will increase costs on the earlier projects, but later projects benefit as the learnt technologies can be deployed more quickly cheaply and accurately.
- **Customer relationships** can also be built up over a number of projects. If a client has trust in a supplier who has done satisfactory work in the past, they are more likely to use that company again.

1.12 MANAGEMENT AND MANAGEMENT CONTROL

1.12.1 MANAGEMENT:

Management involves following activities:

- Planning - deciding what is to be done;

- Organizing - making arrangements;
- Staffing - selecting the right people for the job etc.;
- Directing - giving instructions;
- Monitoring - checking on progress;
- Controlling - taking action to remedy hold-ups;
- Innovating - coming up with new solutions;
- Representing - liaising with clients, users, developer, suppliers and other stakeholders.

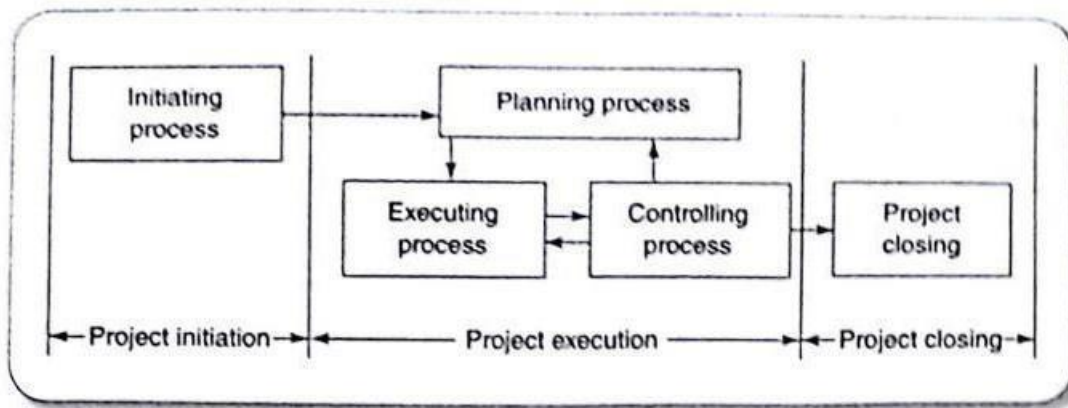


FIGURE 1.5 Principal project management processes

Much of the project manager's time is spent only in three activities, i.e. Project Planning, Monitoring and control. This time period during which these activities are carried out is indicated in Fig 1.5.

It shows that project management is carried out over three well-defined stages or processes irrespective of the methodology used.

In the **Project initiation stage**, an initial plan is made. As a project starts, **the project is monitored and controlled to process as planned**. Initial plan is revised periodically to accommodate additional details and constraints about the project as they become available. Finally, the **project is closed**.

Initial project is undertaken immediately after the feasibility study phase and before starting the requirement analysis and specification process.

Initial project planning involves estimating several characteristics of a project. Based on these estimates all subsequent project activities are planned.

The **monitoring activity** involves monitoring the progress of the project. **Control activities** are initiated to minimize any significant variation in the plan,

Project Planning is an important responsibility of the project Manager. During project planning, the project manger needs to perform a few well-defined activities that have been outlined below/ Several best practices have been proposed for software project planning activities, PRINCE2 is used extensively in UK and Europe . In USA Project management Institute's 'PMBOK' which refers to their publication "A Gude to the Project Management Body of knowledge, is used.

- **Estimation:** The following project attributes are estimated.
- **Cost:** How much is it going to cost to complete the project.
- **Duration:** How long is it going to take to complete the project.
- **Effort:** How much effort would be necessary for completing the project?

The effectiveness of all activities such as scheduling and staffing are planned at later stage.

- **Scheduling:** Based on estimations of effort and duration, the schedules for manpower and other resources are developed.
- **Staffing:** Staff organization and staffing plans are made.
- **Risk Management:** This activity includes risk identification, analysis, and abatement planning.
- **Miscellaneous Plans:** This includes making several other plans such as quality assurance plan, configuration management plan etc.

While carrying out project monitoring and control activities, a project manager may sometimes find it necessary to change the plan to cope with specific situations and make the plan more accurate as more project data becomes available.

1.12.2 MANAGEMENT CONTROL

Management involves setting objectives for a system and monitoring the performance of the system.

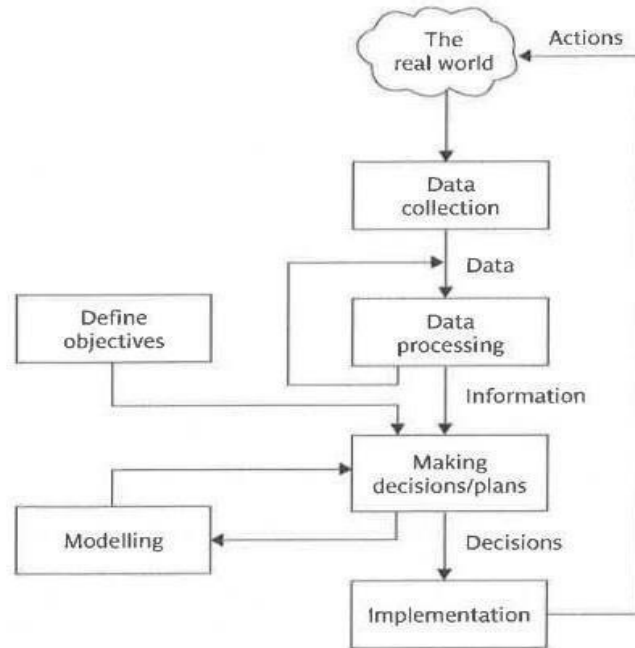


Fig: The Project control cycle

- In the above Fig, local managers involve in data collection. Bare details such as “location X has processed 2000 documents” may not be useful to higher management.
- Data processing is required to transform this raw data into useful information. This might be in such forms as “Percentage of records Processed”, average documents per day per person”, and estimated completion date”.
- In this example , the project management might examine the “estimated completion date” for completing data transfer for each branch. They are comparing actual performance with overall project objectives.
- They might find that one or two branches will fail to complete the transfer of details in time.
- It can be seen that a project plan is dynamic and will need constant adjustment during the execution of the project. A good plan provides a foundation for a good project, but is nothing without intelligent execution.

1.13 PROJECT MANAGEMENT LIFE CYCLE(farahaan)

Software development life cycle denotes (SDLC) the stages through which a software is developed. In contrast to SDLC, the project management life cycle typically starts well before the software development activities start and continues for the entire duration of SDLC. (Fig 1.7)

In Project Management process, the project manager carries out project initiation, planning, execution, monitoring, controlling and closing.

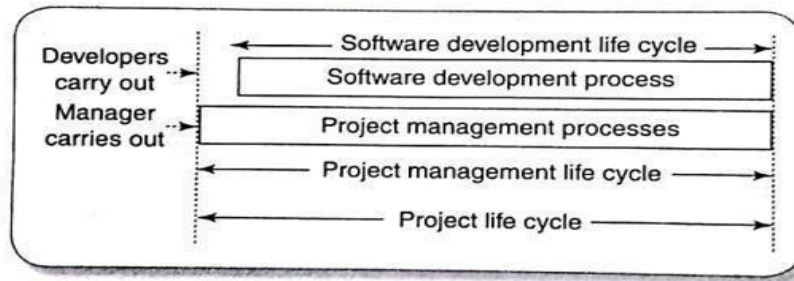


FIGURE 1.7 Project management life cycle versus software development life cycle

The different phases of the project management life cycle are shown in Fig: 1.8.

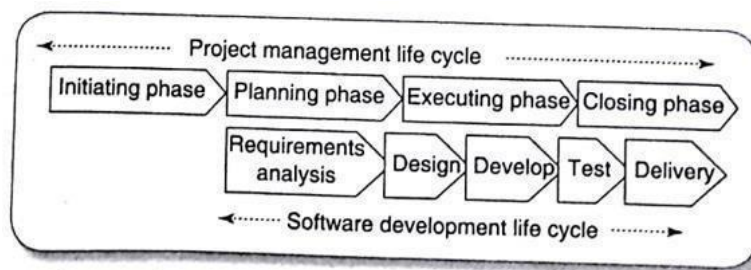


FIGURE 1.8 Different phases of project management life cycle and software development life cycle

1. **Project Initiation:** The project initiation phase starts with project **concept development**. During concept development the different characteristics of the software to be developed are thoroughly understood, which includes, the scope of the project, the project constraints, the cost that would be incurred and the benefits that would accrue. Based on this understanding, a **feasibility study** is undertaken to determine the project would be financially and technically feasible.
Based on feasibility study, the **business case** is developed. Once the top management agrees to the business case, the **project manager** is appointed, the **project charter** is written and finally **project team** is formed. This sets the ground for the manager to start the **project planning phase**.

W5HH Principle: Barry Boehm, summarized the questions that need to be asked and answered in order to have an understanding of these project characteristics.

- Why is the software being built?
- What will be done?
- When will it be done?
- Who is responsible for a function?

- Where are they organizationally located?
- How will the job be done technically and managerially?
- How much of these each resource is needed.

2. **Project Bidding:** Once the top management is convinced by the business case, the **project charter** is developed. For some categories of projects, it may be necessary to have formal bidding process to select suitable vendor based on some cost-performance criteria. The different types of bidding techniques are:

- **Request for quotation(RFQ) :** An organization advertises an RFQ if it has good understanding of the project and the possible solutions.
- **Request for Proposal(RFP) :** An organization had reasonable understanding of the problem to be solved, however, it does not have good grasp of the solution aspects. i.e. may not have sufficient knowledge about different features to be implemented. The purpose of RFP is to get an understanding of the alternative solutions possible that can be deployed and not vendor selection. Based on the RFP process, the requesting organization can form a clear idea of the project solutions required, based on which it can form a statement work (SOW) for requesting RFQ for the vendors.
- **Request for Information (RFI):** An organization soliciting bids may publish an RFI. Based on the vendor response to the RFI, the organization can assess the competencies of the vendors and shortlist the vendors who can bid for the work.

3. **Project Planning:** An importance of the project initiation phase is the project charter. During the project planning the project manger carries out several processes and creates the following documents:

- **Project plan:** This document identifies the project the project tasks and a schedule for the project tasks that assigns project resources and time frames to the tasks.
- **Resource Plan:** It lists the resources , manpower and equipment that would be required to execute the project.
- **Functional Plan:** It documents the plan for manpower, equipment and other costs.
- **Quality Plan:** Plan of quality targets and control plans are included in this document.

- **Risk Plan:** This document lists the identification of the potential risks, their prioritization and a plan for the actions that would be taken to contain the different risks.
4. **Project Execution:** In this phase the tasks are executed as per the project plan developed during the planning phase. Quality of the deliverables is ensured through execution of proper processes. Once all the deliverables are produced and accepted by the customer, the project execution phase completes and the project closure phase starts.
 5. **Project Closure:** Project closure involves completing the release of all the required deliverables to the customer along with the necessary documentation. All the Project resources are released and supply agreements with the vendors are terminated and all the pending payments are completed. Finally, a postimplementation review is undertaken to analyze the project performance and to list the lessons for use in future projects.

1.14 TRADITIONAL VERSUS MODERN MANAGEMENT PRACTICES

Over the last two decades, the basic approach taken by the software industry to develop software has undergone a radical change.

Software is not developed from scratch any more, Software development projects are based on either tailoring some existing product or reusing certain pre-built libraries both will maximize code reuse and compression of project durations.

Other goals include facilitating and accommodating client feedback and client feedbacks and customer participation in project development work and incremental delivery of the product with evolving functionality.

Some Important difference between modern management practices and traditional practices are:

- **Planning Incremental Delivery:** Earlier, projects were simpler and therefore more predictable than the present-day projects. In those days, projects were planned with sufficient detail much before the actual project execution started. After the project initiation, monitoring and control activities were carried out to ensure that the project execution proceeded as per plan, Now, the projects are required to be completed over a much shorter duration, and rapid application development and deployment are considered key strategies.
Instead of making a long-term project completion plan, the project manger now plans all incremental deliveries with evolving functionalities. This type of project management is

often called **extreme project management**. Extreme project management is highly flexible approach that concentrates on human side of project management(e.g. managing project stakeholders).

- **Quality Management:** Customer awareness about product quality has increased significantly. The key responsibility of a project manager now includes assessment of project progress and tracking the quality of all intermediate artifacts.
- **Change Management:** Earlier, when the requirements were signed off by the customer, any changes to the requirements were rarely entertained. Customer suggestions are now actively solicited and incorporated throughout the development process. To facilitate customer feedback, **incremental delivery models** are popularly being used. Product development is being carried out through a series of product versions implementing increasingly greater functionalities. The Project manager plays a key role in product base lining and version control. This has made change management a crucial responsibility of the project manager. Change Management is also known as configuration management.
- **Requirement Management:** In older development methodologies , the requirements had to be identified upfront and these were ‘signed off’ by the customer and frozen before the development could start. At present , in most projects, the requirements change frequently during the development cycle. **Requirement management** has therefore become a systematic process of controlling changes, documenting , analyzing, tracing, prioritizing requirements and then communicating the changes to the relevant stakeholders.
- **Release Management:** Release management concerns planning, prioritizing and controlling the different releases of a software. Modern development processes such as Agile development processes advocate frequent and regular releases of the software to be made to the customer during the software development. Starting with the release of basic or core functionalities of the software, more complete functionalities are made available to the customer every couple of weeks . Hence effective Release Management has become important.
- **Risk Management:** In modern software development practices. Effective risk management is considered very important to the success of a project. A risk is any negative situation that may arise as the project progresses and may threaten the success of the

project. Risk Management involves identification of risks, assessment of the impacts of various risks, prioritization of the risks and preparation of risk-containment plans.

- **Scope Management:** Modern software development encourages customer to come up with change requests. While accepting the requests, three critical project parameters: scope , schedule and project cost are interdependent and related.

Additional Learning—1.12 MANAGEMENT

Case Study: Paul Duggan is the manager of a software development section. On Tuesday at 10.00 am he and his fellow section heads have a meeting with their group manager about the *staffing* requirements for the coming year. Paul has already drafted document ‘bidding’ for staff’. This is based on the work *planned* for his section for the next year. The document is *discussed* at the meeting. At 2.00 pm Paul has a meeting with his senior staff about an important project his section is undertaking. One of the software development staff has just had a road accident and will be in hospital for some time. It is decided that the project can be kept on schedule by transferring another team member from less urgent work to this project. A temporary replacement is to be brought in to do the less urgent work, but this might take a week or so to arrange. Paul has to phone both the personnel manager about getting a replacement and the user for whom the less urgent work is being done explaining why it is likely to be delayed. Identify which of the eight management responsibilities listed above Paul was responding to at different points during his day.

Project Planning: In the project initiation stage, an initial plan is made. As the project start, the project is monitored and controlled to proceed as per the plan. But, the initial plan is refined from time to time to factor in additional details and constraints about the project become available.

Based on the details of Paul Duggan's day, we can map his activities to the eight management responsibilities. The typical management responsibilities include:

1. **Planning:** Setting objectives and deciding on the actions needed to achieve them.
2. **Organizing:** Arranging tasks, people, and other resources to accomplish the work.
3. **Staffing:** Recruiting, selecting, training, and developing employees.
4. **Directing:** Leading, motivating, and communicating with employees.
5. **Controlling:** Monitoring and evaluating performance.
6. **Coordinating:** Ensuring all parts of the organization are working together towards common goals.
7. **Reporting:** Keeping all stakeholders informed.
8. **Budgeting:** Planning and controlling financial resources.

Let's analyze Paul's day:

1. **Drafting the document ‘bidding’ for staff:**
 - **Planning:** Paul is planning the staffing needs for the next year based on the upcoming work for his section.
2. **10:00 am meeting with fellow section heads and group manager:**
 - **Coordinating:** Paul is coordinating with other section heads and the group manager to ensure that the staffing requirements align with the overall needs of the organization.

- **Reporting:** He is discussing and providing information about his staffing plan.
- 3. **2:00 pm meeting with senior staff about an important project:**
 - **Directing:** Paul is leading and discussing how to manage the project, especially in light of the recent accident.
 - **Controlling:** He is ensuring the project stays on schedule despite the unforeseen incident.
- 4. **Deciding on transferring another team member:**
 - **Organizing:** Paul is organizing his team's workload to keep the important project on track by reallocating resources.
 - **Staffing:** This also involves staffing decisions, as he needs to bring in a temporary replacement.
- 5. **Phoning the personnel manager about getting a replacement:**
 - **Staffing:** Paul is working on staffing by arranging for a temporary replacement.
- 6. **Phoning the user about the delay in less urgent work:**
 - **Reporting:** He is informing the user about the situation and the expected delays.

In summary:

- **Planning:** Drafting the document 'bidding' for staff.
- **Coordinating:** 10:00 am meeting with section heads and group manager.
- **Reporting:** 10:00 am meeting and phoning the user about delays.
- **Directing:** 2:00 pm meeting with senior staff.
- **Controlling:** 2:00 pm meeting with senior staff.
- **Organizing:** Deciding on transferring a team member.
- **Staffing:** Phoning the personnel manager and deciding on a temporary replacement.